Software Engineering Project Assignment

Lois Dara Sanglap, Dustin Dwayne Diaz, Justin D. Grumal

Manukau Institute of Technology

502.633 Software Engineering

Dr. Ahmad Wedyan

02/06/2026

1.0 Project Proposal **4**
1.1 Introduction **4**
1.2 System Requirements **4**
1.2.1 Functional Requirements 4
1.2.1.1 Customer-End 4
1.2.1.2 Admin-End 5
1.2.2 Non-Fuctional Requirements 5
1.3 Software Architecture Patterns & Styles **6**
1.3.1 Architecture Style 1: MVC 6
1.3.2 Architecture Style 2: CSM 6
1.3.3 Trade-offs Limitation 7
1.3.4 Chosen Architecture Style: 8
1.3.5 Diagrams 9
1.3.5.2 Sequence Diagram 10
1.3.5.3 ERD 11
1.4 Design Pattern **11**
1.4.1 Pattern 1: Builder 11
1.4.2 Pattern 2: Observer 11
1.4.3 Pattern 3: Singleton 12
1.4.4 Pattern 4: Adapter 12
1.5 Software Quality Attributes **12**
1.5.1 Performance 12
1.5.2 Maintainability 13
1.5.3 Security 13
2.0 System Development and Implementation **14**
2.1 Software Development Process **14**
2.1.1 Prototype 16
2.1.1.2 Member End 17
2.1.1.3 Non-Member End 18
2.1.2 Adherence to Software Architecture - CSM 19
2.1.3 Adherence to Design Patterns 25
2.1.3.1 Builder Pattern 25
2.1.3.2 Observer Pattern 28
2.1.3.3 Singleton Pattern 31
2.1.3.4 Adapter Pattern 32
2.2 Core Functionalities **40**
2.2.1 Authentication and Authorisation (Admin & Customer Login) 40
2.2.2 Primary Functionalities 42
2.2.2.1 Customer Registration 43
2.2.2.2 Booking Class 50
2.2.3 Integration of Software Quality Attributes 55
2.2.3.1 Security 55
2.2.3.2 Performance 60
2.2.3.3 Maintainability 61

2.3 Code Quality **62**
2.3.1 Folder Structure 62
2.3.2 Modularity 64
2.3.2.1 Address.java 64
2.3.2.2 Profile.java 65
2.3.2.3 Security.java 65
2.3.3 Comments 66
3.0 Software Quality Attributes Evaluation and Test Reports **68**
3.1 Unit Testing 68
3.1.1 Customer Class Builder Success 68
3.1.2 Customer Class Builder Fail 69
3.1.3 Customer Class BMR Success 70
3.1.4 Customer Class TDEE Success 70
3.1.5 GymClass Class Builder Success 71
3.1.6 GymClass Class getIntensity Intermediate 73
3.1.7 GymClass Class getIntensity Athlete 73
3.1.8 GymClass Class getIntensity Beginner 74
3.1.9 Trainer Class Builder Success 75
3.1.10 User Class Test Success 77
3.1.11 Database loginUser Success 77
3.1.12 Database getUser Success 78
3.2 Integration Testing 79
3.2.1 Universal Email Verification 79
3.2.2 Booking Class Limit 81
3.2.3 Controller Polishment 82
3.3 Software Quality Attribute 83
3.3.1 Maintainability Evaluation 83
3.3.2 Performance Evaluation 88
3.3.3 Security Evaluation 90
4.0 Software Metrics Collection and Analysis **92**
4.1 Lines of Codes 92
4.1.1 Analysis and Interpretation 92
4.2 Maintainability Index 94
4.2.1 Analysis and Interpretation 95
4.3 Cyclomatic Complexity 96
4.3.1 Analysis and Interpretation 97
Bibliography **101**

1.0 Project Proposal

1.1 Introduction

The purpose of this document is to record the rationale for implementing certain software architecture and design patterns into our project, an Android application for a fictional gym named Unyielding Might (UM). Additionally, this document will also include how unit testing was conducted, ensuring our software meets the functional and non-functional requirements defined in section 1.0.

To give a brief overview of the business, UM is a gym equipped with all the amenities necessary to improve one's health; however, unlike other gyms, UM also conducts classes for both UM members (included in their membership) and non-members (requires payment).

UM's classes are led by qualified professionals and range from circuit training for strength and conditioning to Pilates for flexibility. But due to a low headcount, classes were deemed as an unsustainable business model; therefore, we have developed an Android mobile application to reduce the friction required to book (Customer POV) and manage (Admin POV) a class.

1.2 System Requirements

1.2.1 Functional Requirements

We reviewed UM's business plan and consulted with the fictional client to determine which functions are necessary to implement, allowing our team to define the scope of the project.

1.2.1.1 Customer-End

From the customer's point-of-view, UM's application should be capable of:

1. **Registration** - "The system shall successfully authenticate the user's attempt of registration whenever the user signs in or register so that the app can be accessed."
2. **Booking** - "The system shall be able to manage booking/s when the user attempts to book one or multiple classes so that conflicting classes cannot be booked."

3. **Searching** - “The system shall allow users to search classes by class name or trainer name.”
4. **Notification** - “The system shall notify customers of upcoming and booked class/es when the date approaches so that customers do not miss seeing or attending it.”
5. **Payment** - “The system shall possess a gateway for paying memberships and classes when the user is a non-membership holder so that customers have the liberty to commit to a membership.”

1.2.1.2 Admin-End

From the admin’s point-of-view, UM’s application should be capable of:

1. **Customer management** - The system shall allow admins to view, add, edit, and delete customers.
2. **Booking management** - The system shall allow admins to view, add, edit, and delete bookings made by the customers.
3. **Class management** - The system shall allow admins to view, add, edit, and delete class status.

1.2.2 Non-Fuctional Requirements

1. **Security** - The system shall implement robust security measurements such as email verification and data encryption remediate app vulnerability.
2. **Reliability** - The system shall maintain stability, adapt to user base scale, and handle load without crashes.
3. **Performance** - The system shall ensure high responsiveness by maintaining a maximum response time of 150ms or less for all user-initiated events.
4. **Usability** - Users should be able to successfully navigate through menus upon first interaction with the application.

1.3 Software Architecture Patterns & Styles

1.3.1 Architecture Style 1: MVC

Model View Controller (MVC) is a viable architecture to adopt for UM's mobile application due to each module having a distinct purpose; thus, not interfering with each other, allowing for an organised development lifecycle (GeeksforGeeks, 2025). MVC consists of three essential components, model, view, and controller (GeeksforGeeks, 2025).

- Model - The model layer manages the business logic and communicates with the database.
- View - The view layer is responsible for the presentation of the system.
- Controller - The controller acts as a middleman between the layers, where an input is received from the view layer which is subsequently processed by the model, generating a final output to be shown back in the view layer.

A distinct advantage of the MVC architecture is the clear separation between layers as aforementioned. This allows for easier testing as each layer can be evaluated at a time before combining them. Additionally, if the scope of the application increases, changes made to one layer of the software will minimally affect the others.

The IDE, Android Studio, innately supports this architecture as the "view" layer is essentially the "res" folder, containing all xml files responsible for the appearance of the application. Furthermore, Android Studio supports Java, allowing us to create classes and methods to handle the business logic of the application.

1.3.2 Architecture Style 2: CSM

Client-Server Model (CSM) is a type of network architecture that distributes application structures by dividing tasks between clients and servers (GeeksforGeeks, 2019). Applying CSM to UM's mobile booking application is an excellent choice, as this is the industry standard architecture for real-time booking systems.

The core components of CSM architecture are:

1. Client (Requester) - Sends requests over the network to the server using UM's application, such as requesting to book a class.

2. Server (Backend System) - Receives and processes requests, stores all booking resources, manages client data, prices, and availability, and responds to all client requests. The server then sends the requested information back to the clients.
3. Network - The infrastructure that enables communication between the client and server.
4. Request-Response Protocol - The client sends a request, and the server responds accordingly (e.g., sending a confirmation notification or email).

There are three variations of CSM, which are classified by the number of layers:

- Tier 1 - The client, server, and database all exist on the same machine.
- Tier 2 - The client directly interacts with the database server, typically used for simple applications.
- Tier 3 - A middle layer is introduced between the client and server to add business logic, enhance scalability, and improve security. This approach is commonly used in web applications.

For UM's application, the Tier 2 approach is the most suitable for our project scope, where the client directly interacts with the database layer. The first tier would be the mobile application developed in Android Studio, serving as the user interface and input collection layer. The second tier would be the database management system, responsible for storing, retrieving and securing the data. This tier will have the most of the business logic such as calculations, rules and processing.

1.3.3 Trade-offs Limitation

Criteria	**MVC**	**CSM**
Modifiability	Each layers are easily modifiable with a minimal impact on others	Modifying a business logic may require updates across different tiers

Scalability	User's device can be limited to the processing capacity	Different tiers can be independently scaled which can handle increased user demands
Testability	Each layers can be tested separately before integrating	Testing must be integrated across tiers and increase complex testing
Performability	Complex operation may degrade the device hardware when processing	Heavy processing tasks are offloaded to the server, keeping the client lightweight and responsive
Security Boundaries	All logic run on the client, which means data handling is less isolated	Database is separated from the client, credentials are never exposed
Deployment Complexity	Android application is deployed with no additional infrastructure	API server and database requires deployment and configuration separately

1.3.4 Chosen Architecture Style:

The Client-Server Model (Tier 2) was selected as the best architectural style for UM's mobile Android booking application. The system infrastructure consists of mobile clients and a database. This reflects the two-tier architecture of CSM, making it the most accurate and appropriate representation of UM system design.

Based on the criteria outlined in the table above:

Security - Two-tier will require careful management of database access control, by handling strict user-level permissions within the Database Management System. Ensuring admin and customer log in are separate.

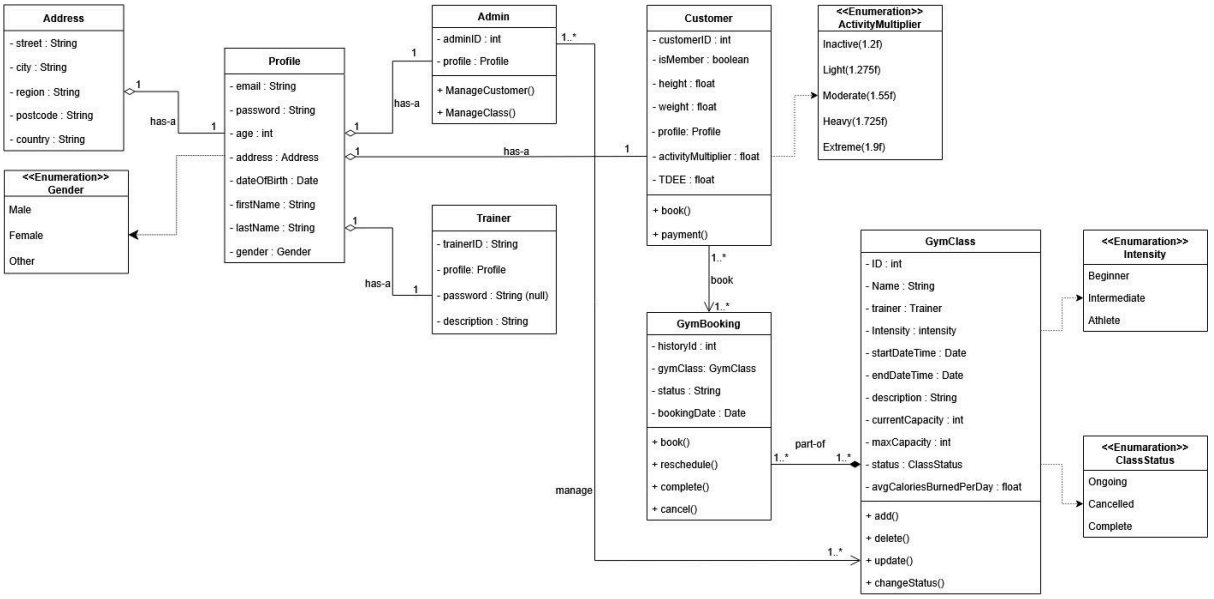
Performability - The client side will always remain responsive because heavy processing is offloaded to the database. In contrast, under the MVC pattern, processing occurs on the device itself, and some clients may not be able to handle complex processing if their device hardware is insufficient.

Scalability - Each tier can scale independently, allowing the application to accommodate a growing user base.

While the CSM serves as the primary architecture that aligns with the system requirements, the Model-View-Controller architecture is also applied within the project to organise the internal structure for the client. The “model” handles the API service class that will be communicating to the Tier 2 backend, verifying the logic and separating the presentation layer. The “view” layer corresponds to the xml files found in the “res” directory, responsible for the user interface. The “controller” layer is implemented through activities.java files as they act as an intermediary between views and models. Combining CSM and MVC ensures the UM application benefits from CSM’s security, scalability, and performability at the system level, while MVC provides a clean, testable and maintainable internal structure within the client's device.

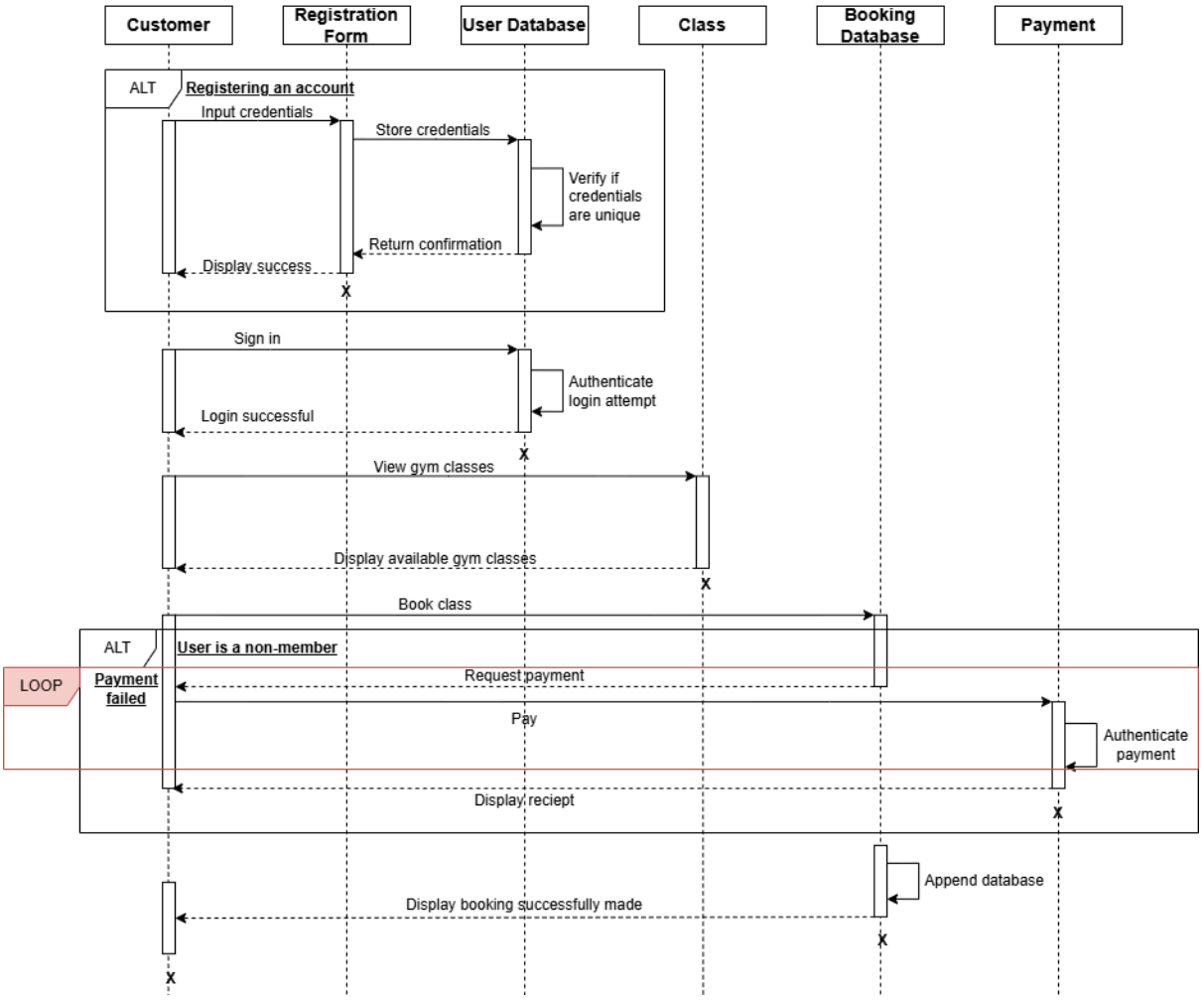
1.3.5 Diagrams

1.3.5.1 Class Diagram



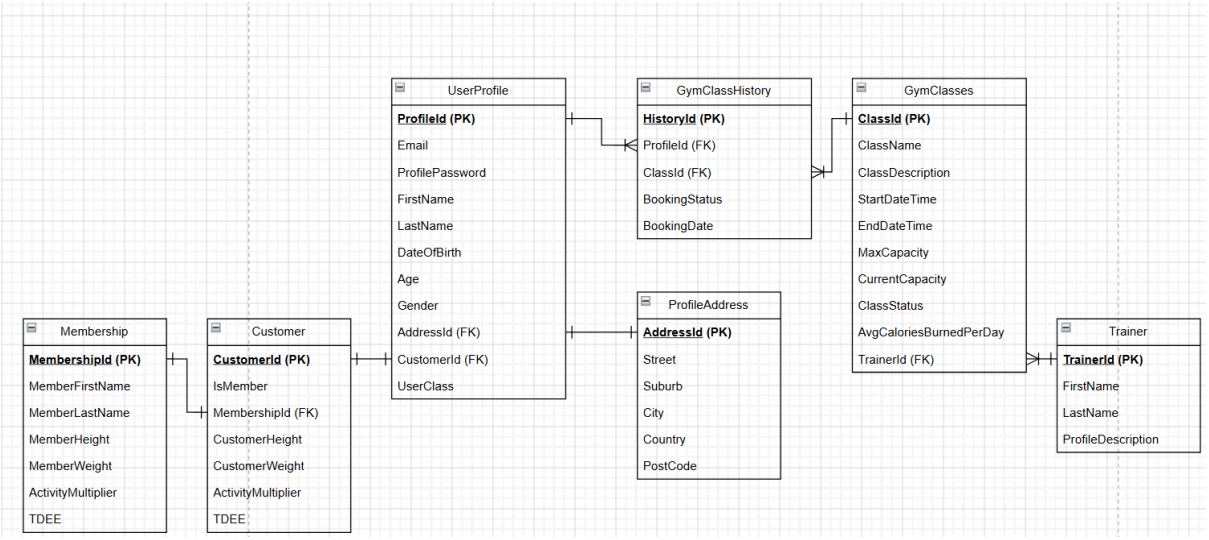
Created in Draw.io

1.3.5.2 Sequence Diagram



Created in Draw.io

1.3.5.3 ERD



Created in Draw.io

1.4 Design Pattern

1.4.1 Pattern 1: Builder

Builder is a creational design pattern that allows for more flexibility in the creation of complex objects due to its step-by-step approach (GeeksforGeeks, 2026). Therefore, a builder pattern can be implemented from the admin's POV when creating gym classes.

The rationality behind builder pattern is because of the variance between gym classes. UM, as a gym, offers a variety of different classes and thus has distinct attributes like date & time, description, and trainer.

1.4.2 Pattern 2: Observer

An observer design is a behavioral pattern where a state change in an object will automatically update other instances that have a one-to-many relationship with it. The one who triggers the state change are the subject while the one who reacts to the changes are the observers (GeeksforGeeks, 2026).

Observer design is perfect for implementing UM's booking feature as it creates an event-driven environment where interactions from the users will automatically update attributes states for classes such as its status and capacity.

An example of a scenario for the system is when you use the app. A user books Yoga Class as there is still 1 more slot left. The system will then update the status and the capacity of the class to be “Full” and “25/25”. Other users who browse for classes will see that Yoga Class is full and they are unable to book it.

To implement the Observer in the system, a key feature to be present in the code is listening to events, and that is where the execution of the changes and notification occurs. Once a booking event is detected, the system immediately updates the class attributes and broadcasts these changes to all connected users, ensuring the UI and server stay in sync without manual intervention.

1.4.3 Pattern 3: Singleton

The singleton pattern ensures that a class only has one instance throughout the application and gives access to that instance (GeeksforGeeks, 2026). It will be applied for the database in the UM application, every time a class in the application calls the Database, it always gets the same object. Singleton will prevent that by reusing the same instance every time.

1.4.4 Pattern 4: Adapter

According to GeeksforGeeks (2026), an adapter is a structural design pattern that allows incompatible interfaces to collaborate. This pattern will be implemented to achieve a Recycler View in UM’s homepage, allowing our software to display multiple gym classes retrieved from UM’s database. The gym classes will be represented as cards, displaying details such as the name of the class, the trainer conducting it, the time when the class starts, etc. Moreover, a Recycler View will improve maintainability since it reuses the same layout of the card; therefore, if the card requires amendments, all the cards will reflect the changes.

1.5 Software Quality Attributes

1.5.1 Performance

Performance is a key quality attribute for UM's booking application, especially given the non-functional requirement that all user-triggered actions must respond within 150ms. The Client-Server Model helps achieve this by moving heavy tasks

like handling bookings, checking class availability, and processing payments to the database server. This keeps the client lightweight and responsive, no matter what device the user has.

Additionally, the Observer pattern improves performance by creating an event-driven system. When class details such as availability or capacity change, those updates are automatically sent to all connected users in real time. This removes the need for the client to constantly ask the server for updates, cutting down on unnecessary network calls and making sure users always see accurate class availability without the risk of double-booking. (**Note**: This feature has not been implemented to reduce scope; however, observer is still used to send confirmation emails whenever the customer books a class)

1.5.2 Maintainability

Maintainability is an important quality attribute because it allows the system to be updated and expanded easily as UM's business needs change over time. The Client-Server Model supports maintainability through its separation into tiers, meaning each tier can be changed or updated independently without affecting the others. For example, updates to the database structure can be made without having to redeploy the Android client app, which reduces the risk of disrupting the entire system during updates.

Additionally, the Builder pattern improves maintainability by offering a structured and flexible way to create class objects. Since UM offers many different gym classes, each with unique attributes like date, time, trainer, and description, adding a new class type only requires introducing new parameters to the existing builder. This avoids the need to rewrite or restructure the existing class creation logic, minimising the chance of unintended side effects on other parts of the system. As a result, the codebase stays organised and manageable as the application grows.

1.5.3 Security

Security is a fundamental quality attribute for UM's booking application because the system handles sensitive personal and payment information for both members and non members. The Client Server Model's two tier separation ensures that the database is handled in a strict user level permission within the Database

Management System, which significantly reduces the attack surface of the application. User credentials and personal details are protected through password hashing, making sure sensitive data remains unreadable even if a data breach occurs. This also keeps the data inaccessible to administrators, upholding the principle of least privilege.

Furthermore, the payment gateway used for non member pay per class transactions requires encrypted communication between tiers. This ensures that financial data is securely transmitted and never intercepted during the request response cycle. In addition, the separation of administrator and customer facing endpoints reduces the risk of privilege escalation. This prevents unauthorised users from accessing or manipulating administrative functions such as customer management, booking management, and class management. Together, these measures align with the system's non-functional security requirement of implementing robust security measures including email verification and data encryption to remediate application vulnerabilities. (**Note**: Payment has been cut to reduce scope but was intended for users without a membership)

2.0 System Development and Implementation

2.1 Software Development Process

The applied software development process for this project is an Agile framework called scrum. Scrum is widely used to help teams manage complex projects by dividing work into short, iterative periods known as sprints (Drumond, 2024). Choosing scrum will provide structured and manageable tasks for our project,l enabling us to produce high-quality software efficiently.

Three phases of scrum:

1. Planning phase - This phase focuses on planning the and preparation of the project
    - Requirement gathering
    - Software architecture
    - UML diagrams
    - Wireframes
    - Prototype

2. Sprint phase - Development processes in sprints. This phase features designing, coding, testing and reviewing.
    - Created project files
    - Installed frameworks
    - Created folders
    - Frontend development (Forms, Login, UI layout)
    - Backend development (Database, API handling, Login system)
    - Testing
3. Release phase - After development is completed final testing is going to be performed to fix remaining bugs, finishing the documentation, presentation and the release of the software.
    - Final testing
    - User feedback
    - UI improvements
    - Presentation
    - Launching

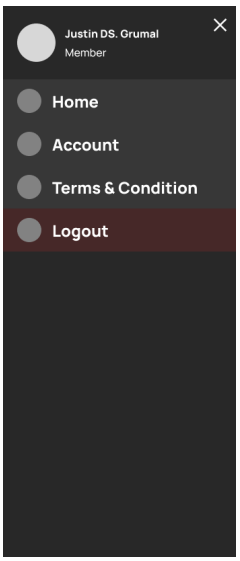
2.1.1 Prototype

Link to prototype activities:

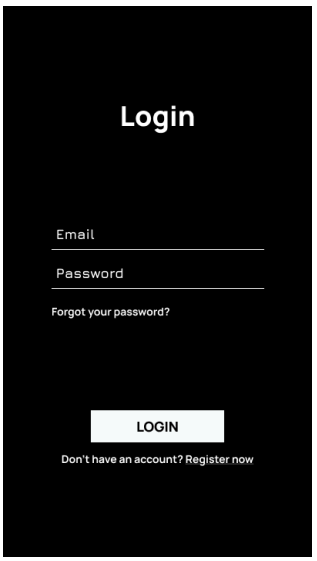
https://www.figma.com/design/wphUAzDg7ULmYFO1pOY5uX/Unyielding-Might-Prototype?node-id=0-1&t=O0VDEH0jYftV5bkj-1



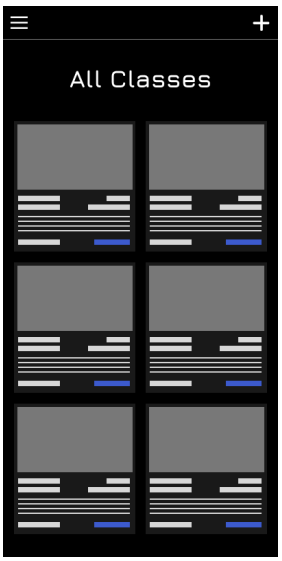
Welcome Page



Sidebar

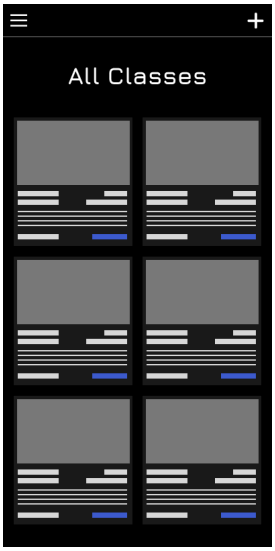


Login Page

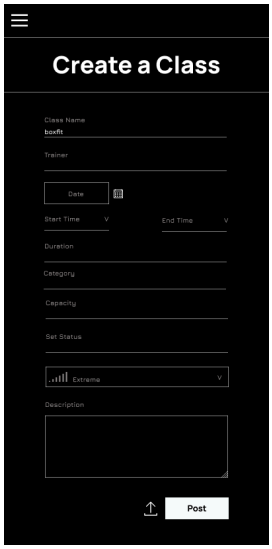


Homepage

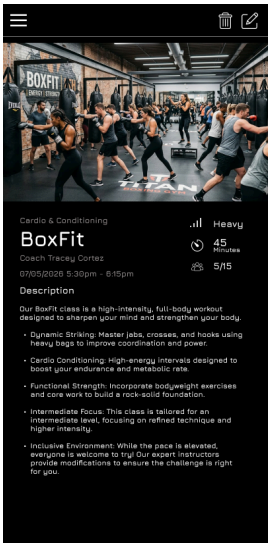
2.1.1.1 Admin End



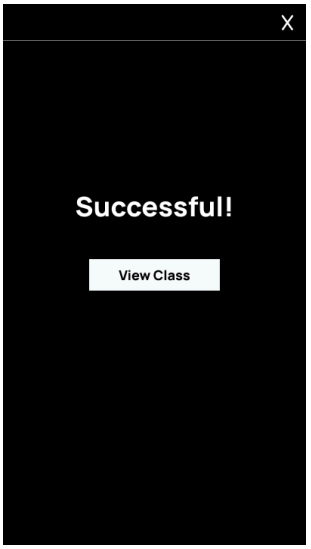
Home page



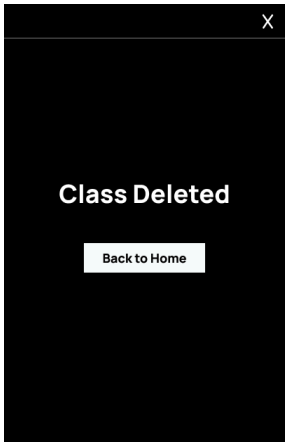
Create class page

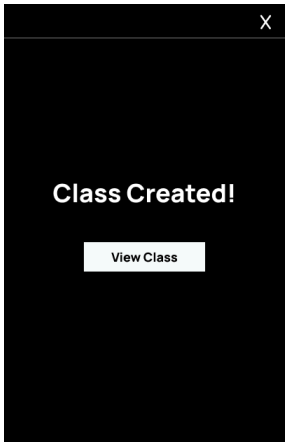


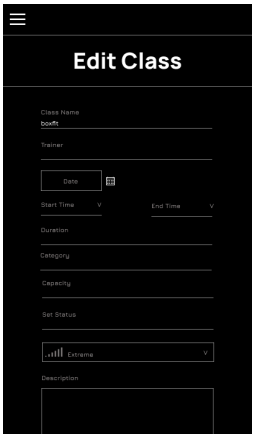
Class view page

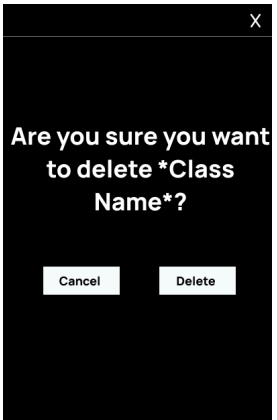


edit successful page



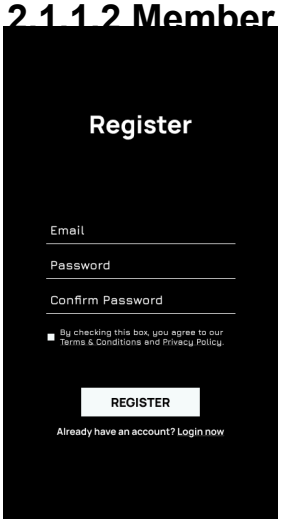


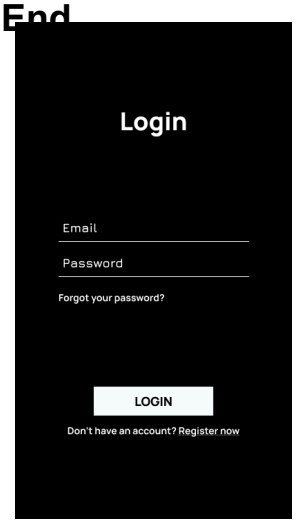


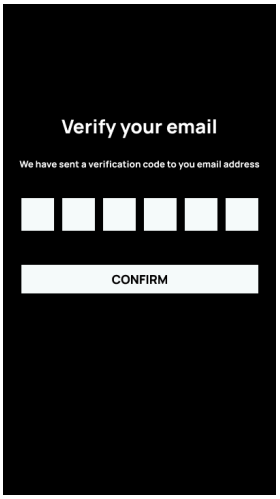


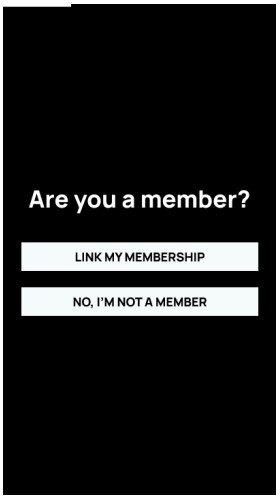
Class deleted page *Class created page* *Edit class* *delete page*

2.1.1.2 Member End

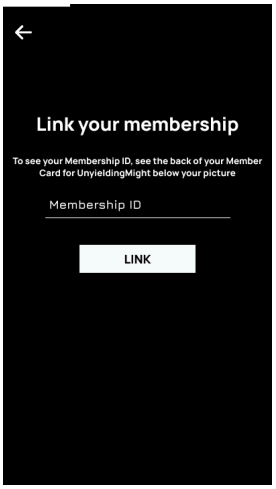


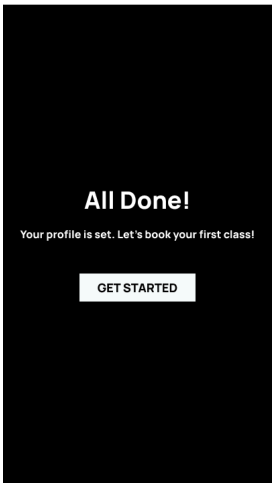


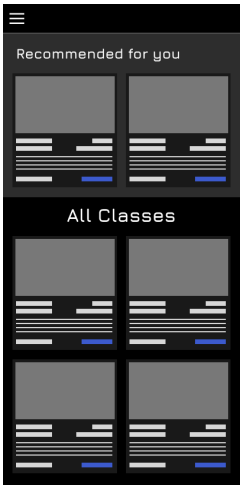


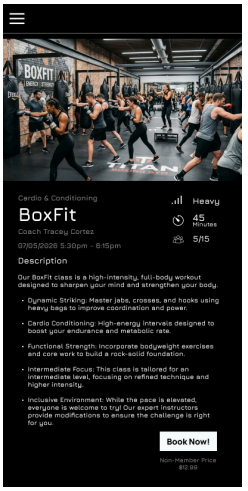


Register page *login page* *email verification page* *are you a member page*

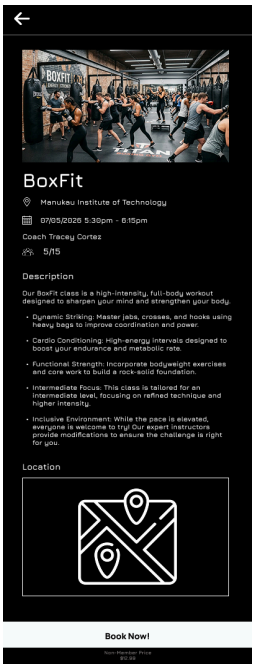


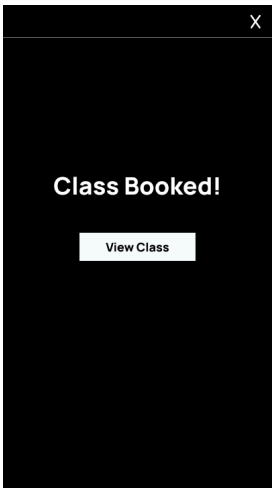






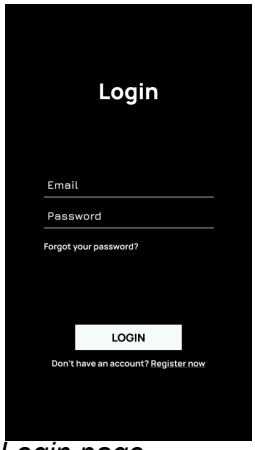
Link membership page *All done page* *Homepage* *Class page*

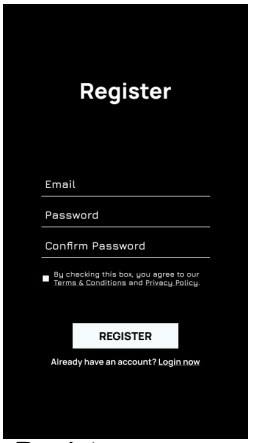


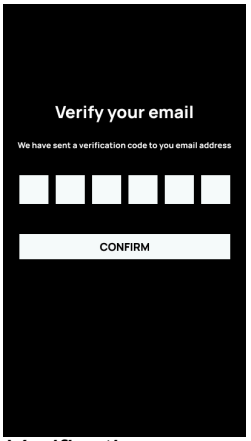


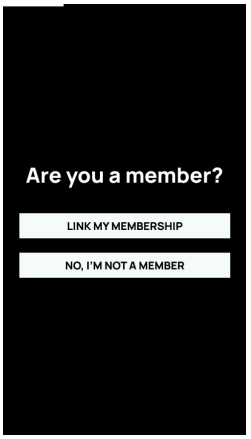
Book class page *Booked page*

2.1.1.3 Non-Member End



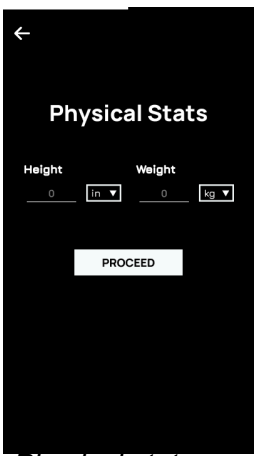


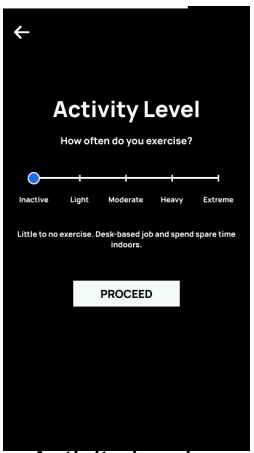


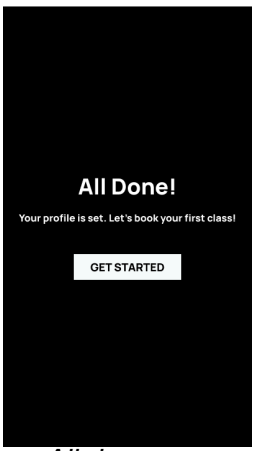


Login page *Register page* *Verification page* *are you member page*

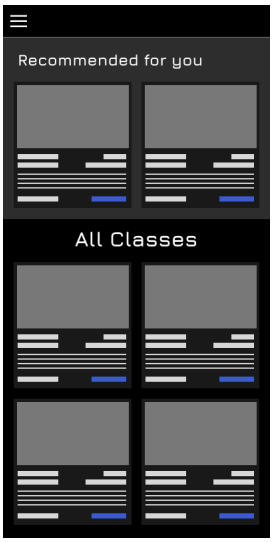




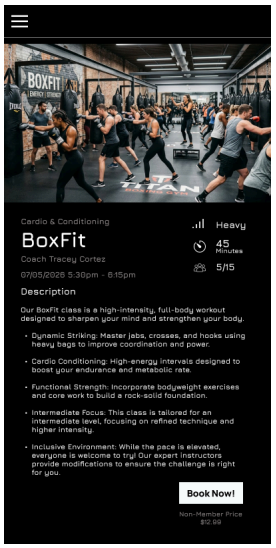




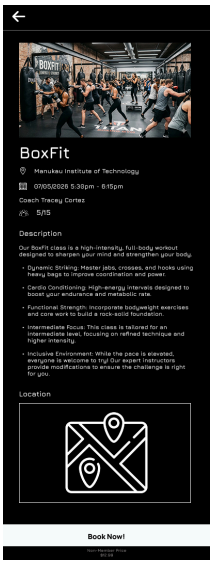
Profile page *Physical stat page* *Activity level page* *All done page*



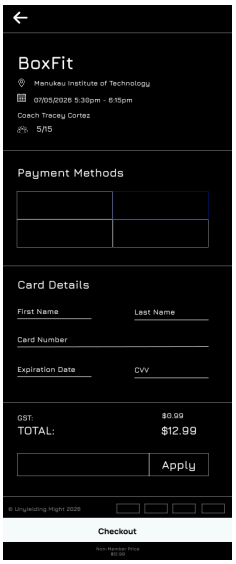
Home page



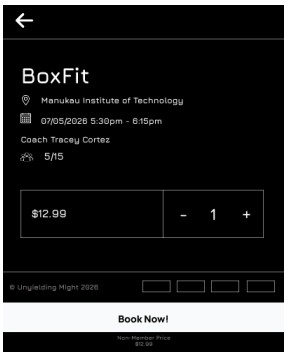
Class page



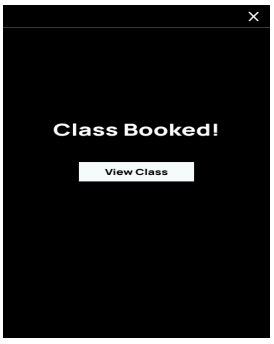
Class book page



Payment page



Book page

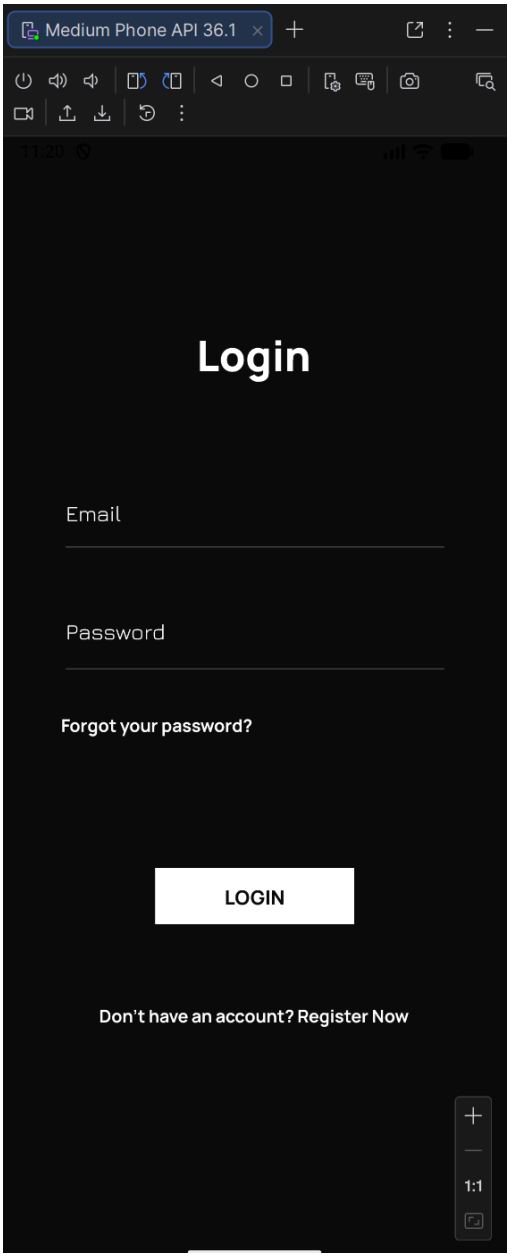


Booked page

2.1.2 Adherence to Software Architecture - CSM

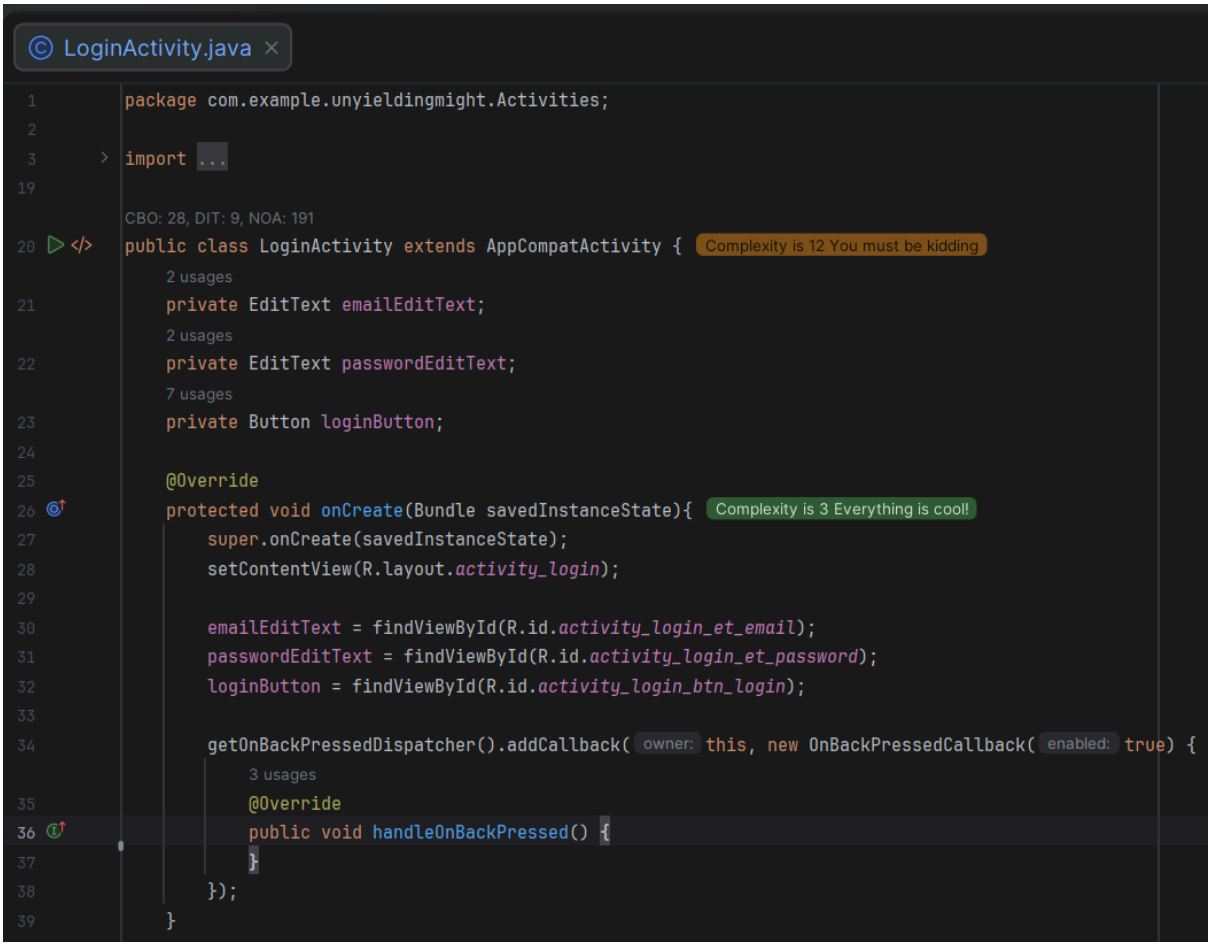
Our software complies with the chosen software architecture, 2-tier CSM. According to Geeks for Geeks (2026), a 2-tier CSM consists of a client layer (tier 1) and a server layer (tier 2). The client layer is the software itself, displaying the UI to the user, retrieving their inputs, and then processing it before requesting data from the server layer. In short, tier 1 is UM's Android application. However, tier 2 is the SQL database hosted by MIT's server.

Login will be taken as an example to demonstrate that 2-tier CSM has been implemented. Upon booting up the software, the user will be directed to the login activity, where the user is prompted to input their email and password as seen in the image below. Assuming the user is an already registered customer, the user can attempt to input their login credentials to login.



(Login activity)

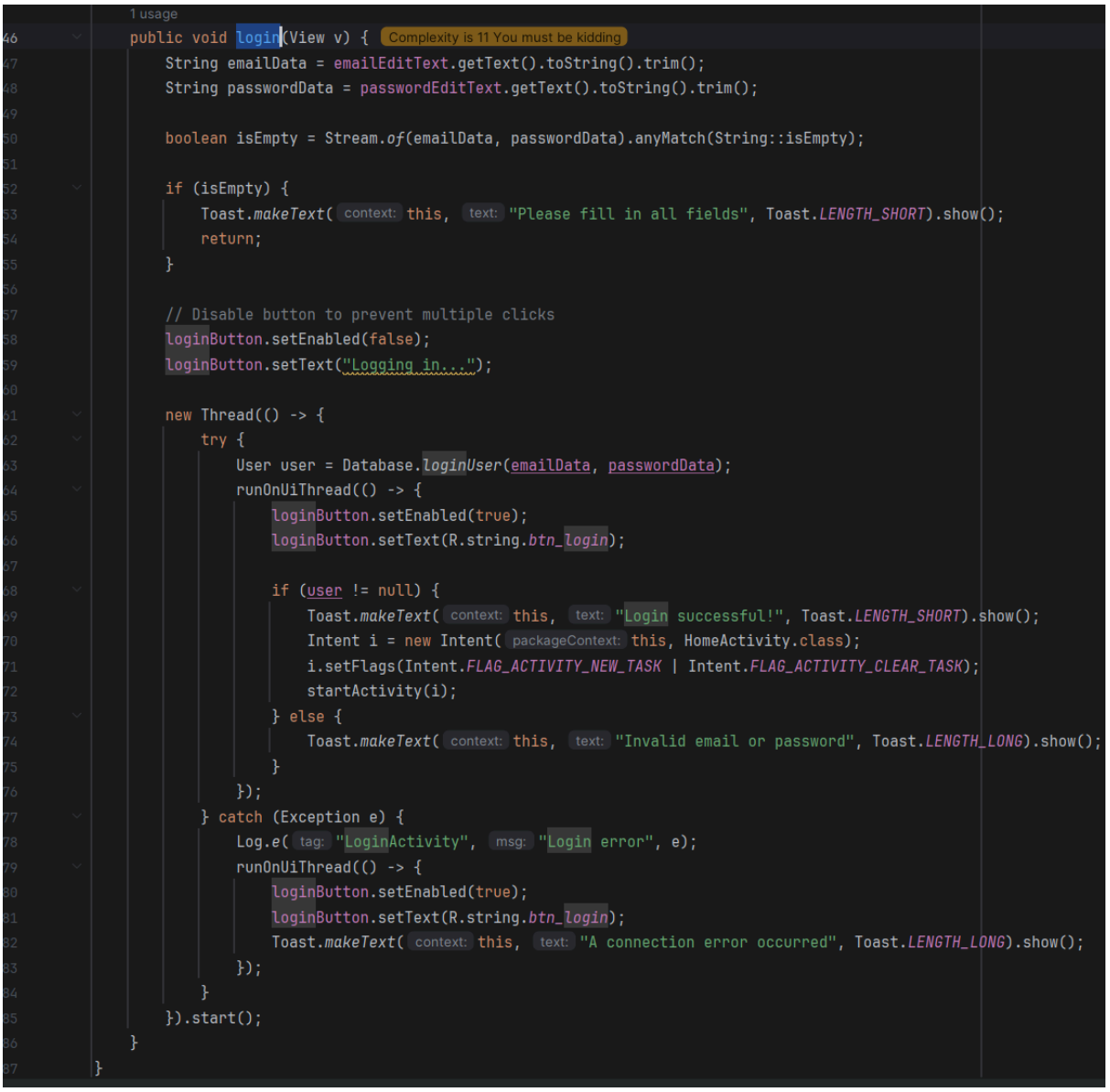
Using MVC (Model View Controller) to add additional context on how CSM is implemented, LoginAcitivity.java acts like a “Controller”, since it acts as a middle man between the “Models” (java classes with business logic) and “Views” (.xml files) (GeeksforGeeks, 2026). On lines 21-23, EditText, and Button are native UI classes required to be defined in order to identify whether a view is a button or an editable text box. EditText objects are able to store values; however, to store a value from the user, the view is first required to be found using “findViewById(R.id.___)”. It is the equivalent of document.getElementById() in JavaScript.



(LoginActivity.java - Part 1)

The function login() is called whenever the object “loginButton” is clicked. .OnClickListener() is not used to trigger the login() function since it is bound to the button in the .xml file (Check images below for reference). The data from the EditText Views are obtained using .getText() to the corresponding aforementioned objects (emailEditText and passwordEditText). If any of the Edit Text Views are empty, a toast will pop up, informing the user all fields must be filled.

Whenever static methods from Database.java are called, they are located within a new Thread() instead of the UI Thread. new Thread() is primarily used to prevent stutters and crashes in the client layer (tier 1) whenever a request to the database is made.



```
1 usage
public void login(View v) {  Complexity is 11 You must be kidding
    String emailData = emailEditText.getText().toString().trim();
    String passwordData = passwordEditText.getText().toString().trim();

    boolean isEmpty = Stream.of(emailData, passwordData).anyMatch(String::isEmpty);

    if (isEmpty) {
        Toast.makeText( context: this,  text: "Please fill in all fields", Toast.LENGTH_SHORT).show();
        return;
    }

    // Disable button to prevent multiple clicks
    loginButton.setEnabled(false);
    loginButton.setText("Logging in...");

    new Thread(() -> {
        try {
            User user = Database.loginUser(emailData, passwordData);
            runOnUiThread(() -> {
                loginButton.setEnabled(true);
                loginButton.setText(R.string.btn_login);

                if (user != null) {
                    Toast.makeText( context: this,  text: "Login successful!", Toast.LENGTH_SHORT).show();
                    Intent i = new Intent( packageContext: this, HomeActivity.class);
                    i.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
                    startActivity(i);
                } else {
                    Toast.makeText( context: this,  text: "Invalid email or password", Toast.LENGTH_LONG).show();
                }
            });
        } catch (Exception e) {
            Log.e( tag: "LoginActivity",  msg: "Login error", e);
            runOnUiThread(() -> {
                loginButton.setEnabled(true);
                loginButton.setText(R.string.btn_login);
                Toast.makeText( context: this,  text: "A connection error occurred", Toast.LENGTH_LONG).show();
            });
        }
    }).start();
}
}
```

(LoginActivty.java - Part 2)



(activity_login - onClick evidence)

The static Database method, loginUser(), requires 2 parameters, email and password. loginUser() calls another method getUser(), passing the parameters email and password. If the user is present within the database, an object of User is returned, enabling the user to login and start the new activity, HomeActivity.

```
public static User loginUser(String email, String password) {
    // Validates the user credential and returns the user
    User user = getUser(email, password);
    if (user != null) {
        getInstance().currentUser = user;
    }
    return user;
}
```

(Database.java - loginUser())

getUser() returns an object of User by executing a SQL query to find if a record of the user exists in the database by using a WHERE clause. .getConnection() is another method that can be found within Database.java to establish a connection between the client (tier 1) and the database (tier 2). If a connection exists, the following code will execute. PreparedStatement is a part of the JDBC Plugin class. It is used since the SQL query requires a parameter which is represented by a “?” (PreparedStatement (Java Platform SE 8), n.d.) . To provide the parameter into the SQL query, .setString() is used. “1” indicates the first instance where “?” is found and the second parameter is what “?” is being replaced with, which is email. To execute the SQL query, .executeQuery() is used. Since SELECT returns data, it is stored in an object ResultSet.

rs.next() returns a boolean if the SELECT query finds a record that contains the email inputted by the user. If a record is found, the password, which is hashed, will be compared if it is the same. Additionally, the role of the user is checked to see if the user is either a customer or an admin.

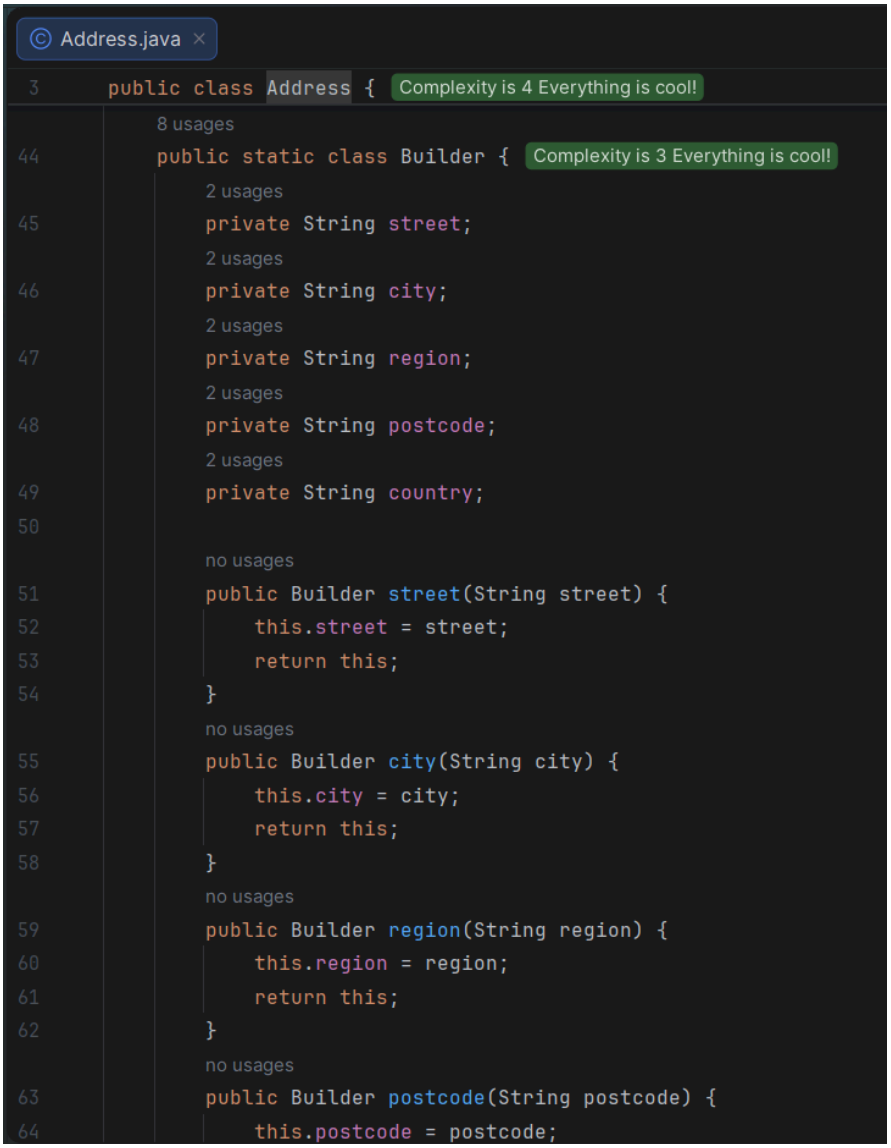
```
153     // Fetch the user from the database
154 @   public static User getUser(String email, String password) {  Complexity is 11 You must be kidding
155         String sql = "SELECT * FROM UserProfile WHERE Email = ?";
156         Connection conn = getConnection();
157         if (conn == null) return null;
158         try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
159             pstmt.setString( i: 1, email);
160             ResultSet rs = pstmt.executeQuery();
161             if (rs.next()) {
162                 String storedHash = rs.getString( s: "ProfilePassword");
163                 String inputHash = Security.hashData(password);
164
165                 // Check the hashed input and the hashed password stored in the database
166                 if (storedHash.equals(inputHash)) {
167                     String roleStr = rs.getString( s: "UserClass");
168                     User.Role role = (roleStr.equalsIgnoreCase( anotherString: "ADMIN"))
169                             ? User.Role.ADMIN : User.Role.CUSTOMER;
170
171                     // Create user class for the logged user
172                     return new User(
173                             rs.getInt( s: "ProfileId"),
174                             username: rs.getString( s: "FirstName") + " " + rs.getString( s: "LastName"),
175                             storedHash,
176                             rs.getString( s: "Email"),
177                             role
178                     );
179                 }
180             }
181         } catch (Exception e) { Log.e( tag: "DATABASE",  msg: "User Fetching Error: " + e.getMessage()); }
182         return null;
183     }
```

(Database.java - getUser)

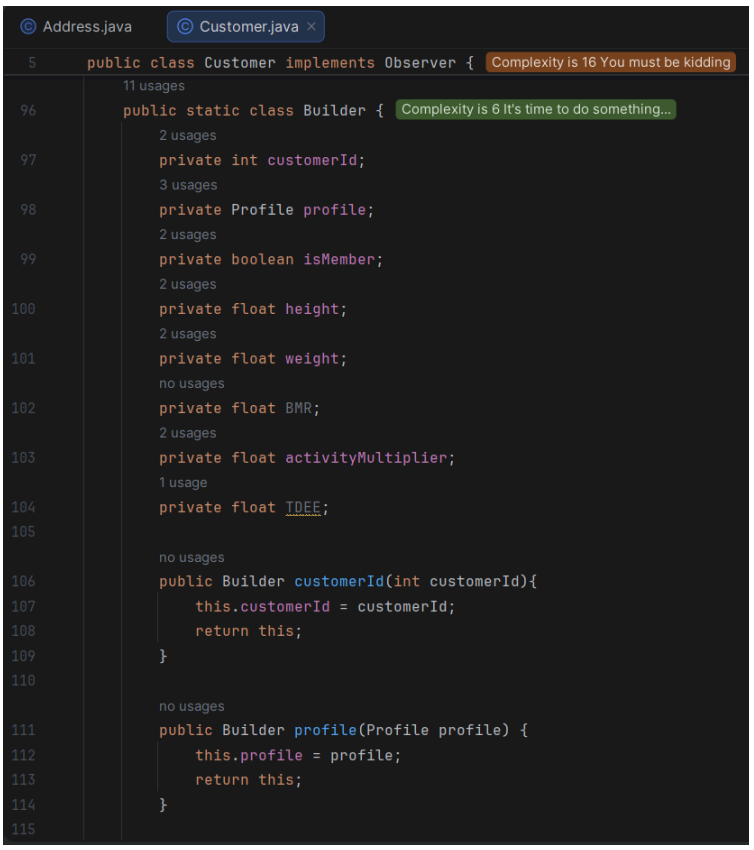
2.1.3 Adherence to Design Patterns

2.1.3.1 Builder Pattern

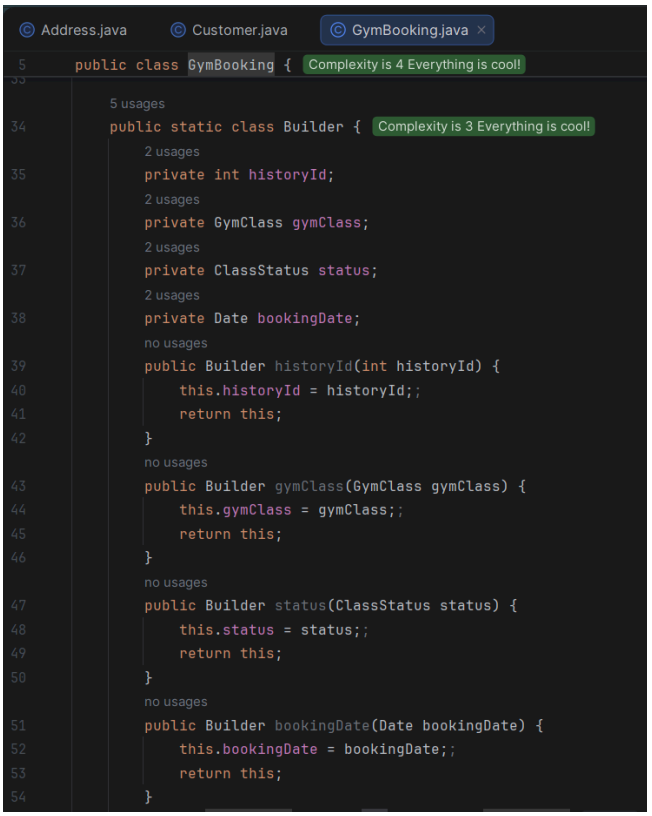
The builder pattern allows us to create an object step-by-step (RefactoringGuru, n.d.). This design pattern is implemented through the major class entities for the app which are Address, Customer, GymBooking, GymClass, Profile, and Trainer. These classes are used in the Database class to place the data from the query to its designated data for the class to be used in the app.



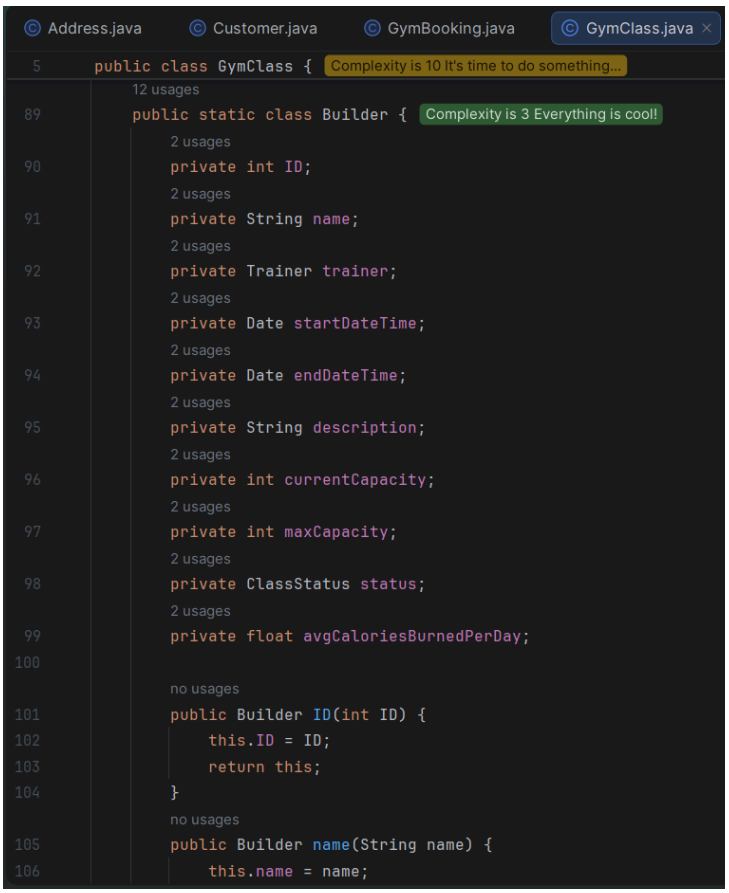
(Address.java)



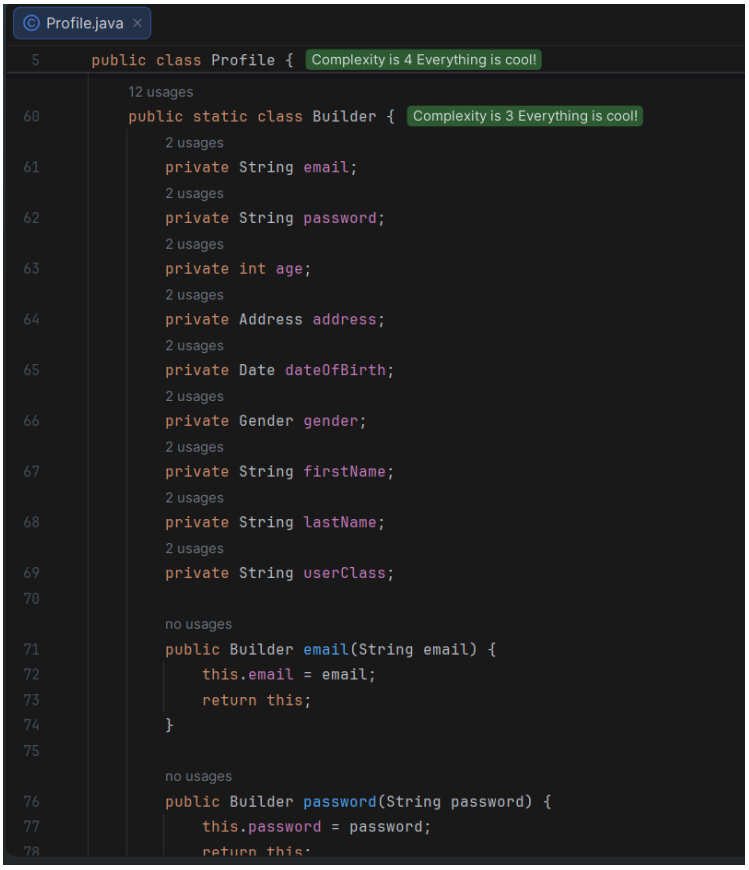
(Customer.java)



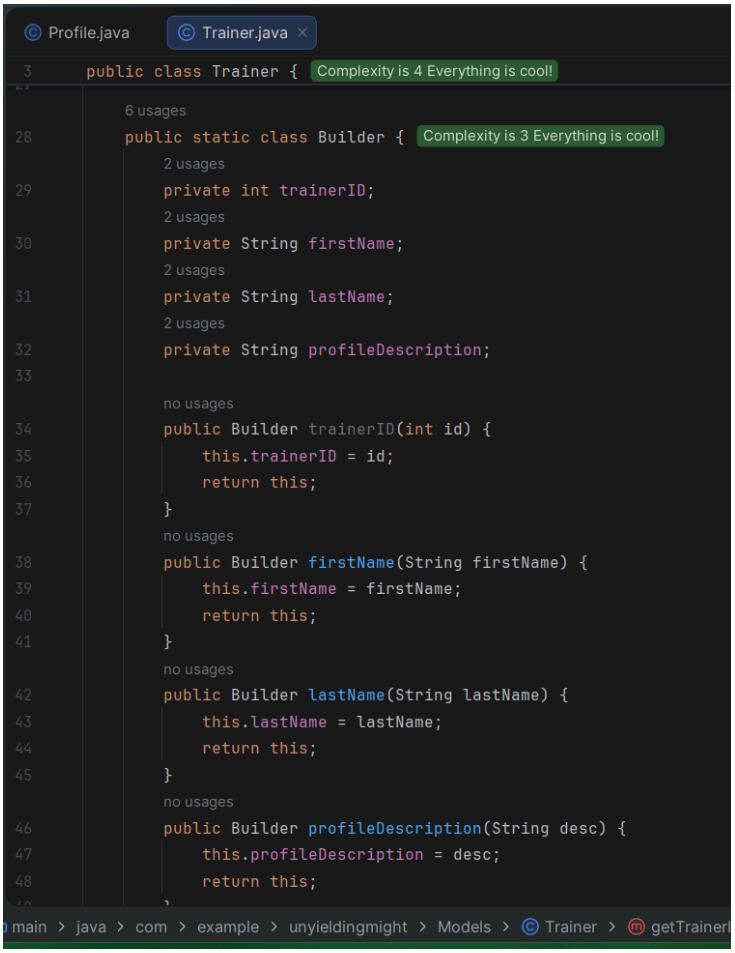
(GymBooking.java)



(GymClass.java)



(Profile.java)



(Trainer.java)

2.1.3.2 Observer Pattern

The observers are the ones that are getting the update from the subjects (RefactoringGuru, n.d.). In this case, for the newsletter feature which was never implemented but the function is already there; the observer pattern is implemented through the customer class where it acts as the subscribers to the newsletter and the subject is the service. This implementation was planned for the newsletter subscription feature for customers who are subscribed to be notified. In “NewsletterSubscriber.java” class, a function called “notifySpecificObserver” can be found in line 42, and is used in the database to send out a newsletter which for now is being used after a user books a class, they will receive an email of booking confirmation.

```java
public interface Observer {
    2 implementations
    void update();
}
```

(Observer.java)

```java
public interface Subject {
    1 implementation
    void add(Observer obs);
    no usages  1 implementation
    void remove(Observer obs);
    no usages  1 implementation
    void notifyObserver();
}
```

(Subject.java)

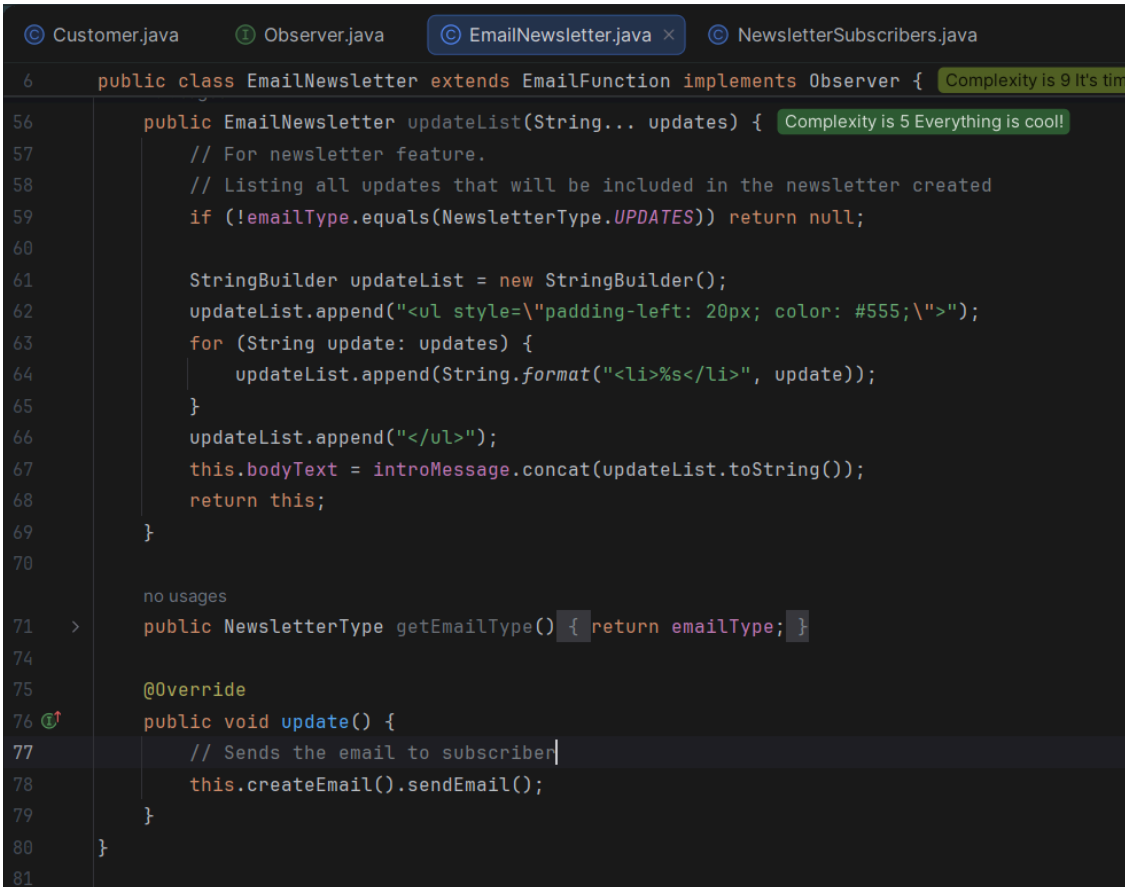
Customer.java

```java
public class Customer implements Observer {  Complexity is 16 You must be kidding
    @Override
    public void update() {
        NewsletterSubscribers subscribers = Database.getNewsletterSubscribers();
        NewsletterType updateType = subscribers.getLatestUpdateType();
        // Personalize the email
        EmailNewsletter emailNotification = new EmailNewsletter
        (updateType, profile.getFirstName());

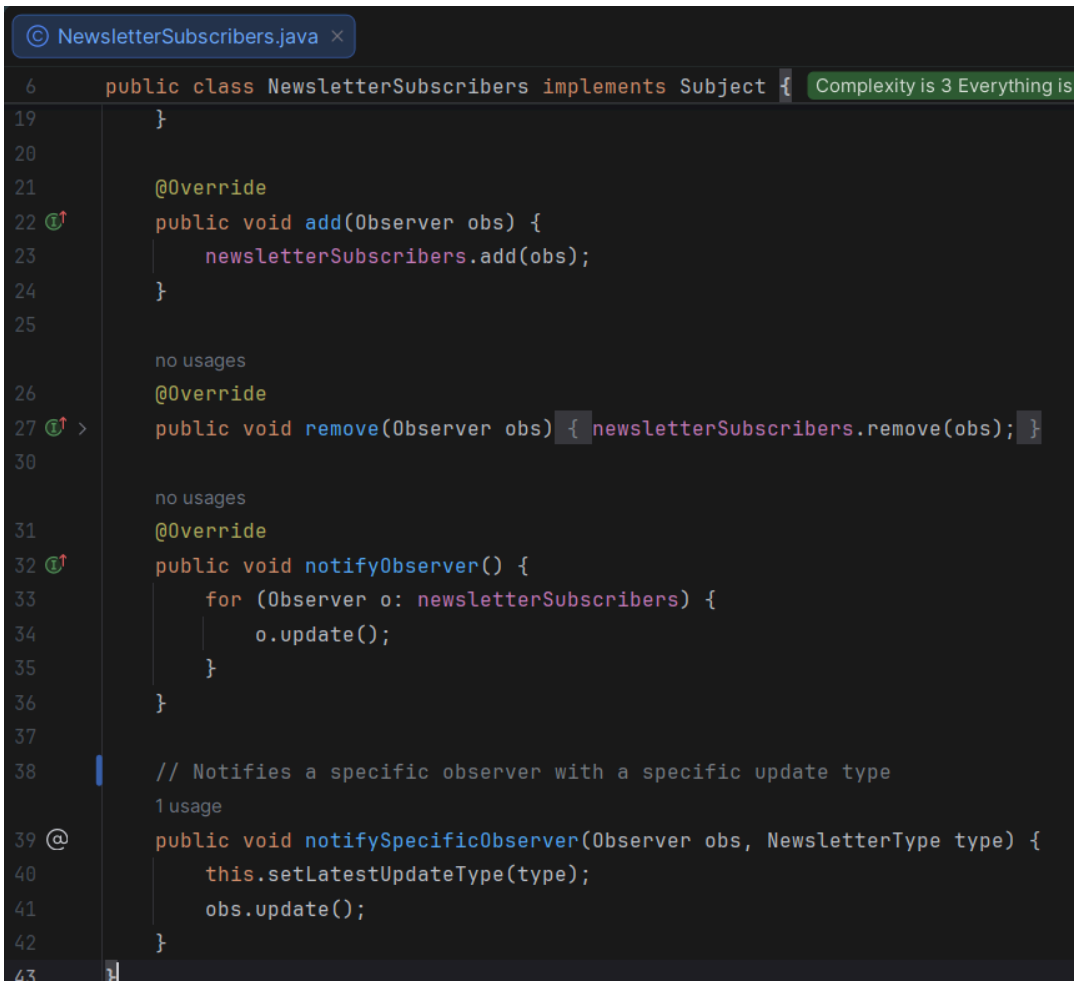
        emailNotification.setSender( senderName: "Dustin @ UnyieldingMight",
            BuildConfig.SMTP_USER).setReceiver(profile.getFirstName(),
            profile.getEmail()).createEmail().sendEmail();

        String result = "Mail Result for " + profile.getEmail() +
            ": "+ emailNotification.getResponseString();
        System.out.println(result);
    }
```

(Customer.java)



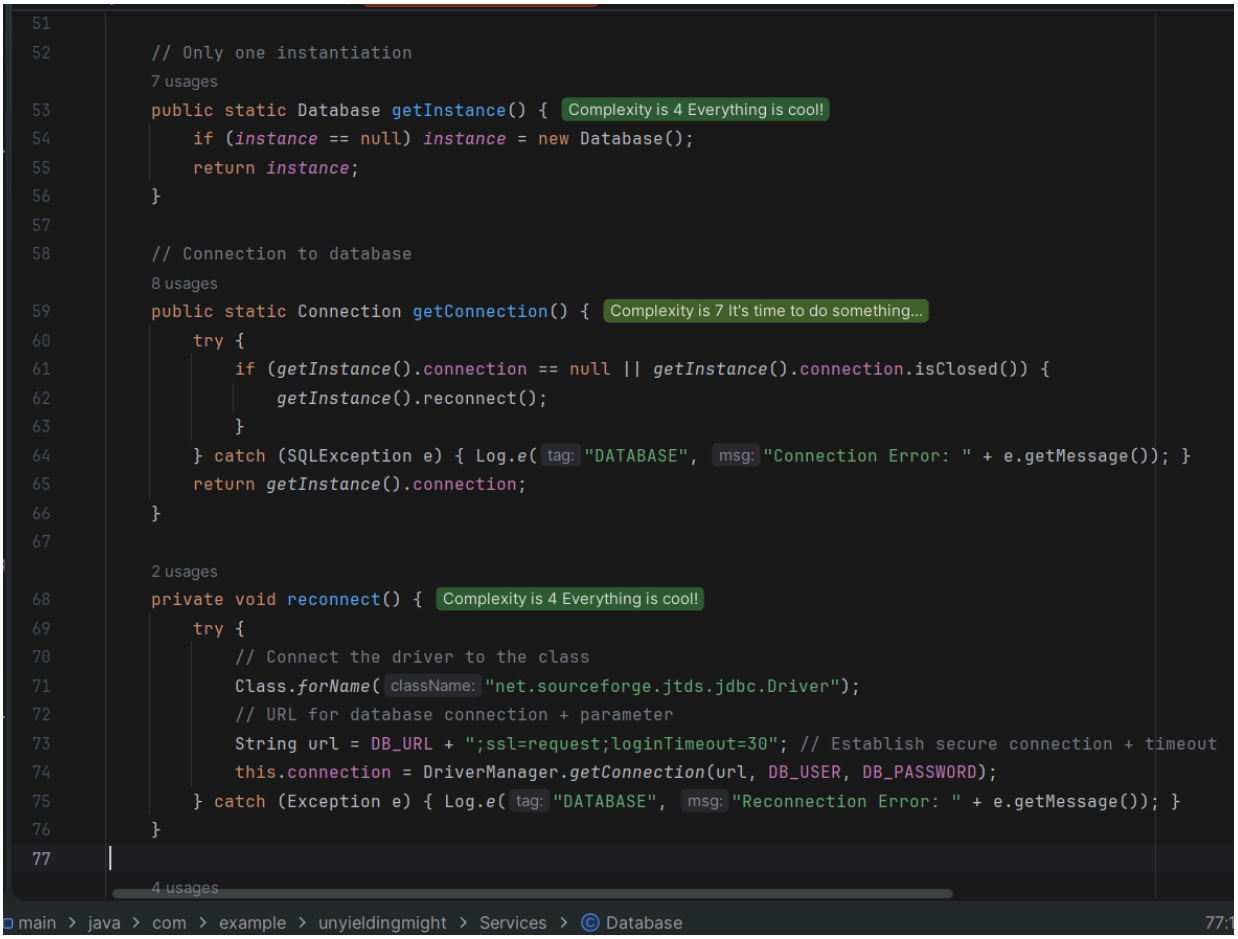
(EmailNewsletter.java)



(NewsletterSubscribers.java)

2.1.3.3 Singleton Pattern

Following the singleton pattern provides global context to the entire app when attributes and methods within the database are being accessed. This ensures data and performance consistency (RefactoringGuru, n.d.). Only the “Database” class follows the singleton pattern as shown below:



```
51
52      // Only one instantiation
        7 usages
53      public static Database getInstance() {  Complexity is 4 Everything is cool!
54          if (instance == null) instance = new Database();
55          return instance;
56      }
57
58      // Connection to database
        8 usages
59      public static Connection getConnection() {  Complexity is 7 It's time to do something...
60          try {
61              if (getInstance().connection == null || getInstance().connection.isClosed()) {
62                  getInstance().reconnect();
63              }
64          } catch (SQLException e) { Log.e( tag: "DATABASE",  msg: "Connection Error: " + e.getMessage()); }
65          return getInstance().connection;
66      }
67
        2 usages
68      private void reconnect() {  Complexity is 4 Everything is cool!
69          try {
70              // Connect the driver to the class
71              Class.forName( className: "net.sourceforge.jtds.jdbc.Driver");
72              // URL for database connection + parameter
73              String url = DB_URL + ";ssl=request;loginTimeout=30"; // Establish secure connection + timeout
74              this.connection = DriverManager.getConnection(url, DB_USER, DB_PASSWORD);
75          } catch (Exception e) { Log.e( tag: "DATABASE",  msg: "Reconnection Error: " + e.getMessage()); }
76      }
77
        4 usages
```

main > java > com > example > unyieldingmight > Services > Database 77:1

(Database.java)

2.1.3.4 Adapter Pattern

Implementation of an adapter pattern can be found in the folder, “Models” and "Activities", under the file name “Adapter.java”, “HomeActivity.java”, and “InfoActivity.java”. The role of Adapter.java is to implement a RecyclerView, where data from the database can be displayed dynamically on cards without the constraint of having to create distinct cards for each gym class from scratch (Android Developers, n.d.). The implementation of the adapter pattern can be seen in the collection of images below.

```
33
34      //    Wraps the view object inside of ViewHolder
35          @NonNull
36          @Override
37          public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
38              View view = LayoutInflater.from(parent.getContext()).inflate(R.layout.recycler_class, parent,  attachToRoot: false);
39              return new ViewHolder(view);
40          }
```

(Adapter.java - Part 1)

```
92          @Override
93          public int getItemCount() { return dataList != null ? dataList.size() : 0; }
96      //    Holds the view for each item in the recycler view
            4 usages
97          public static class ViewHolder extends RecyclerView.ViewHolder{
                2 usages
98              CardView recCard;
                2 usages
99              ImageView recImage;
                2 usages
100             TextView recTitle, recInstructor, recIntensity, recDate, recStartTime, recEndTime, recCurCap, recMaxCap, recDesc;
                1 usage
101             public ViewHolder(@NonNull View itemView) {
102                 super(itemView);
103                 recCard = itemView.findViewById(R.id.recycler_class_card);
104                 recImage = itemView.findViewById(R.id.recycler_class_iv_class);
105                 recTitle = itemView.findViewById(R.id.recycler_class_tv_title);
106                 recInstructor = itemView.findViewById(R.id.recycler_class_tv_instructor);
107                 recIntensity = itemView.findViewById(R.id.recycler_class_tv_intensity);
108                 recDate = itemView.findViewById(R.id.recycler_class_tv_date);
109                 recStartTime = itemView.findViewById(R.id.recycler_class_tv_startTime);
110                 recEndTime = itemView.findViewById(R.id.recycler_class_tv_endTime);
111                 recCurCap = itemView.findViewById(R.id.recycler_class_tv_currentCapacity);
112                 recMaxCap = itemView.findViewById(R.id.recycler_class_tv_maxCapacity);
113                 recDesc = itemView.findViewById(R.id.classDescription);
114             }
115         }
116
117     }
```

(Adapter.java - Part 2)

onCreateViewHolder() is a method that returns ViewHolder. The class “ViewHolder” can be found within the same Adapter.java file on line 93. ViewHolder is used to cache the references to where certain Views (Images, texts) are, preventing the software having to repeatedly perform the same action. In the image above, “findViewById()” is akin to the JavaScript equivalent, “document.getElementById()” (Easy Tuto, 2021).

The purpose of onCreateViewHolder() is to initialise ViewHolder and its Views (Android Developers, n.d.). It does this by first getting the context from the RecyclerView, then LayoutInflater reads the recycler_class.xml in order to identify the Views within the layout. The method then proceeds to wrap the View object into ViewHolder.

```java
//    Binds the data from <GymClass> to the views within RecyclerView
    @Override
    public void onBindViewHolder(@NonNull ViewHolder holder, int position) {  Complexity is 6 It's time to do something...
//        Gets the position of class within dataList and uses the position to retrieve an object of GymClass
        GymClass gymClass = dataList.get(position);
        SimpleDateFormat dateFormat = new SimpleDateFormat( pattern: "dd/MM/yyyy", Locale.getDefault());
        SimpleDateFormat timeFormat = new SimpleDateFormat( pattern: "HH:mm", Locale.getDefault());
//        Sets the image,texts, and values to the corresponding views within the holder
        holder.recImage.setImageResource(R.drawable.boxfit);
        holder.recTitle.setText(gymClass.getName());
        holder.recInstructor.setText(gymClass.getTrainer().getName());

        GymClass.Intensity intensity = gymClass.getIntensity(userTdee);
        holder.recIntensity.setText(intensity.name());

        holder.recDate.setText(dateFormat.format(gymClass.getStartDateTime()));
        holder.recStartTime.setText(timeFormat.format(gymClass.getStartDateTime()));
        holder.recEndTime.setText(timeFormat.format(gymClass.getEndDateTime()));

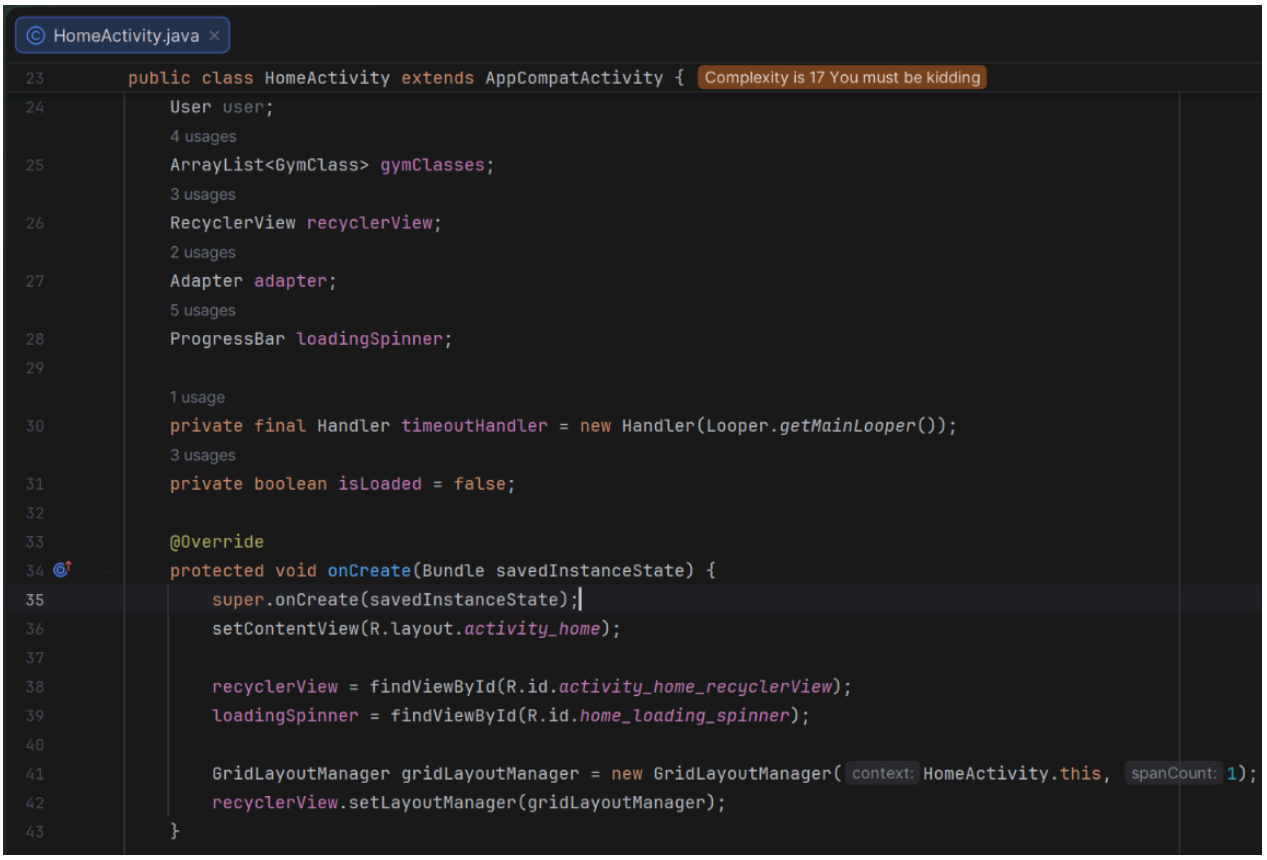
        holder.recDesc.setText(gymClass.getDescription());
        holder.recCurCap.setText(String.valueOf(gymClass.getCurrentCapacity()));
        holder.recMaxCap.setText(String.valueOf(gymClass.getMaxCapacity()));
//        Listens if a card gets clicked
        holder.recCard.setOnClickListener(new View.OnClickListener() {  Complexity is 5 Everything is cool!
            @Override
            public void onClick(View view) {  Complexity is 4 Everything is cool!
//                Gets the current position of the card
                int pos = holder.getAbsoluteAdapterPosition();
                if (pos == RecyclerView.NO_POSITION) return;
                GymClass current = dataList.get(pos);
//                New intent to start the activity InfoActivity
                Intent intent = new Intent(context, InfoActivity.class);
//                Transfers the class data to InfoActivity so it can be displayed
                intent.putExtra( name: "Image", R.drawable.boxfit);
                intent.putExtra( name: "Title", current.getName());
                intent.putExtra( name: "Instructor", current.getTrainer().getName());
                intent.putExtra( name: "Intensity", current.getIntensity(userTdee).name());
                intent.putExtra( name: "Date", dateFormat.format(current.getStartDateTime()));
                intent.putExtra( name: "StartTime", timeFormat.format(current.getStartDateTime()));
                intent.putExtra( name: "EndTime", timeFormat.format(current.getEndDateTime()));
                intent.putExtra( name: "CurCap", String.valueOf(current.getCurrentCapacity()));
                intent.putExtra( name: "MaxCap", String.valueOf(current.getMaxCapacity()));
                intent.putExtra( name: "Desc", current.getDescription());
                intent.putExtra( name: "AvgCalorie", current.getAvgCaloriesBurnedPerDay());
                intent.putExtra( name: "ClassId", current.getID());
                context.startActivity(intent);
            }
        });
    }
```

(Adapter.java - Part 3)

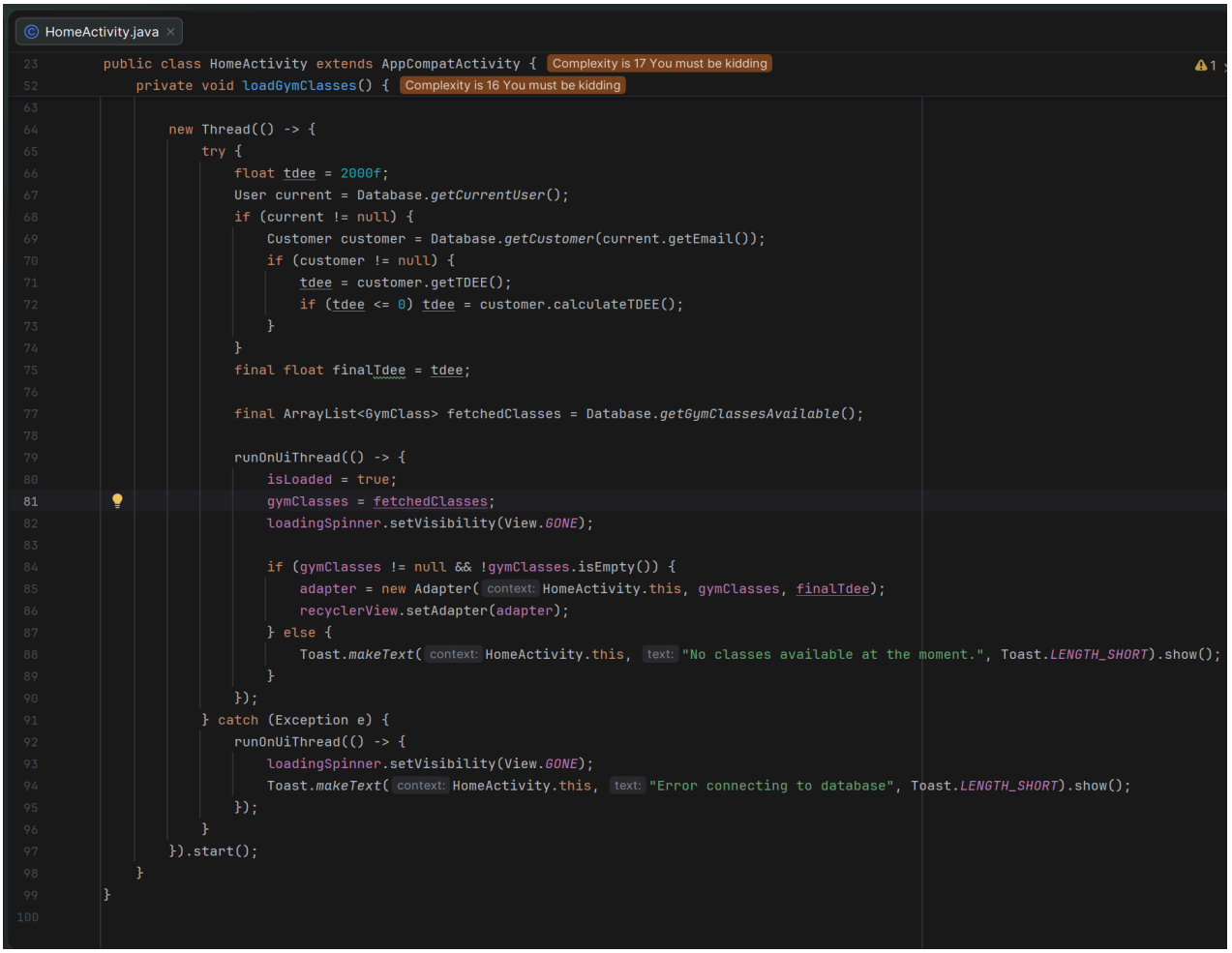
The method, onBindViewHolder(), “binds” the data from the database associated with the gym class to populate the ViewHolder and its Views, as seen in the image above. For instance, “holder.recTitle.setText(gymClass.getName)”. “recTitle”, was defined within the class ViewHolder as aforementioned, as it refers to

the Text View from recycler_class.xml with the id “recycler_class_tv_title”. “.setText” is similar to “.innerText” in JavaScript, where the content of the Text View is replaced with the title (name) of the gym class.

Moreover, when the Card View (recCard) is clicked, it calls a function to redirect the user to a new activity, Home. The function first gets the position of the card within the list. It then uses the position to obtain the data associated with that card from the ArrayList, "dataList". The data is then put into an intent so it can be displayed in Info Activity.



(HomeActivity.java - Part 1)



```java
public class HomeActivity extends AppCompatActivity {  // Complexity is 17 You must be kidding
    private void loadGymClasses() {  // Complexity is 16 You must be kidding

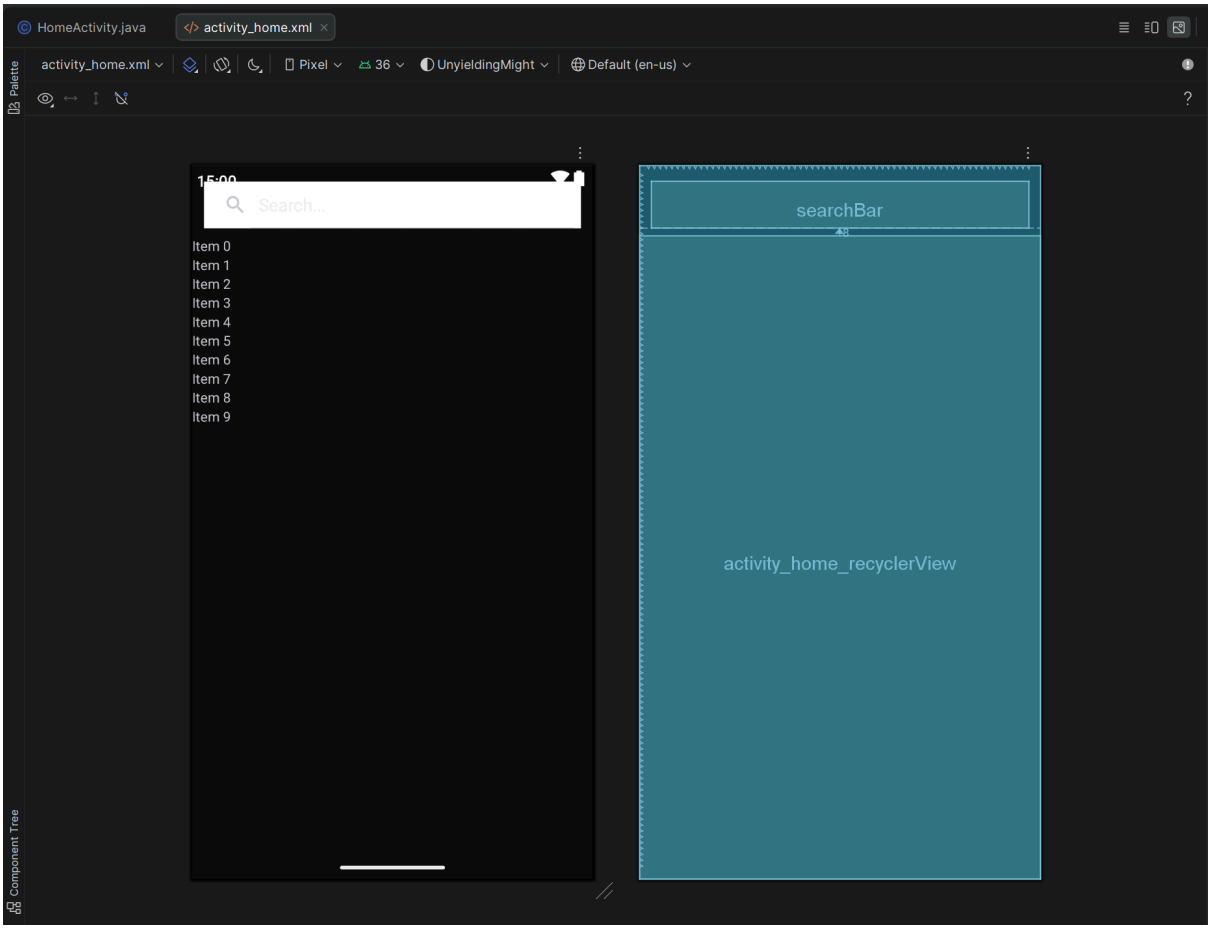
        new Thread(() -> {
            try {
                float tdee = 2000f;
                User current = Database.getCurrentUser();
                if (current != null) {
                    Customer customer = Database.getCustomer(current.getEmail());
                    if (customer != null) {
                        tdee = customer.getTDEE();
                        if (tdee <= 0) tdee = customer.calculateTDEE();
                    }
                }
                final float finalTdee = tdee;

                final ArrayList<GymClass> fetchedClasses = Database.getGymClassesAvailable();

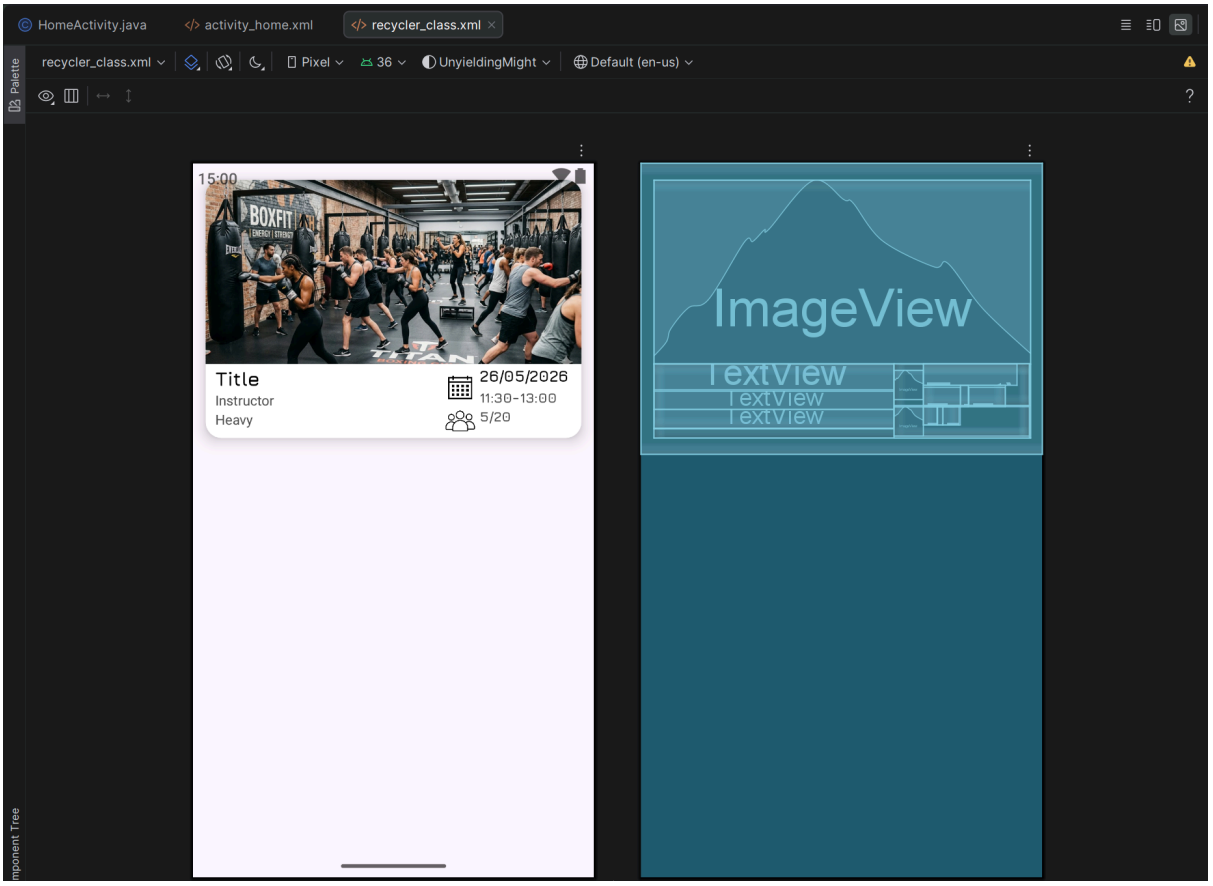
                runOnUiThread(() -> {
                    isLoaded = true;
                    gymClasses = fetchedClasses;
                    loadingSpinner.setVisibility(View.GONE);

                    if (gymClasses != null && !gymClasses.isEmpty()) {
                        adapter = new Adapter( context: HomeActivity.this, gymClasses, finalTdee);
                        recyclerView.setAdapter(adapter);
                    } else {
                        Toast.makeText( context: HomeActivity.this,  text: "No classes available at the moment.", Toast.LENGTH_SHORT).show();
                    }
                });
            } catch (Exception e) {
                runOnUiThread(() -> {
                    loadingSpinner.setVisibility(View.GONE);
                    Toast.makeText( context: HomeActivity.this,  text: "Error connecting to database", Toast.LENGTH_SHORT).show();
                });
            }
        }).start();
    }
}
```

(HomeActivity.java - Part 2)



(activity_home.xml)



(recycler_class.xml)

HomeActivity.java | InfoActivity.java

```java
package com.example.unyieldingmight.Activities;

import ...

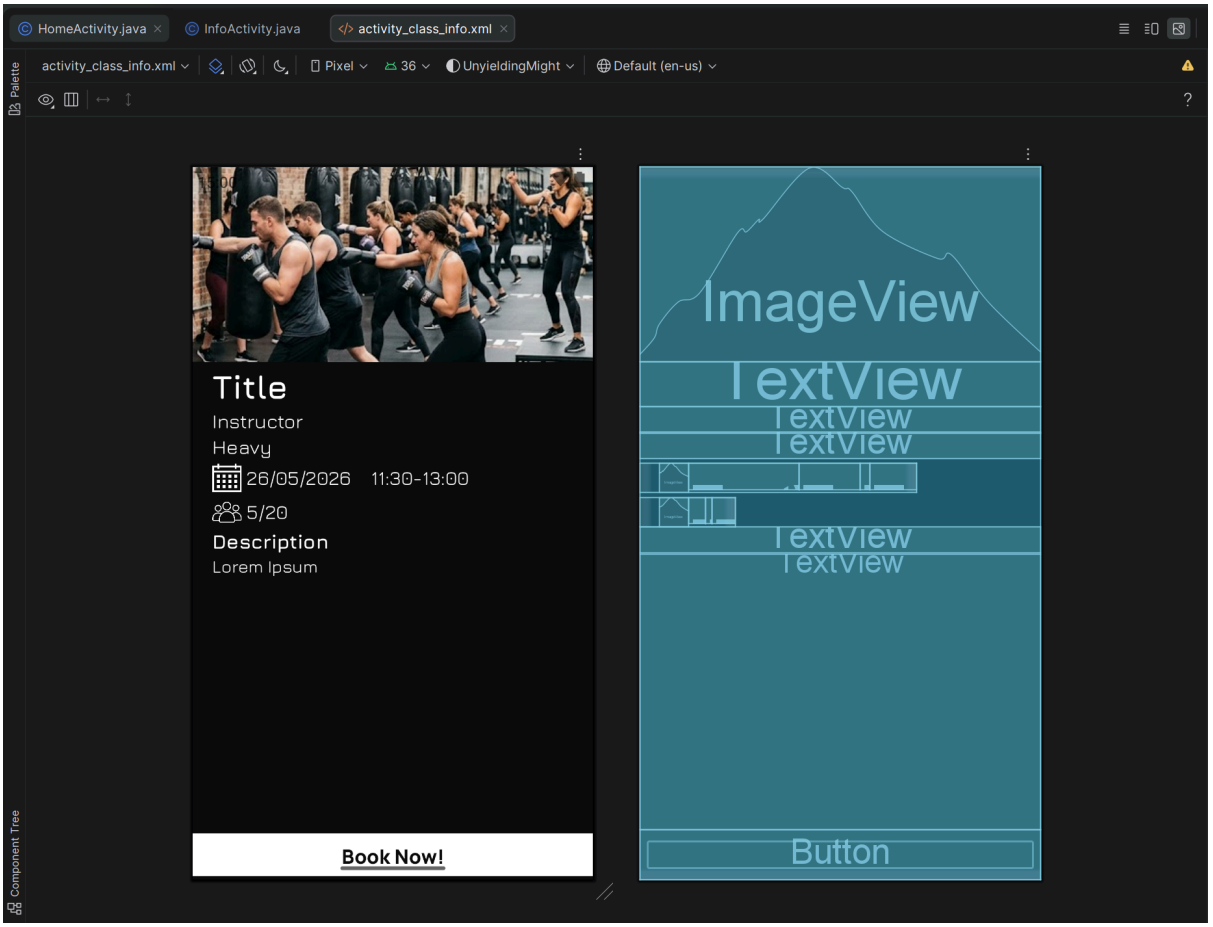
// For activity_class_info.xml
CBO: 25, DIT: 9, NOA: 200, TCC: 0.0
public class InfoActivity extends AppCompatActivity {  Complexity is 21 You must be kidding
    2 usages
    ImageView infoImage;
    2 usages
    TextView infoTitle, infoInstructor, infoIntensity, infoDate, infoStartTime, infoEndTime, infoCurCap, infoMaxCap, infoDesc;
    11 usages
    Button clickButton;
    4 usages
    int classId = -1;

    @Override
    protected void onCreate(Bundle savedInstanceState) {  Complexity is 20 You must be kidding
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_class_info);

        infoImage = findViewById(R.id.activity_class_iv_class);
        infoTitle = findViewById(R.id.activity_class_tv_title);
        infoInstructor = findViewById(R.id.activity_class_tv_instructor);
        infoIntensity = findViewById(R.id.activity_class_tv_intensity);
        infoDate = findViewById(R.id.activity_class_tv_date);
        infoStartTime = findViewById(R.id.activity_class_tv_startTime);
        infoEndTime = findViewById(R.id.activity_class_tv_endTime);
        infoCurCap = findViewById(R.id.activity_class_tv_currentCapacity);
        infoMaxCap = findViewById(R.id.activity_class_tv_maxCapacity);
        infoDesc = findViewById(R.id.activity_class_tv_description);
        clickButton = findViewById(R.id.activity_class_btn_book);

        Bundle bundle = getIntent().getExtras();
        if (bundle != null){
            classId = bundle.getInt( key: "ClassId",  defaultValue: -1);
            infoImage.setImageResource(bundle.getInt( key: "Image"));
            infoTitle.setText(bundle.getString( key: "Title"));
            infoInstructor.setText(bundle.getString( key: "Instructor"));
            infoIntensity.setText(bundle.getString( key: "Intensity"));
            infoDate.setText(bundle.getString( key: "Date"));
            infoStartTime.setText(bundle.getString( key: "StartTime"));
            infoEndTime.setText(bundle.getString( key: "EndTime"));
            infoCurCap.setText(bundle.getString( key: "CurCap"));
            infoMaxCap.setText(bundle.getString( key: "MaxCap"));
            infoDesc.setText(bundle.getString( key: "Desc"));
        }
```

(InfoActivity.java)

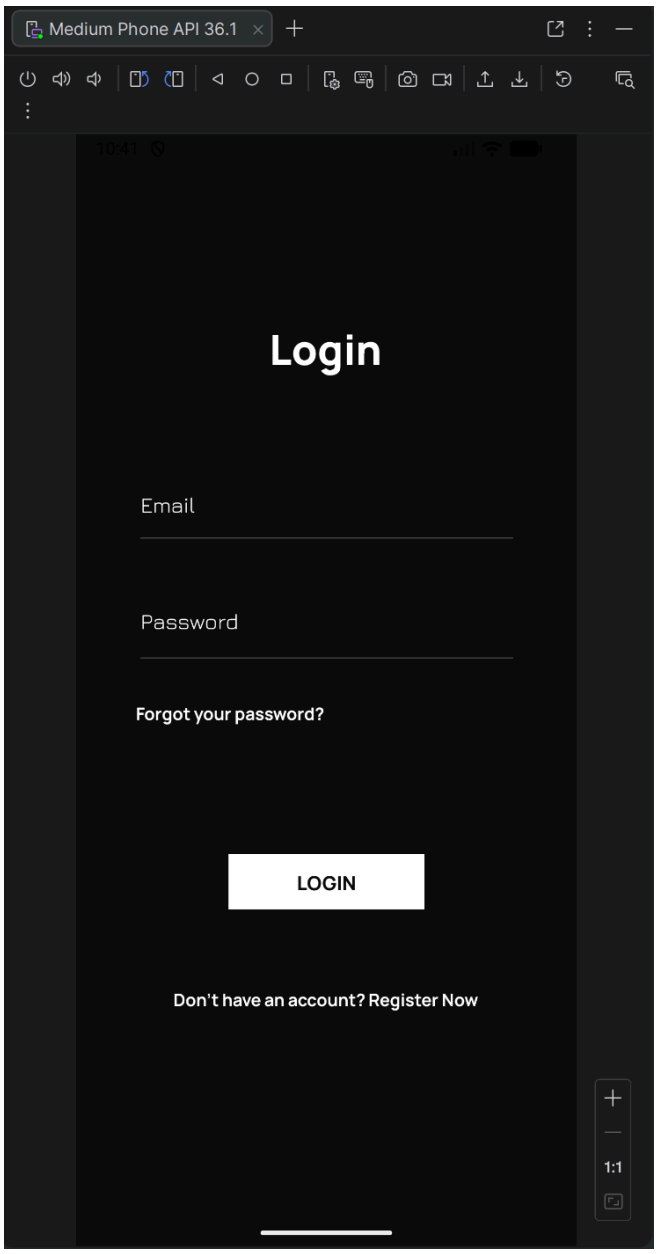


(activity_class_info.xml)

2.2 Core Functionalities

2.2.1 Authentication and Authorisation (Admin & Customer Login)

Both admin and customer utilises the same UI for logging in as seen in the image below.



(Login Activity)

Our software can distinguish between an admin and a customer by performing a "SELECT WHERE" query in the SQL database. The query will first locate the user from the "UserProfile" table, using the provided email. Once the user has been identified by their email, the data in the "UserClass" column is retrieved, which has the corresponding value of "ADMIN" or "CUSTOMER". If the user is a customer, they will be able to book a class and receive a booking confirmation email; whereas, admins are able to edit the class's details. The code for this can be found in lines 144-183 within "Database.java", which is in the "Services" folder. Images are attached below as evidence.

```java
1 usage
public static User loginUser(String email, String password) {  Complexity is 4 Everything is cool!
    // Validates the user credential and returns the user
    User user = getUser(email, password);
    if (user != null) {
        getInstance().currentUser = user;
    }
    return user;
}

// Fetch the user from the database
1 usage
public static User getUser(String email, String password) {  Complexity is 11 You must be kidding
    String sql = "SELECT * FROM UserProfile WHERE Email = ?";
    Connection conn = getConnection();
    if (conn == null) return null;
    try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString( i: 1, email);
        ResultSet rs = pstmt.executeQuery();
        if (rs.next()) {
            String storedHash = rs.getString( s: "ProfilePassword");
            String inputHash = Security.hashData(password);

            // Check the hashed input and the hashed password stored in the database
            if (storedHash.equals(inputHash)) {
                String roleStr = rs.getString( s: "UserClass");
                User.Role role = (roleStr.equalsIgnoreCase( anotherString: "ADMIN"))
                        ? User.Role.ADMIN : User.Role.CUSTOMER;

                // Create user class for the logged user
                return new User(
                        rs.getInt( s: "ProfileId"),
                        username: rs.getString( s: "FirstName") + " " + rs.getString( s: "LastName"),
                        storedHash,
                        rs.getString( s: "Email"),
                        role
                );
            }
        }
    } catch (Exception e) { Log.e( tag: "DATABASE",  msg: "User Fetching Error: " + e.getMessage()); }
    return null;
}
```

(Database.java - loginUser & getUser method)

Results Messages

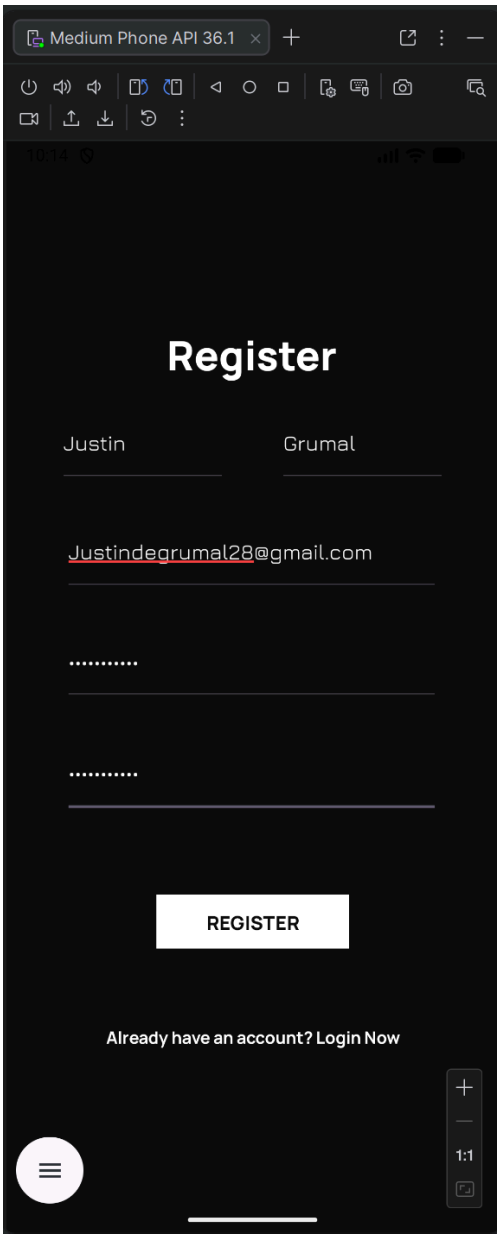
	ProfileId	Email	ProfilePassword	Age	FirstName	LastName	DateOfBirth	Gender	AddressId	CustomerId	UserClass
1	1	grum7@manukaumail.com	ef92b778bafe771e89245b89ecbc08a44a4e166c06659911...	24	Justin	Grumal	2001-11-07	MALE	1	NULL	ADMIN
2	2	loisdds30@gmail.com	ef92b778bafe771e89245b89ecbc08a44a4e166c06659911...	24	Lois Dara	Sanglap	2002-03-05	FEMALE	1	NULL	ADMIN
3	3	nepomucenodwayne@gmail.com	ef92b778bafe771e89245b89ecbc08a44a4e166c06659911...	21	Dustin Dwayne	Diaz	2005-04-14	MALE	1	NULL	ADMIN

(SQL Database IP_G1_Gym - UserProfile table)

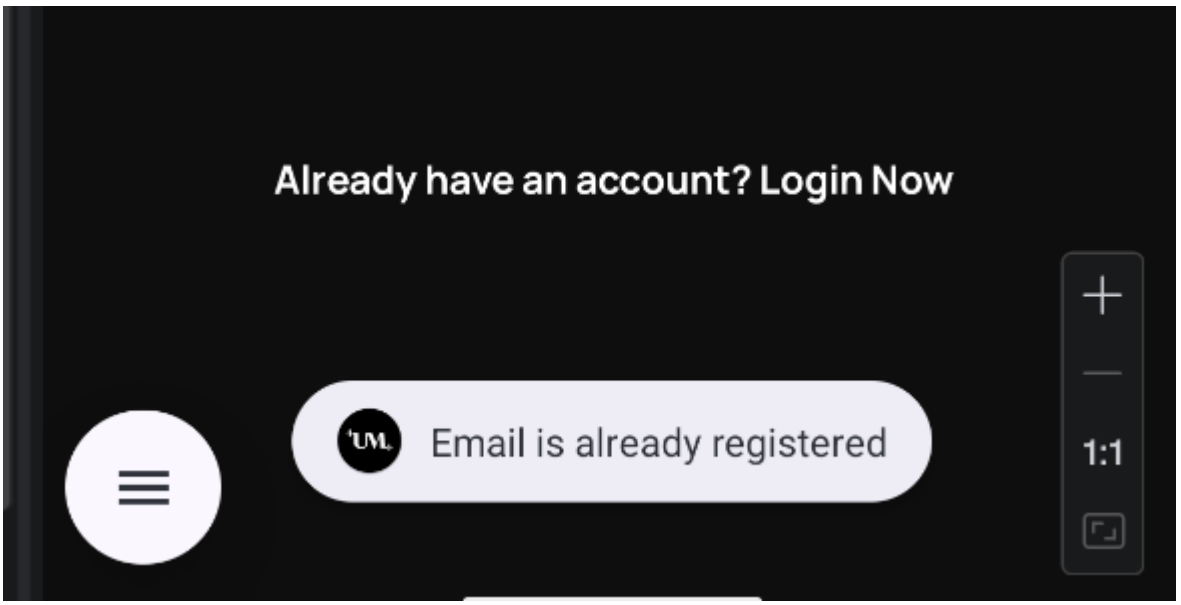
2.2.2 Primary Functionalities

2.2.2.1 Customer Registration

Below are a collection of images demonstrating the procedure to register an account for Unyielding Might. To navigate to the registration activity, click the “Register Now” link found in the login activity. Once redirected, the user will be asked to input their first name, last name, email, and password. If the user tries to register an account with an existing email, our software will inform the user that the email is already registered.

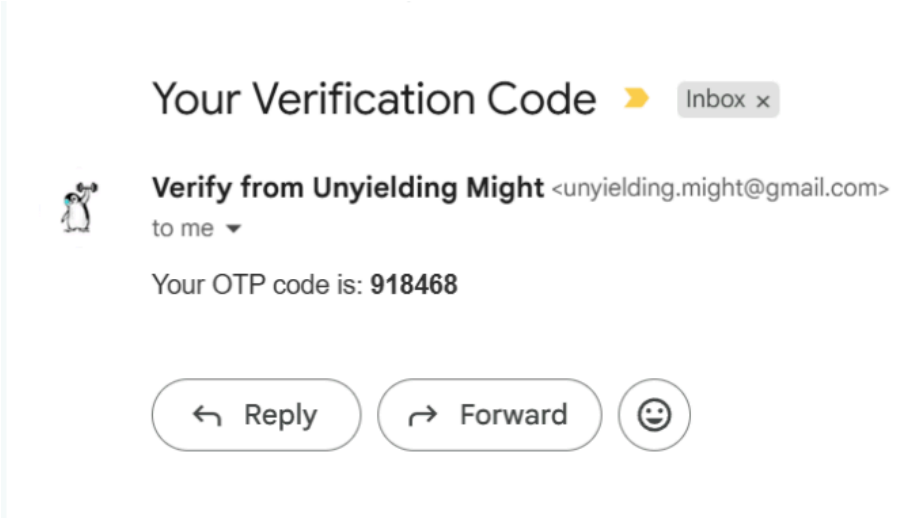


(Registration Activity - Form filled)

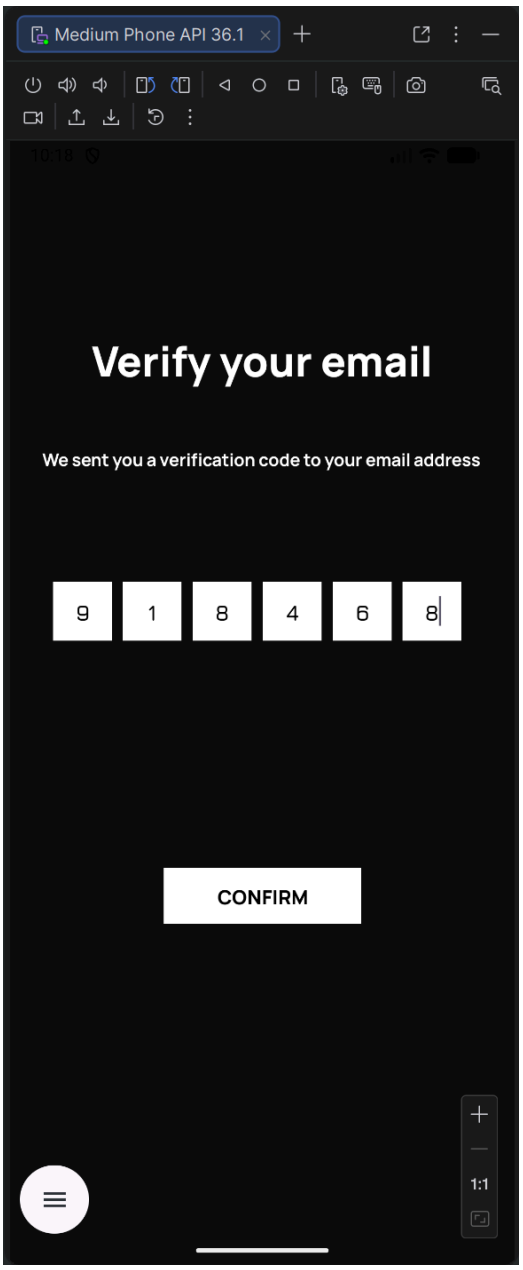


(Registration Activity - toast)

Upon clicking register, the user will receive an email in either their inbox or spam. The contents of the email is a 6-digit OTP (One Time Password) code, which the user has to input in the following screen in order to proceed registering.

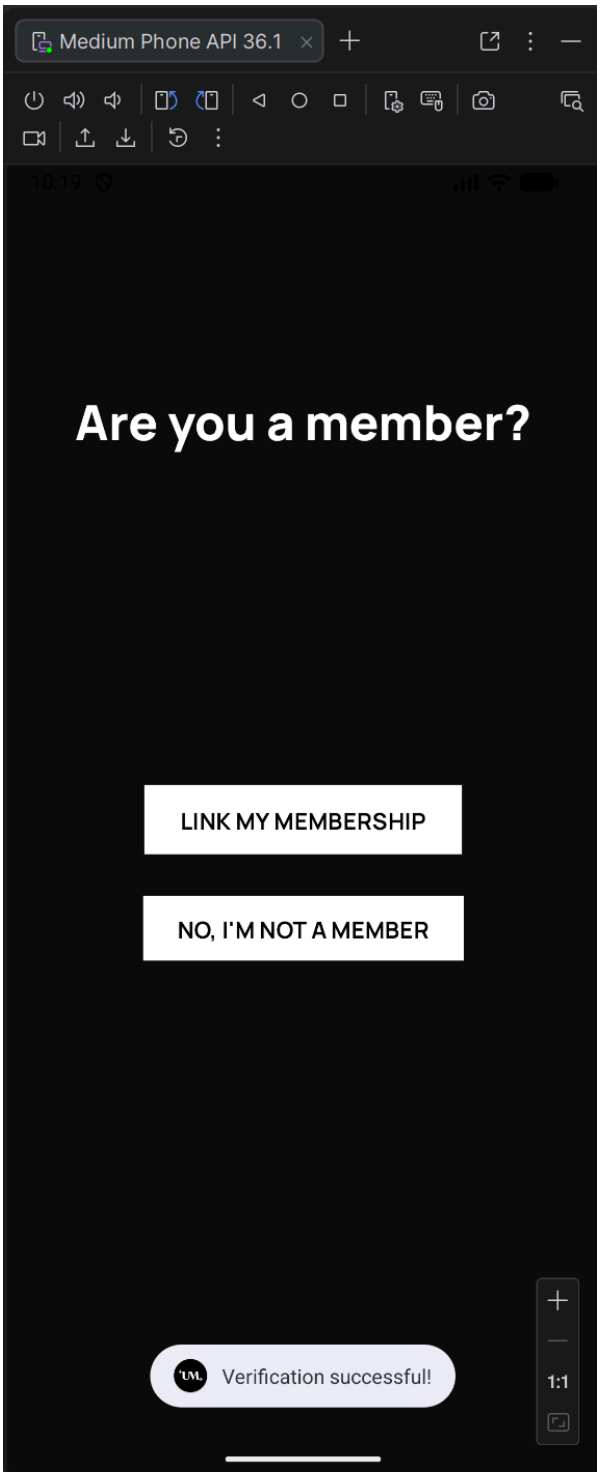


(Email - OTP)



(Verify Activity - OTP filled)

After the user inputs the OTP and their identity has been verified, the user will be prompted to either link their existing membership or select “No, I’m Not a Member”. However, in this demonstration, we will assume the user does not have a prior membership in Unyielding Might. To see what linking your membership is like, refer to Section 3.3.3.



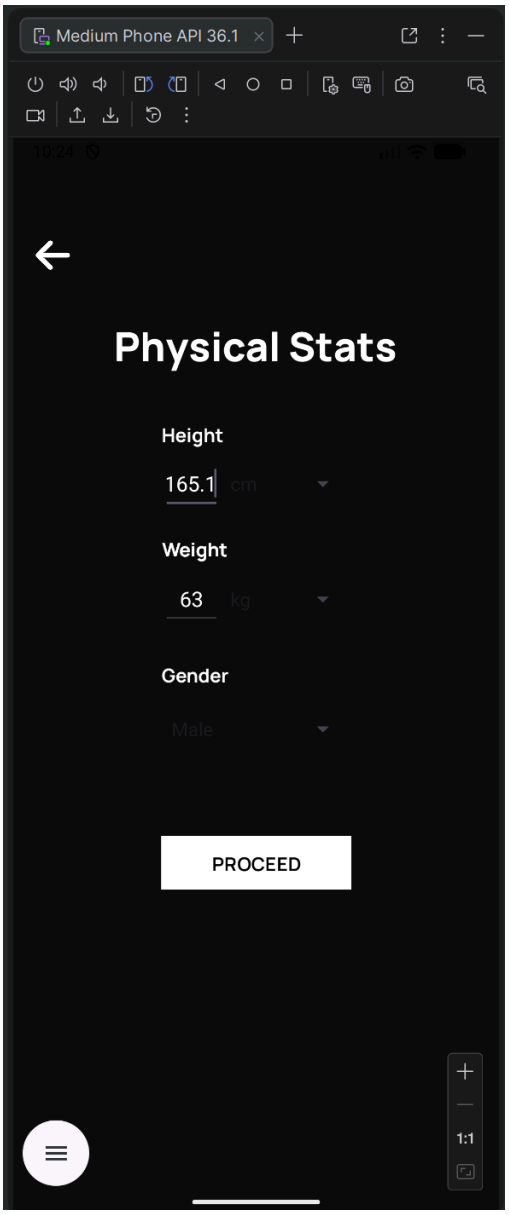
(Link membership Activity)

The user will be prompted to complete their profile by inputting their date of birth and address as seen in the image below.



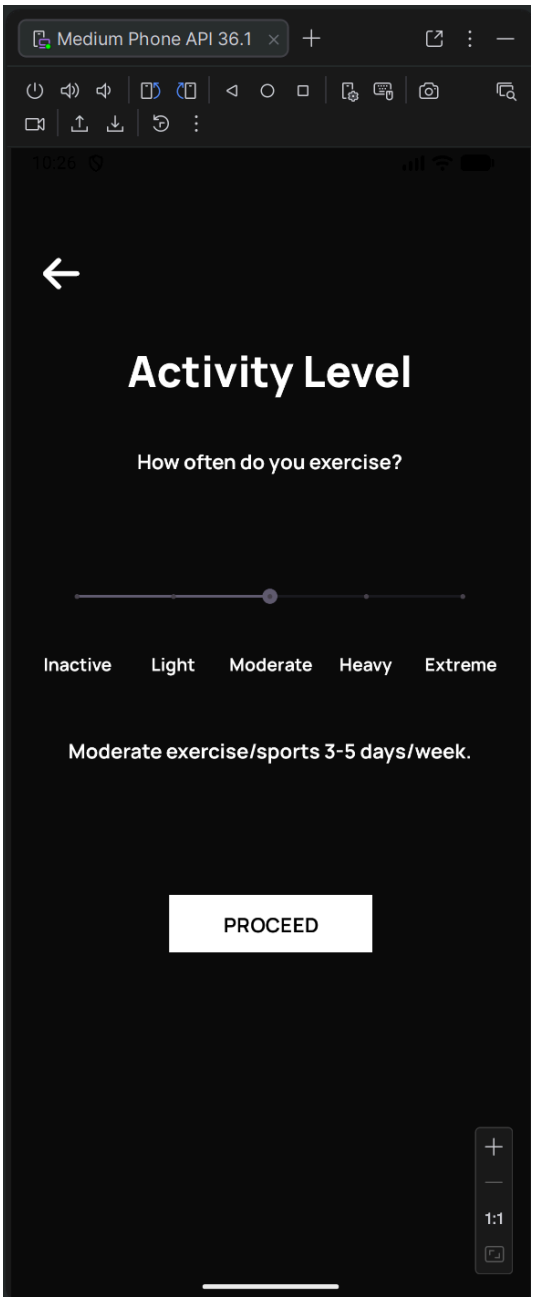
(Profile Activity - Form Filled)

Once the user inputs their details, the user will be asked to enter their physical stats, including their height, weight, and gender. Height and weight are accompanied by a drop-down UI, empowering the user to choose either metric (cm, kg) or imperial (ft, pounds) units respectively. The inclusion of this feature improves UX since the user may only know their stats in a specific unit.



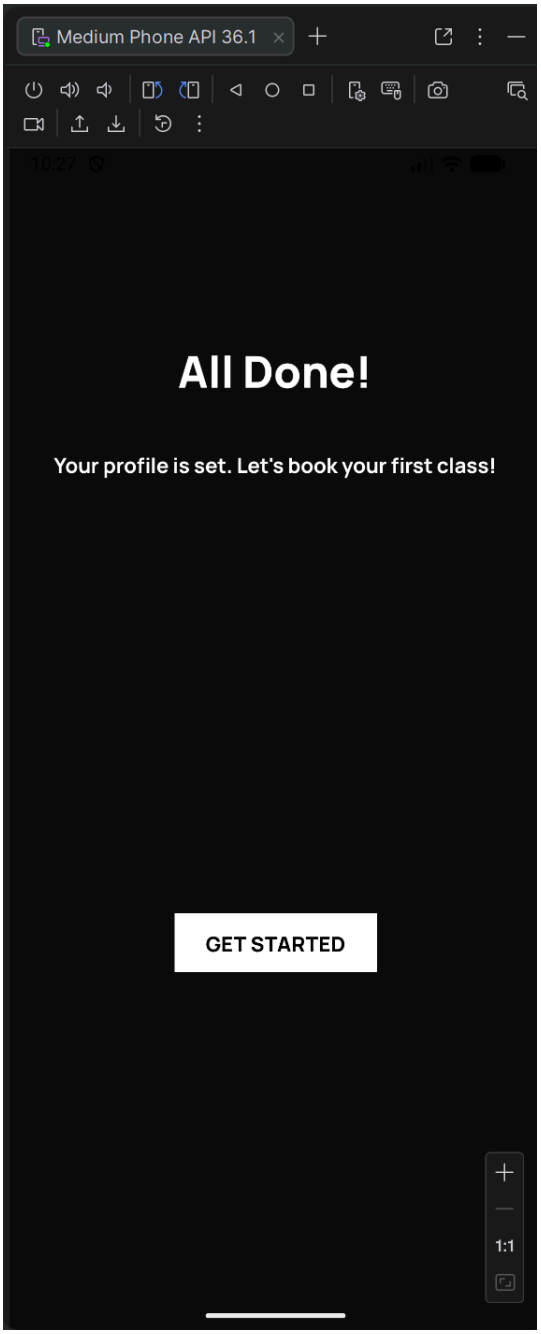
(Stats Activity - Form Filled)

After entering their stats, the user will be greeted with the last activity, activity level. Using a seek bar, the user can select how active they are, ranging from inactive to extreme. Each activity level is accompanied with a short description which is displayed dynamically.



(Activity level Activity)

Upon finishing registration, the user will be greeted with this activity. When clicking the button “Get Started”, the user will be redirected back to the login screen, where the user is now able to login.

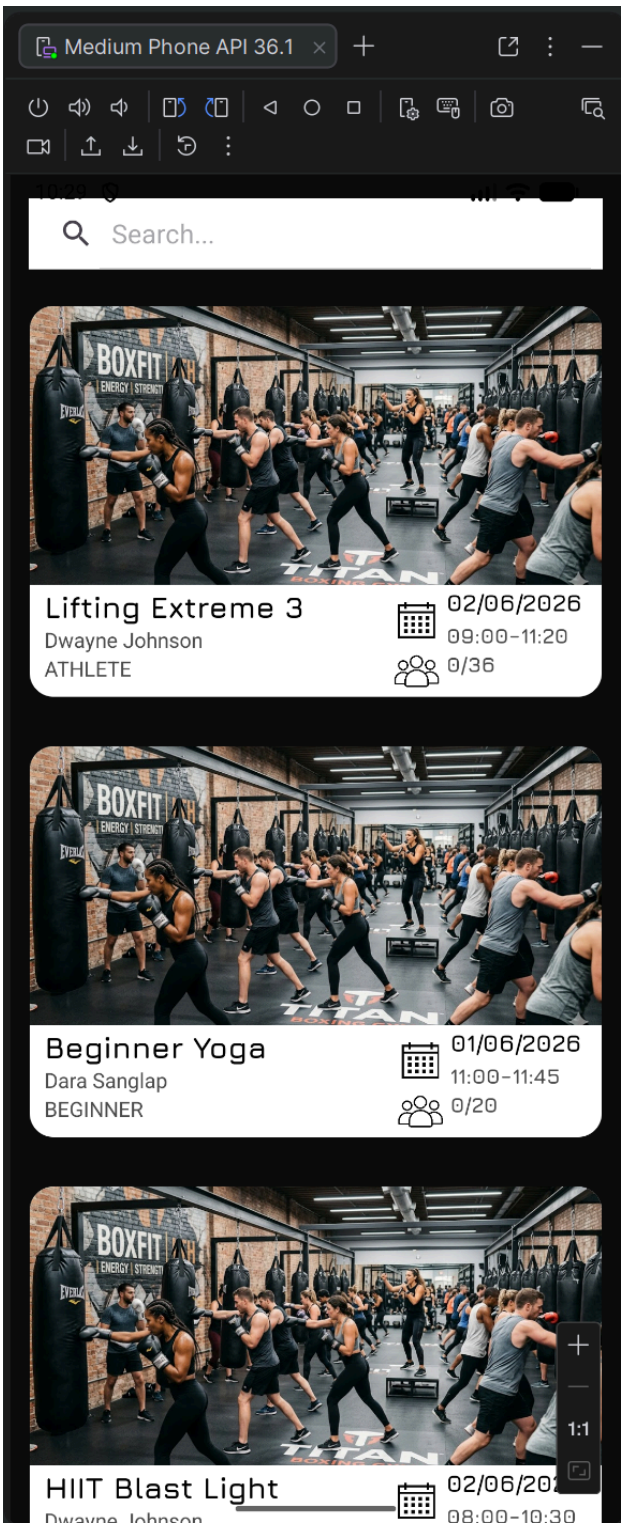


(Done Activity)

2.2.2.2 Booking Class

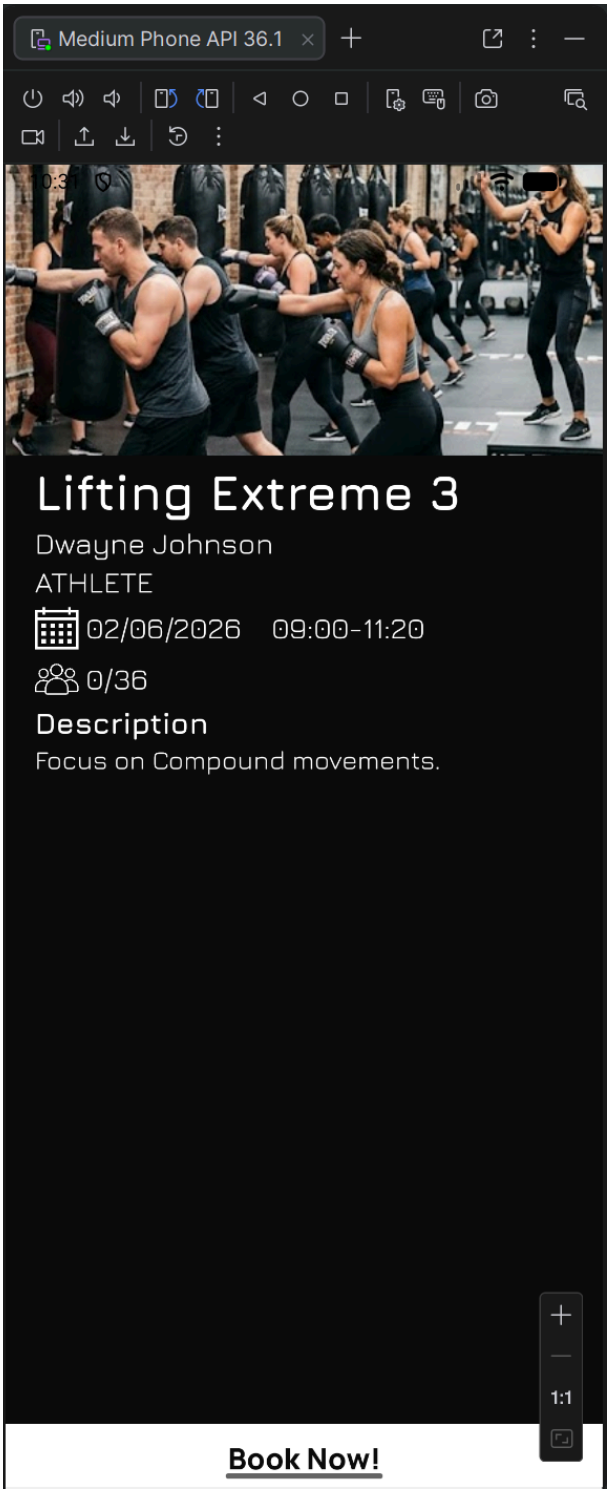
When logged in as a customer, the user is able to book a class. However, to book a class, the user must first decide which class to attend. The home activity utilises a RecyclerView to display a list of cards, showcasing what classes are

currently available to book. Upon interacting with a card, the user can view the full description of the class as seen in the collection of images below.



(Home Activity)

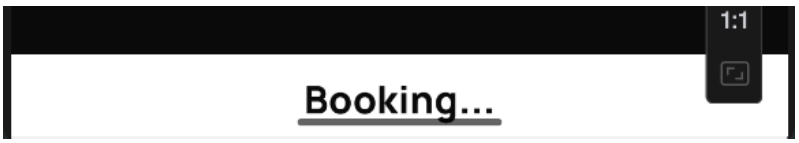
Note: *Search bar functionality has not been implemented.*



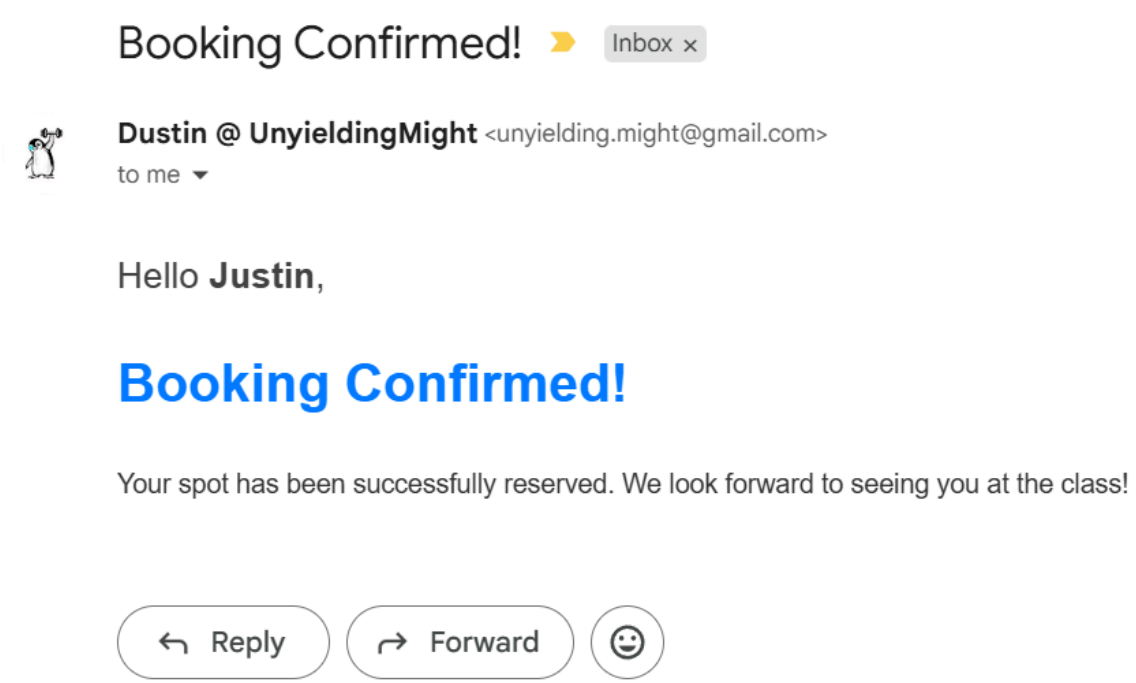
(Info Activity)

When the user (customer) is in the Info Activity, the button on the bottom of the screen will display “Book Now!”. The user can proceed to book the class by interacting with the button. Once the class has been booked the user will receive an email, confirming they have booked the class successfully. If the user attempts to

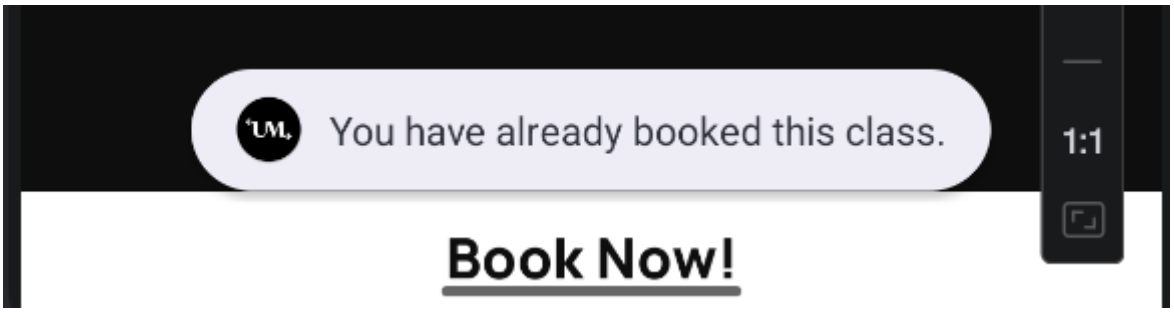
book the same class twice a toast notification will appear on the screen, informing them that they have already booked the class.



(Info Activity - Book Now! button upon clicking it)

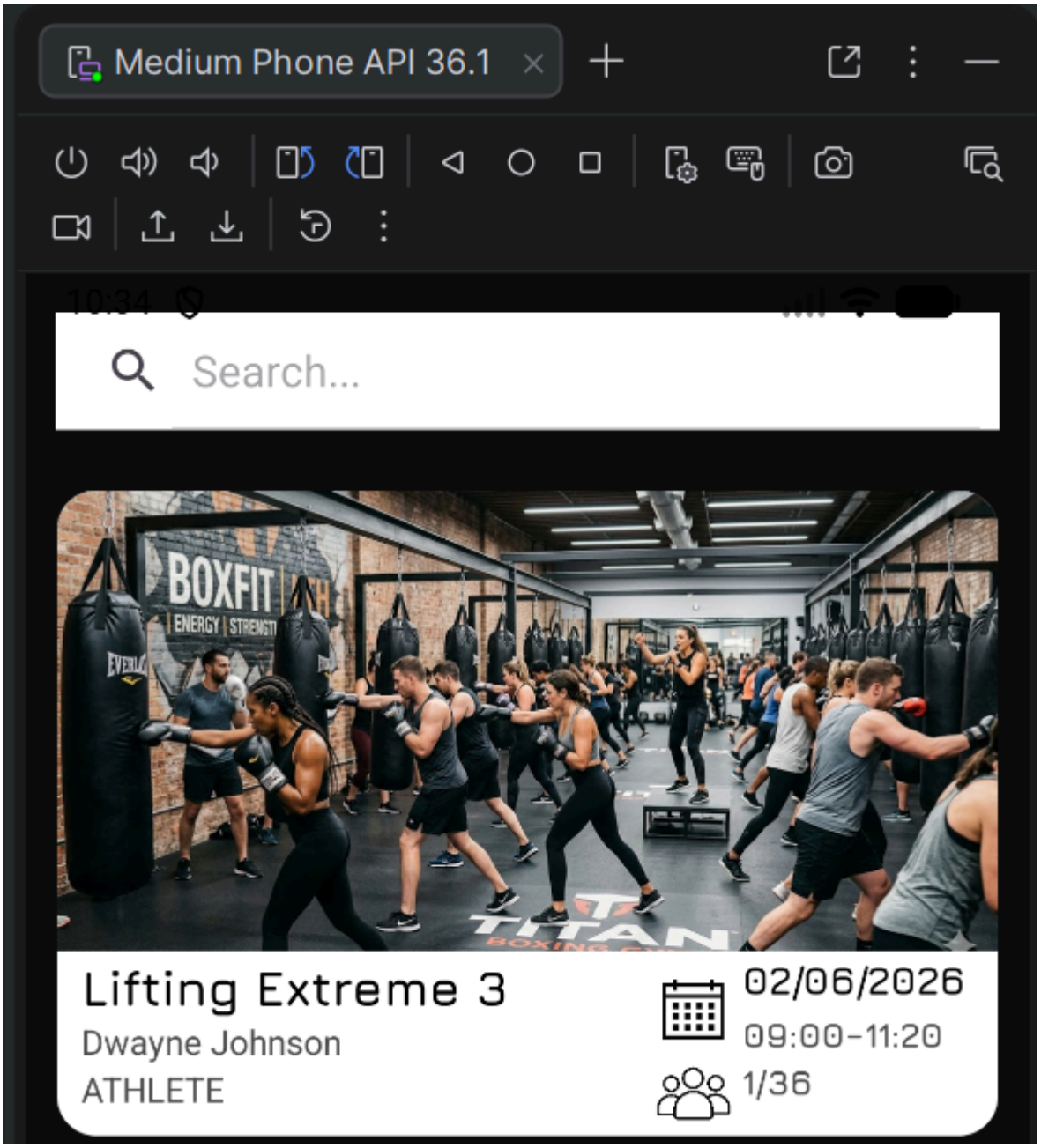


(Email - Booking confirmed)



(Info Activity - Toast for booking the same class twice)

Once booking has been confirmed, the SQL database will update accordingly and increase the “currentCapacity” by 1. This is showcased if the user navigates back to the Home Activity as seen in the image below.



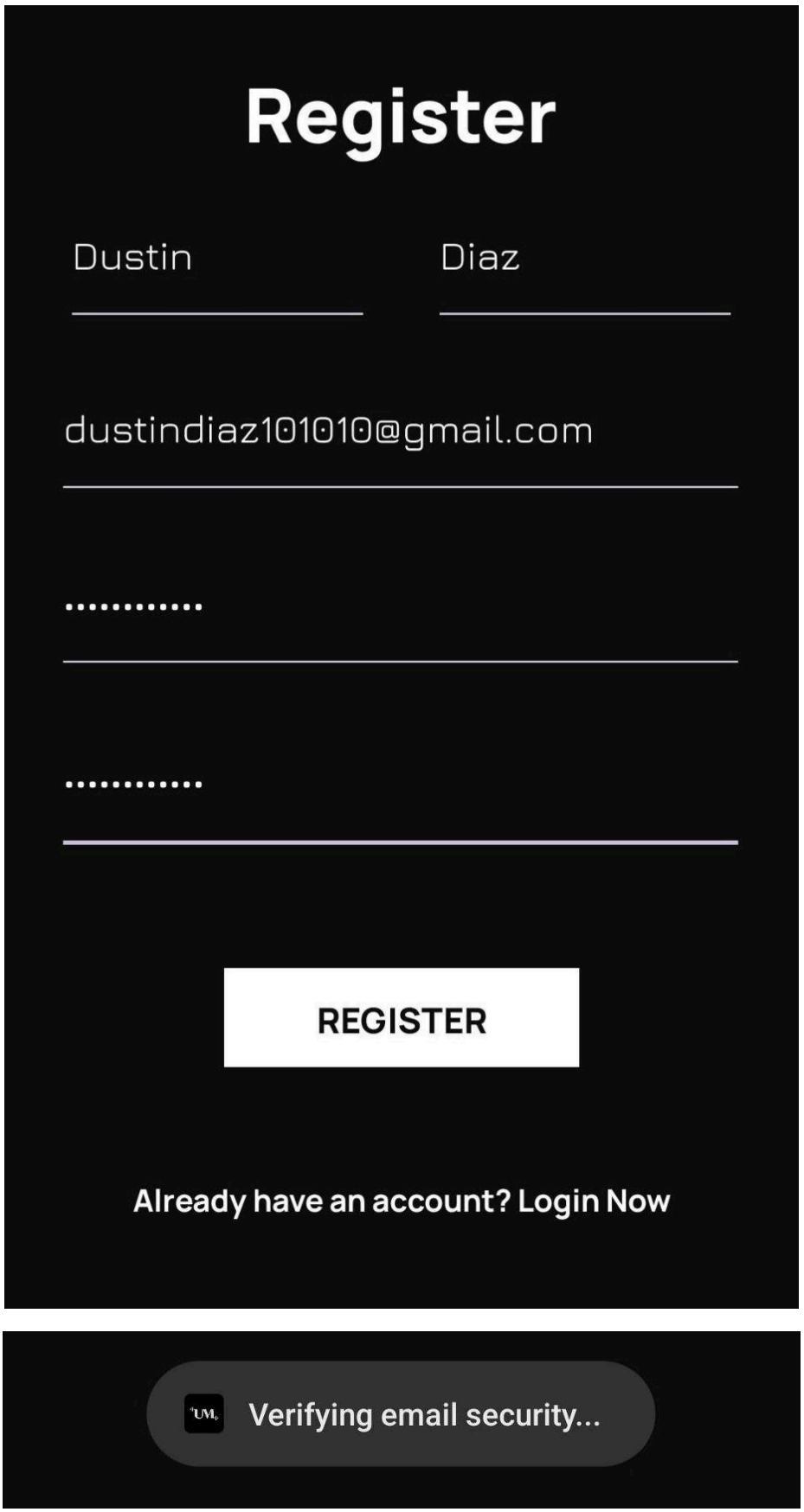
(Home activity - Current capacity updated by 1)

2.2.3 Integration of Software Quality Attributes

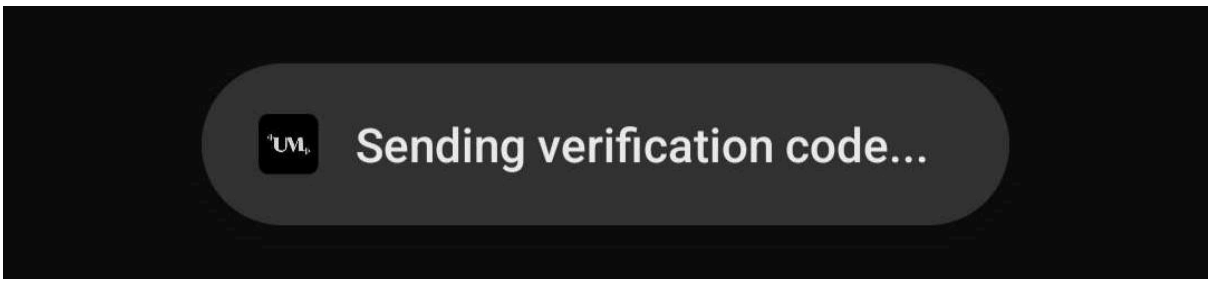
2.2.3.1 Security

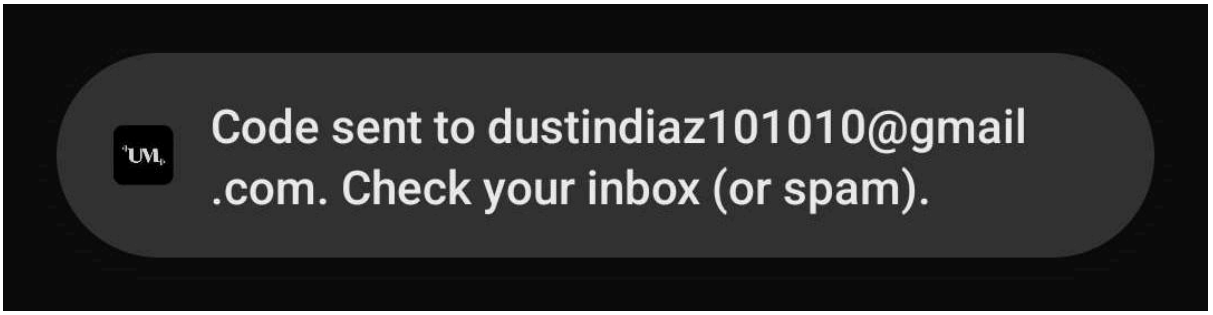
There are many security elements found within the application's code but the three notable ones are email verification with OTP, role access control, and data encryption.

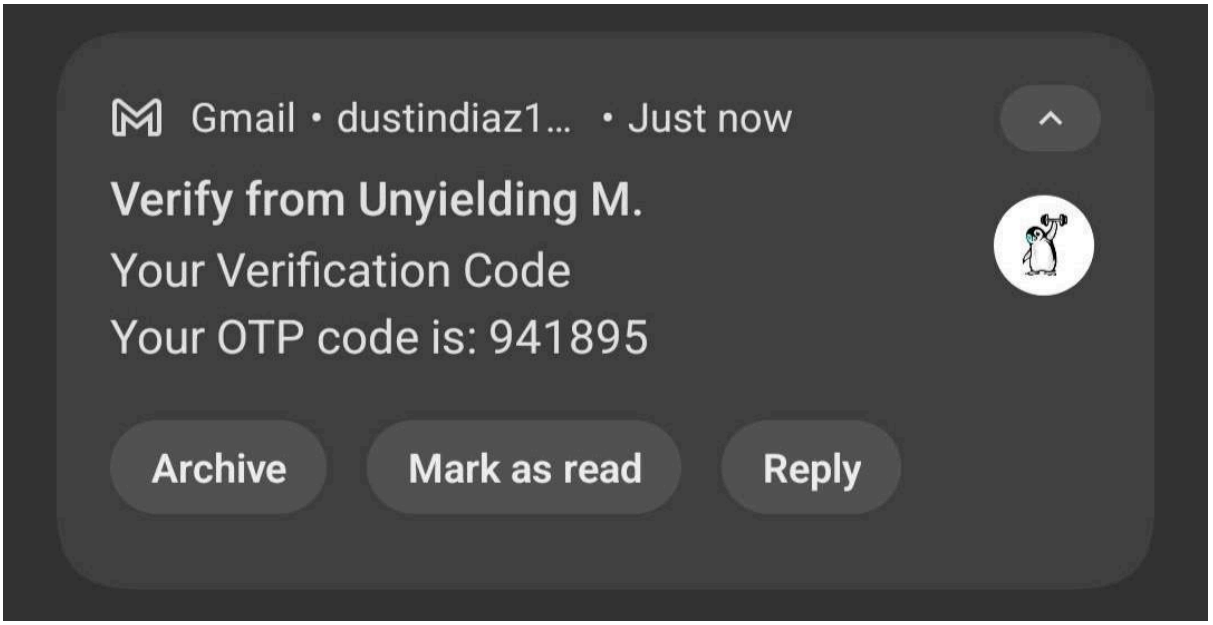
The email verification process first checks the email with QuickEmailVerification API which checks the email if it is a valid email, not a disposable one, and if it is safe to send. Once the email goes through that process, the given email will be sent an OTP generated by the EmailOTP class which generates a number from 1-999999. After the email, the user will then have to input the OTP and the system checks if it is the same number generated compared to what the user has input. Once validated, the registration process which gets the customer's personal information.

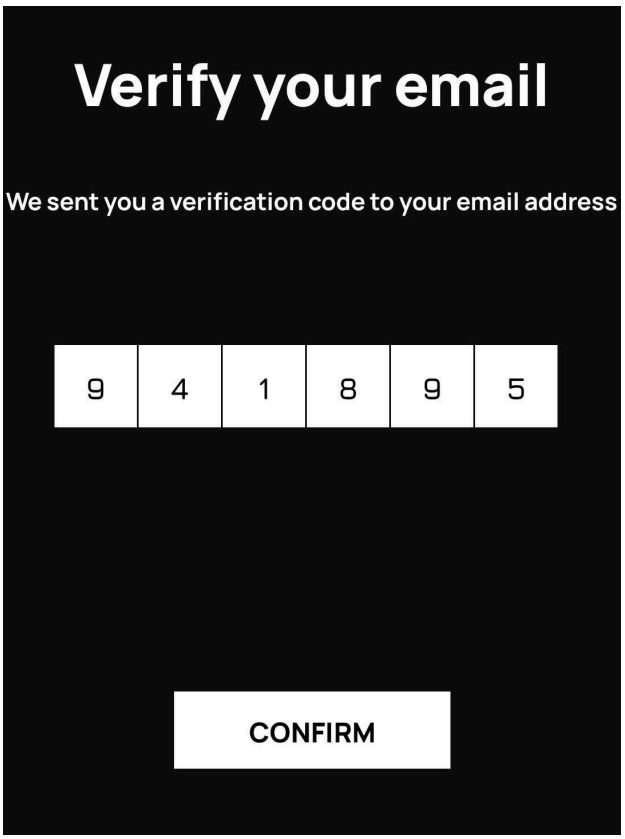

Register
Dustin
Diaz
dustindiaz101010@gmail.com
REGISTER
Already have an account? Login Now

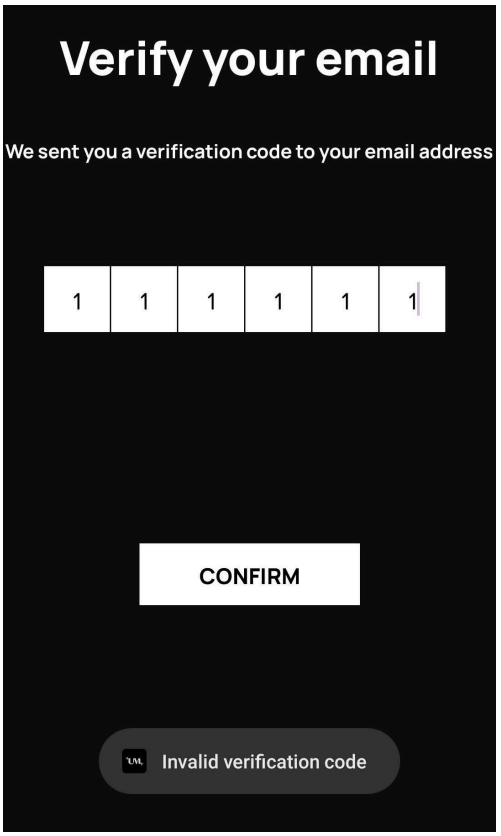
Verifying email security...

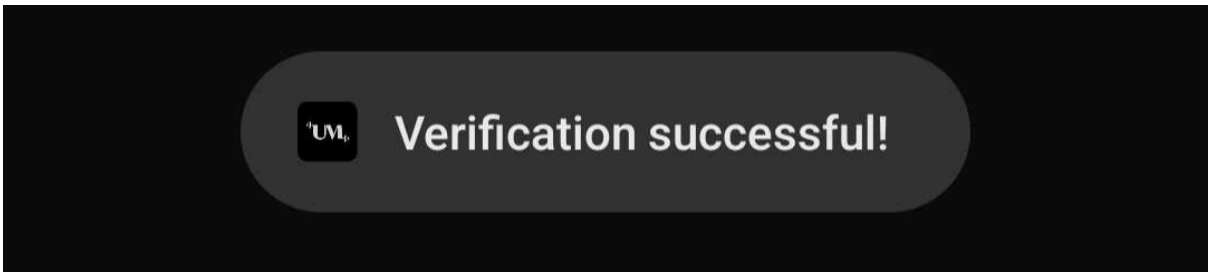

Sending verification code...


Code sent to dustindiaz101010@gmail
.com. Check your inbox (or spam).


Gmail • dustindiaz1... • Just now
Verify from Unyielding M.
Your Verification Code
Your OTP code is: 941895
Archive
Mark as read
Reply

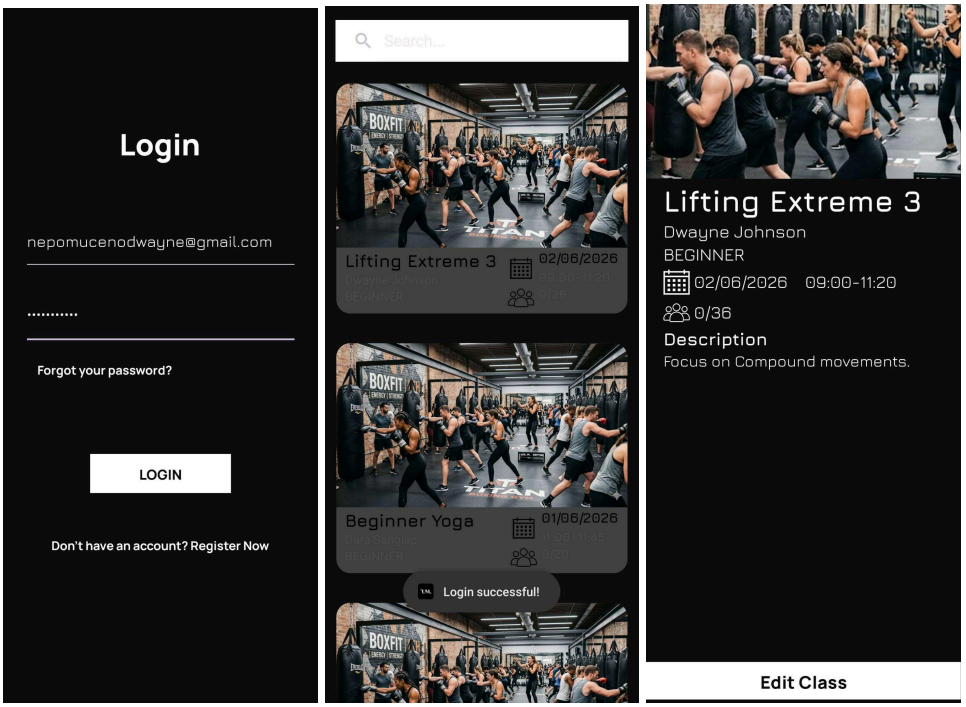




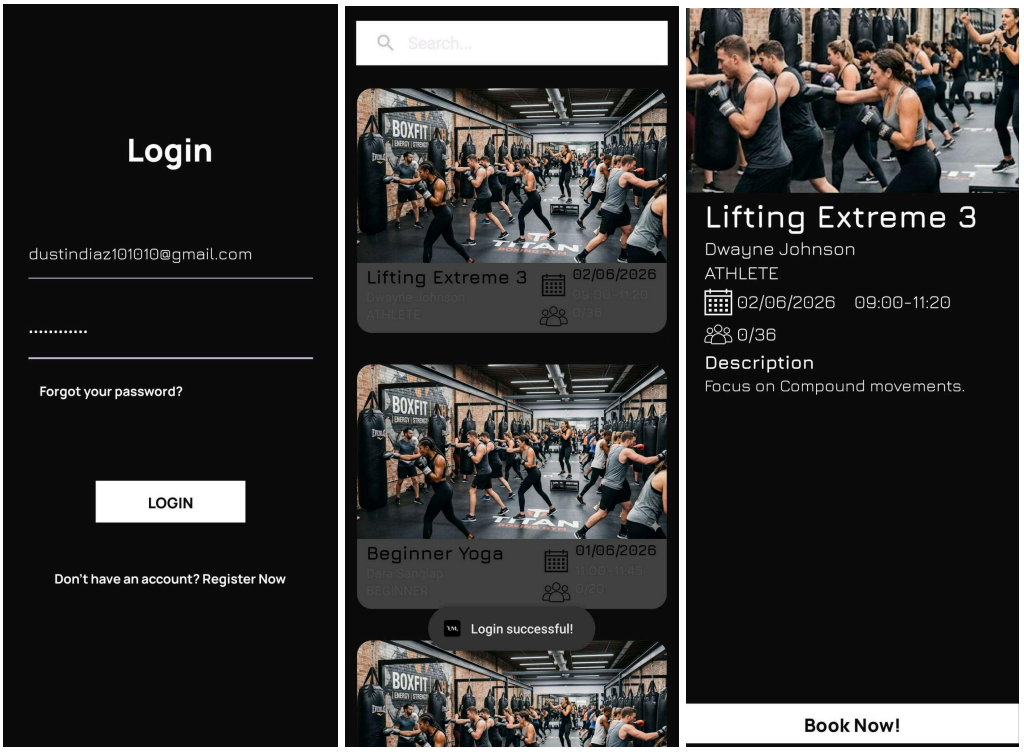


For checking the role of the email. Only 3 admins are registered in the system's database. In order to check the role of the user, The getUser function in the Database class will then fetch the data from the database in the UserProfile table which stores the user credentials and personal information will check the UserClass attribute to see if it is an "ADMIN" or "CUSTOMER". Both will be brought to the HomeActivity which is the same look but the difference is for the button where it will display "Edit Class" for admin and "Book Now!" for customers.

Admin Authentication:



Customer Authentication:



For integrating data encryption, SHA-256 may not be the best but does the job to showcase data encryption from human readable data to a fixed length of 64 characters string. The encryption can be done using the MessageDigest class with SHA-256 encryption and passing the data in it where it would return a list of 32 bytes

and for each byte it is checked and converted into its unsigned value building the final hashed data. The data encrypted using this function is then stored in the database which only applies for the password. The code snippet shown is inspired by Gregory Cernera's code on his Medium article (Cernera, 2022).

```
public static String hashData(String text) throws Exception {  Complexity is 6
    // Encrypting data using SHA256
    MessageDigest md = MessageDigest.getInstance( algorithm: "SHA-256");

    byte[] hashBytes = md.digest(text.getBytes(StandardCharsets.UTF_8));

    StringBuilder hexString = new StringBuilder();
    for (byte b : hashBytes) {
        String hex = Integer.toHexString( i: 0xff & b);
        if (hex.length() == 1) hexString.append('0');
        hexString.append(hex);
    }
    return hexString.toString();
}
```

	ProfileId	Email	ProfilePassword	Age	FirstName	LastNa
1	1	grum7@manukaumail.com	ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d…	24	Justin	Gruma
2	2	loisdds30@gmail.com	ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d…	24	Lois Dara	Sangla
3	3	nepomucenodwayne@gmail.com	ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d…	21	Dustin Dwayne	Diaz
4	4	dustindiaz101010@gmail.com	e9a0475f28a6e2d1495a0922e946ba5a6f5264ca52212d704532fa40…	21	Dustin	Diaz

2.2.3.2 Performance

To ensure UM's application meets its performance requirement, threading has been utilised throughout the .java activity files. Threading is a preventative measure to allow static Database methods to run without causing the system to freeze or crash (Android Developers, 2024). By implementing threading, Database methods do not occupy the UI thread, allowing the screen to still render and output a response. The image below is an example of how threading is integrated.

```
new Thread(() -> {
    try {
        User user = Database.loginUser(emailData, passwordData);
        runOnUiThread(() -> {
            loginButton.setEnabled(true);
            loginButton.setText("LOGIN");

            if (user != null) {
                Toast.makeText( context: this,  text: "Login successful!", Toast.LENGTH_SHORT).show();
                Intent i = new Intent( packageContext: this, HomeActivity.class);
                i.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
                startActivity(i);
            } else {
                Toast.makeText( context: this,  text: "Invalid email or password", Toast.LENGTH_LONG).show();
            }
        });
    } catch (Exception e) {
        Log.e( tag: "LoginActivity",  msg: "Login error", e);
        runOnUiThread(() -> {
            loginButton.setEnabled(true);
            loginButton.setText("LOGIN");
            Toast.makeText( context: this,  text: "A connection error occurred", Toast.LENGTH_LONG).show();
        });
    }
}).start();
```

(LoginActivity.java)

2.2.3.3 Maintainability

To provide the application a fine-grade maintainability, the database is comprehensively crafted with an extremely precise design to ensure that the system does not log existing email registered to another user and it shall not allow membership to be registered to someone who does not own the membership by validating the email.

For the structure of the project, the Model-View-Controller architecture is followed where the Models act as the processor of the information, Views output the response, and Controllers handle the request and input from the user. This structure can be found within the main file directory where each component is categorised as “Activities”, “Models”, “Services”, and “Utils” (Utilities). These three jointly work together to handle the flow of the data process in a readable manner.

For the classes found in the entire codebase, it possesses a well-structured design for the attributes and method in order to present a straightforward context for better code support and easier maintenance. Following design patterns made the

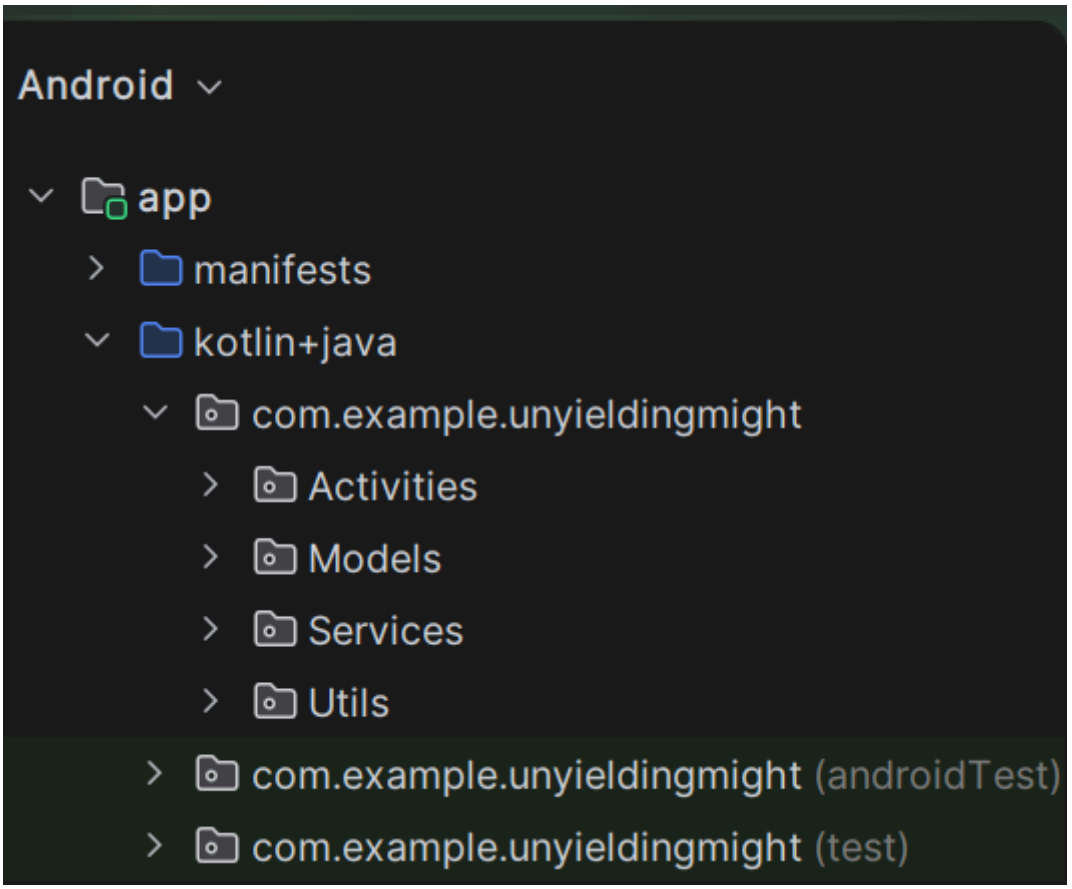
classes have reusable and human readable use for upcoming changes and maintainability integration.

2.3 Code Quality

2.3.1 Folder Structure

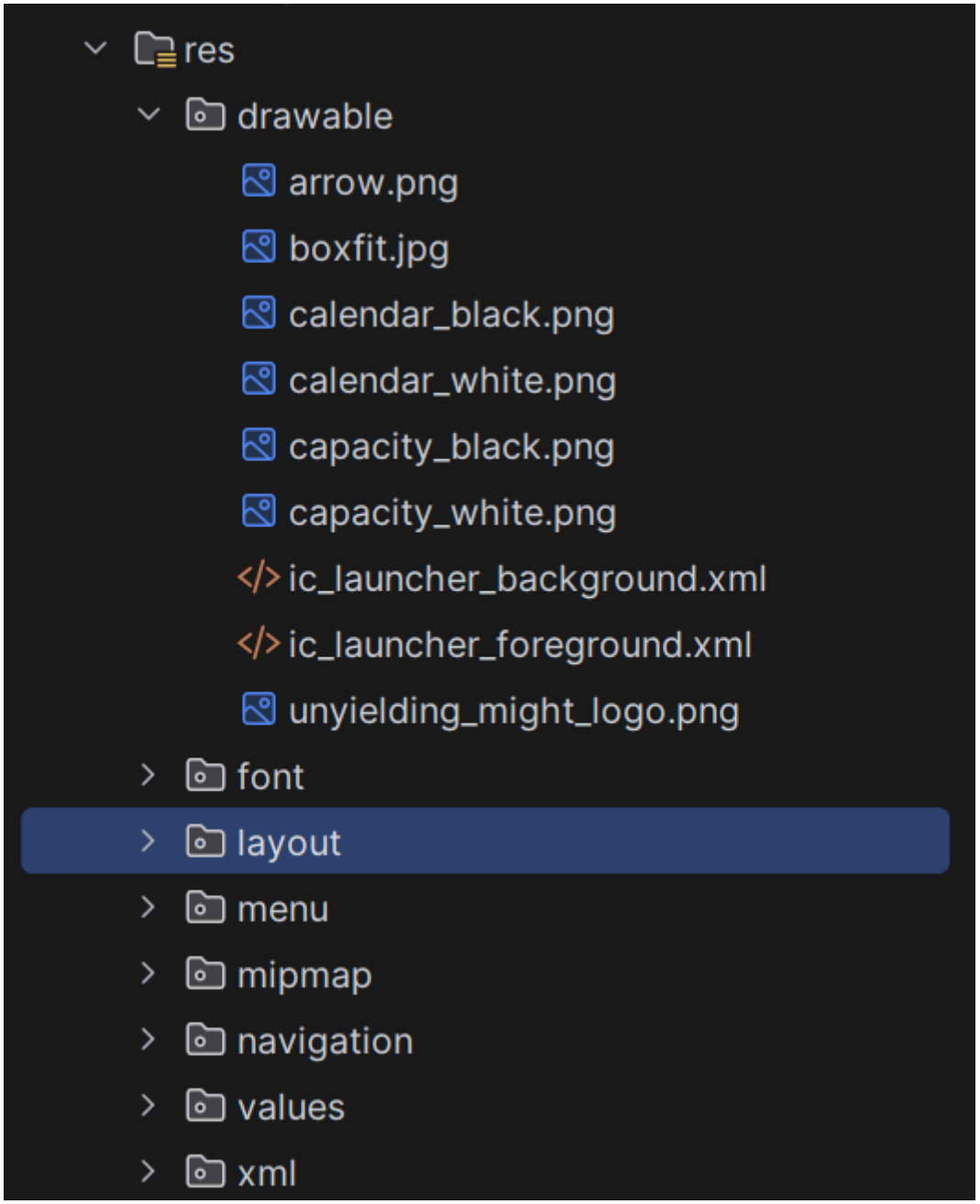
All .java files are found in “com.example.unyieldingmight”. The files are separated into 4 distinct folders, Activities, Models, Services, and Utils. In the “Activities” folder, files pertaining to the activities can be found (ex. LoginActivity.java, RegisterActivity.java, etc.). In “Models”, files containing business logic can be found, separated from the UI (ex. GymClass.java, Customer.java, etc.). In “Services”, files related to external services like database and email functionality can be found (Database.java, EmailOTP.java, etc.). Lastly, “Utils” contains files stores helper classes. For example, “conversion.java”, converts imperial units like feet to metric units like cm.

Additionally, test cases used in section 3.0 can be found in “com.example.unyieldingmight (test)”.

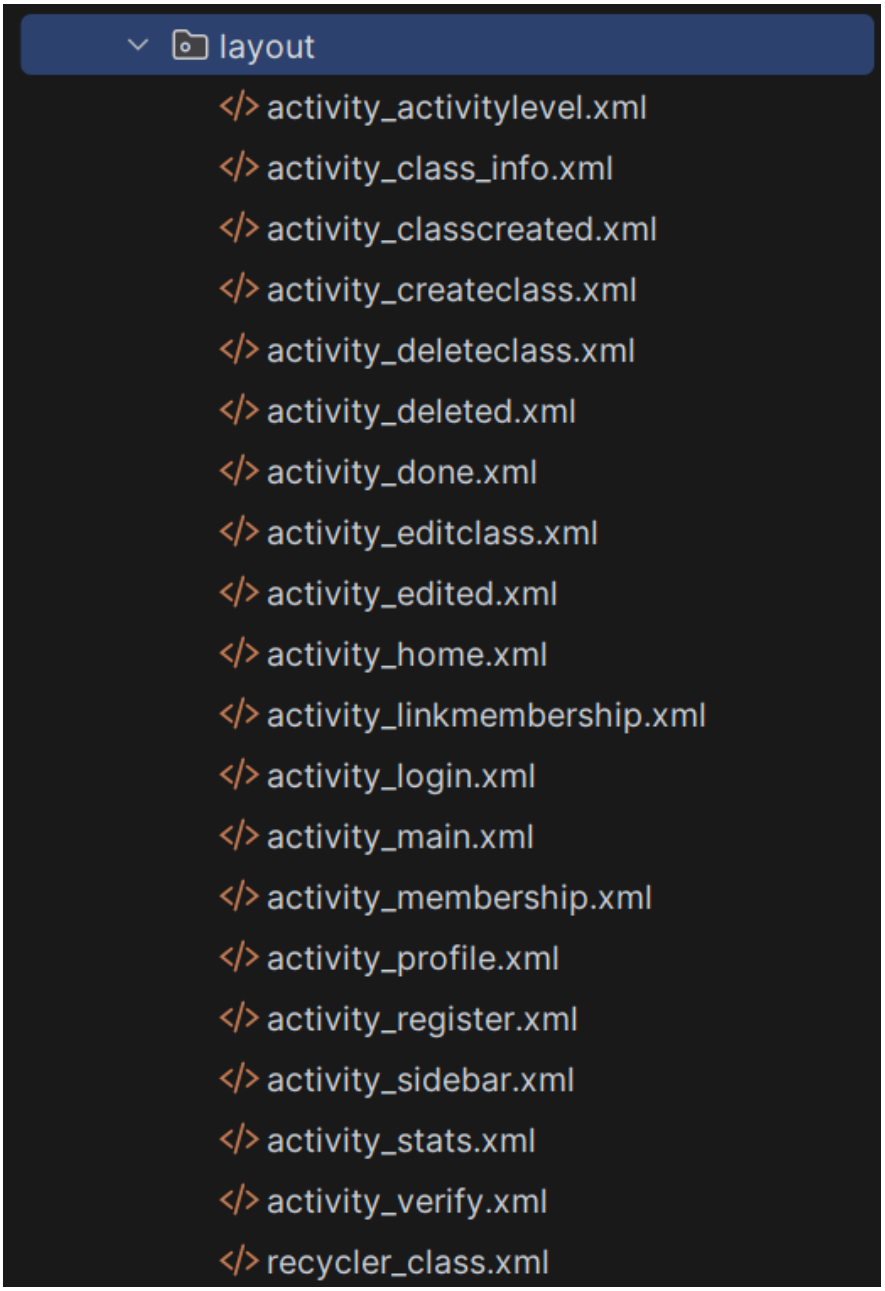


(. java File Structure)

To find assets used to make the UI, go to the “drawable” folder. To find .xml files that were used to make the structure of the UI, navigate to the “layout” folder. All .xml files follow the convection of activity_name. Furthermore, .xml files must be in lowercase as uppercase is not allowed. To find strings and arrays used to display the UI, go to “values” and locate “sstrings.xml”.



(Resource Folder - drawable)



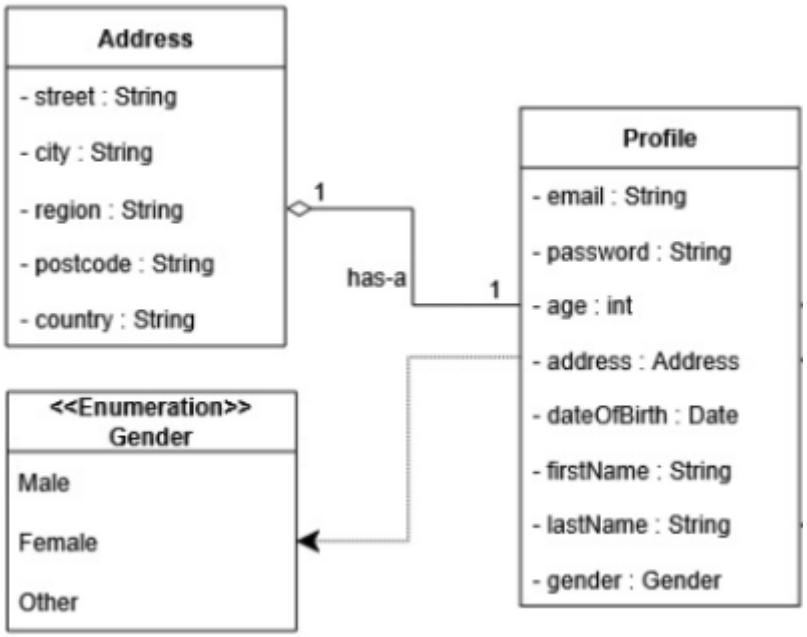
(Resource Folder - layout)

2.3.2 Modularity

2.3.2.1 Address.java

Address.java follows the builder pattern; therefore, it has a single responsibility: constructing an Address object. An address object contains the attributes: street, city, region, postcode, and country. These attributes are used to create an object of class “Profile”, since a profile has an address. Furthermore, a

snippet of the class diagram from section 1.3.5.1 is attached below to support this claim.



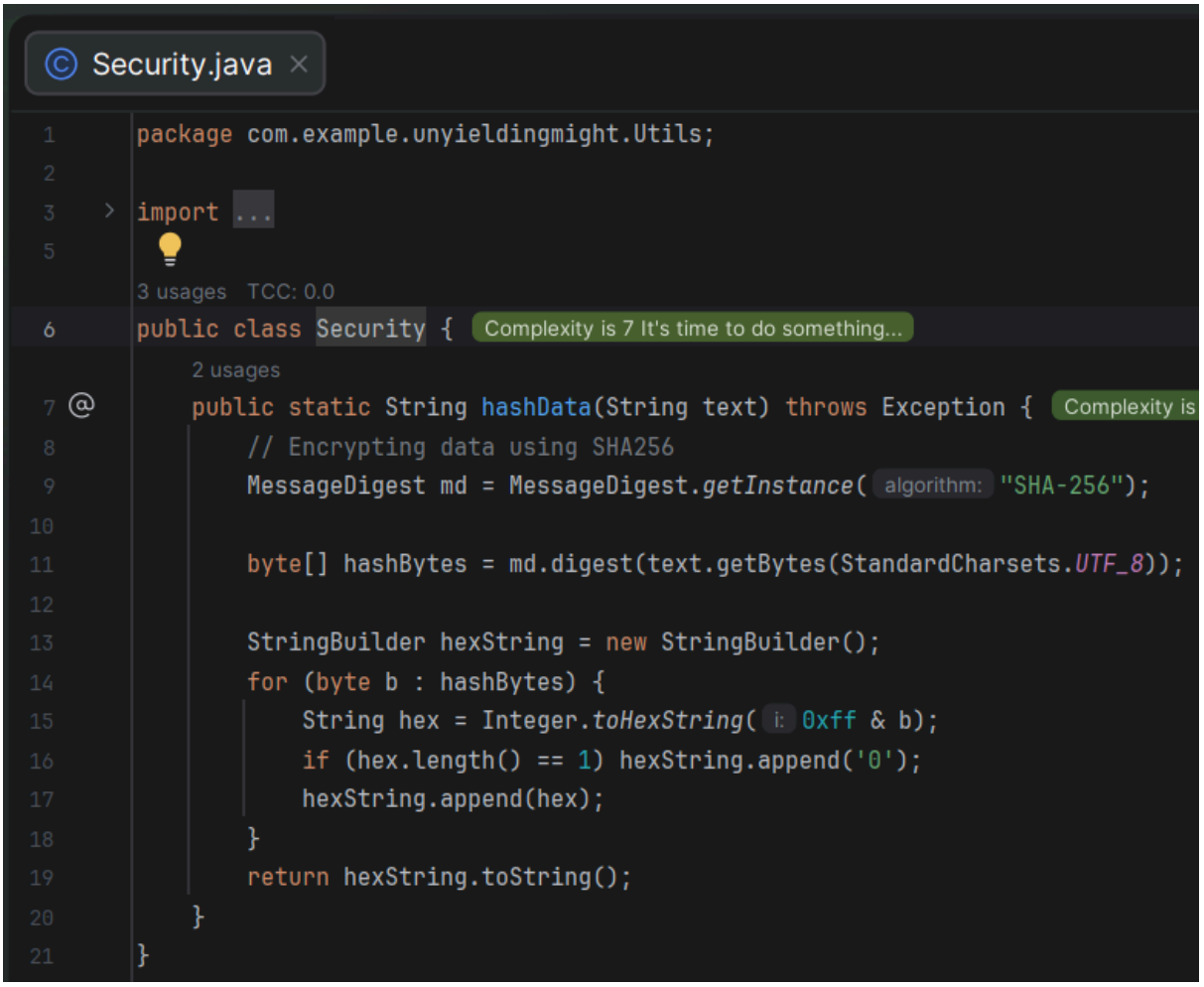
(Class Diagram - Address, Profile, and Gender)

2.3.2.2 Profile.java

Profile.java is akin to Address.java, since its sole responsibility is to build a Profile object. A profile contains the attributes, email, password, age, address (from class address), dateOfBirth, firstName, lastName, and gender (enum).

2.3.2.3 Security.java

Security.java is a utility with the sole responsibility of hashing the user's password when it is stored in the database. As indicated within the image below, Security.java has one method, hashData().



(Security.java)

2.3.3 Comments

Comments concerning key decisions can be found in the source files; however, for accessibility, some comments are included here:

Database.java

- // Connection to the database is always checked for each function (if conn == null)
- // Autocommit is also set to false when inserting a data in a table to prevent identity from incrementing
- // ResultSet is the result from the query executed

Security.java

- // Implement SHA-256 one-way cryptographic hashing to securely store user credentials and ensure password integrity.

Conversion.java

- // Conversion class converts the height or weight into its default unit that it is going to be stored in for the database

LoginActivity.java

- // Use of threading ensures that fetching database and sending email functionality causes no error occurs while waiting for the task that is running asynchronously

Adapter.java

- // Adapter for reusable card components for loading all gym classes

3.0 Software Quality Attributes Evaluation and Test Reports

3.1 Unit Testing

3.1.1 Customer Class Builder Success

GymClass.java CustomerTest.java

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
21  public class CustomerTest {
30      public void setUp(){
31          address = new Address.Builder()
32                  .street("19H Street")
33                  .city("Manukau")
34                  .region("Auckland")
35                  .postcode("123A")
36                  .country("New Zealand")
37                  .build();
38
39          profile = new Profile.Builder()
40                  .email("example@email.com")
41                  .password("password123")
42                  .age(20)
43                  .address(address)
44                  .dateOfBirth(date)
45                  .gender(Gender.MALE)
46                  .firstName("John")
47                  .lastName("Smith")
48                  .userClass("ADMIN")
49                  .build();
50
51          customer = new Customer.Builder()
52                  .customerId(1)
53                  .profile(profile)
54                  .isMember(true)
55                  .height(180f)
56                  .weight(75f)
57                  .activityMultiplier(ActivityMultiplier.EXTREME)
58                  .build();
```

```
61
62      @Test
63      public void CustomerBuilderSuccess(){
64          Assert.assertEquals( expected: 1, customer.getCustomerId());
65          Assert.assertEquals(profile, customer.getProfile());
66          Assert.assertTrue(customer.isMember());
67          Assert.assertEquals( expected: 180f, customer.getHeight(), delta: 180);
68          Assert.assertEquals( expected: 75f, customer.getWeight(), delta: 75);
69          Assert.assertEquals(ActivityMultiplier.EXTREME.getActivityMultiplier(), customer.getActivityMultiplier(), delta: 1.9);
70          Assert.assertEquals( expected: 3382f, customer.getTDEE(), delta: 3382);
71      }
72
```

All Results

Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

1	0	0	1.019s	100%
tests	failures	skipped	duration	successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
CustomerTest	com.example.unyieldingmight.CustomerTest	1	0	0	0.008s	100%

3.1.2 Customer Class Builder Fail

GymClass.java | CustomerTest.java

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
21  public class CustomerTest {
73      @Test
74      public void CustomerBuilderFail(){
75          try{
76              customer2 = new Customer.Builder()
77                      .build();
78          }
79          catch (Exception e){
80              assertEquals( expected: "Profile is needed", e.getMessage());
81          }
82      }
```

All Results

Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

1	0	0	1.106s	100%
tests	failures	skipped	duration	successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
CustomerTest	com.example.unyieldingmight.CustomerTest	1	0	0	0.008s	100%

3.1.3 Customer Class BMR Success

```
@Test
public void BMRSuccess(){
    Assert.assertEquals( expected: 1780f,
            customer.calculateBMR
                    (customer.getWeight(),
                    customer.getHeight(),
                    customer.getProfile().getAge(),
                    customer.getProfile().getGender()),  delta: 1780);
}
```

All Results

● Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

1	0	0	1.823s
tests	failures	skipped	duration

100% successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
CustomerTest	com.example.unyieldingmight.CustomerTest	1	0	0	0.009s	100%

3.1.4 Customer Class TDEE Success

```
    @Test
    public void TDEESuccess(){
        Assert.assertEquals( expected: 3382f,customer.calculateTDEE(), delta: 3382);
    }
}
```

All Results

● Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

1	0	0	1.816s
tests	failures	skipped	duration

100% successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
CustomerTest	com.example.unyieldingmight.CustomerTest	1	0	0	0.009s	100%

3.1.5 GymClass Class Builder Success

GymClassTest.java

```
15  public class GymClassTest {
20
21      @Before
22      public void setUp() {
23          trainer = new Trainer.Builder()
24                  .trainerID(1)
25                  .firstName("Dara")
26                  .lastName("Sanglap")
27                  .profileDescription("Testing trainer")
28                  .build();
29
30        gymClass = new GymClass.Builder()
31                .ID(1)
32                .name("HIIT")
33                .trainer(trainer)
34                .startDateTime(date)
35                .endDateTime(date)
36                .description("Testing gym class class")
37                .currentCapacity(12)
38                .maxCapacity(15)
39                .status(ClassStatus.ONGOING)
40                .avgCaloriesBurnedPerDay(1300)
41                .build();
42
43
44      }
```

```
45      @Test
46      public void GymClassBuilderSuccess(){
47          Assert.assertEquals( expected: 1, gymClass.getID());
48          Assert.assertEquals( expected: "HIIT", gymClass.getName());
49          Assert.assertEquals(trainer, gymClass.getTrainer());
50          Assert.assertEquals(date, gymClass.getStartDateTime());
51          Assert.assertEquals(date, gymClass.getEndDateTime());
52          Assert.assertEquals( expected: "Testing gym class class", gymClass.getDescription());
53          Assert.assertEquals( expected: 12, gymClass.getCurrentCapacity());
54          Assert.assertEquals( expected: 15, gymClass.getMaxCapacity());
55          Assert.assertEquals(ClassStatus.ONGOING, gymClass.getStatus());
56          Assert.assertEquals( expected: 1300, gymClass.getAvgCaloriesBurnedPerDay(),  delta: 0.0);
57
58      }
```

All Results

Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

1	0	0	1.572s
tests	failures	skipped	duration

100% successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
GymClassTest	com.example.unyieldingmight.GymClassTest	1	0	0	0.008s	100%

3.1.6 GymClass Class getIntensity Intermediate

```
//This is for the intermediate level testing
// ratio = 400 / 2000 = 0.20 → INTERMEDIATE
@Test
public void getIntensity_Intermediate(){
    GymClass intermediateClass = new GymClass.Builder()
            .ID(1)
            .name("HIIT")
            .trainer(trainer)
            .startDateTime(date)
            .endDateTime(date)
            .description("Testing gym class")
            .currentCapacity(12)
            .maxCapacity(15)
            .status(ClassStatus.ONGOING)
            .avgCaloriesBurnedPerDay(400f)
            .build();

    Assert.assertEquals(GymClass.Intensity.INTERMEDIATE, intermediateClass.getIntensity());
}
```

All Results

Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

1	0	0	1.527s
tests	failures	skipped	duration

100% successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
GymClassTest	com.example.unyieldingmight.GymClassTest	1	0	0	0.008s	100%

3.1.7 GymClass Class getIntensity Athlete

```java
//This test is already part of the first instance that was created in the setUp method
// ratio = 1300 / 2000 = 0.65 → ATHLETE
@Test
public void getIntensity_Athlete(){
    Assert.assertEquals(GymClass.Intensity.ATHLETE, gymClass.getIntensity( userTdee: 10));
}
```

All Results

Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

1	0	0	2.306s
tests	failures	skipped	duration

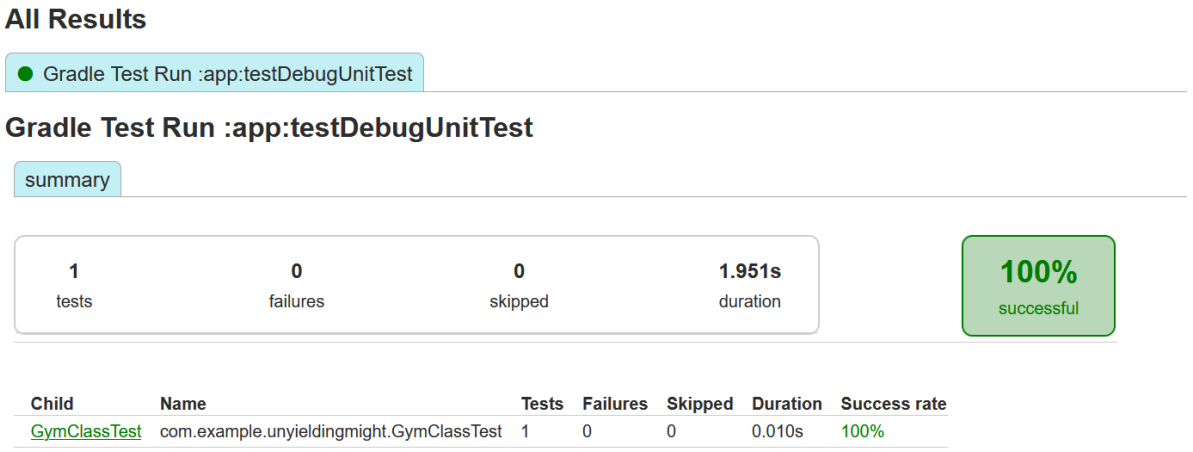
100% successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
GymClassTest	com.example.unyieldingmight.GymClassTest	1	0	0	0.010s	100%

3.1.8 GymClass Class getIntensity Beginner

```java
//This is for the beginner level testing
// ratio = 300 / 2000 = 0.15 → BEGINNER
@Test
public void getIntensity_Beginner(){
    GymClass beginnerClass = new GymClass.Builder()
            .ID(1)
            .name("HIIT")
            .trainer(trainer)
            .startDateTime(date)
            .endDateTime(date)
            .description("Testing gym class")
            .currentCapacity(12)
            .maxCapacity(15)
            .status(ClassStatus.ONGOING)
            .avgCaloriesBurnedPerDay(300f)
            .build();

    Assert.assertEquals(GymClass.Intensity.BEGINNER, beginnerClass.getIntensity());
}
```



All Results

Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

1	0	0	1.951s
tests	failures	skipped	duration

100% successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
GymClassTest	com.example.unyieldingmight.GymClassTest	1	0	0	0.010s	100%

3.1.9 Trainer Class Builder Success

```java
import org.junit.Assert;
import org.junit.Before;
import org.junit.Test;

public class TrainerTest {
    4 usages
    private Trainer trainer;

    @Before
    public void setUp() {
      trainer = new Trainer.Builder()
                .trainerID(1)
                .firstName("Dara")
                .lastName("Sanglap")
                .profileDescription("Testing trainer")
                .build();
    }

    @Test
    public void TrainerBuilderSuccess() {
        Assert.assertEquals( expected: 1, trainer.getTrainerID());
        Assert.assertEquals( expected: "Dara Sanglap", trainer.getName());
        Assert.assertEquals( expected: "Testing trainer", trainer.getProfileDescription());
    }
}
```

All Results

● Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

1	0	0	1.680s
tests	failures	skipped	duration

100% successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
TrainerTest	com.example.unyieldingmight.TrainerTest	1	0	0	0.012s	100%

3.1.10 User Class Test Success

```java
public class UserTest {
    6 usages
    public User user;

    @Before
    public void setup(){
        user = new User( id: 1, username: "Dara", password: "123", email: "example@unyiedingmight.com", User.Role.ADMIN);
    }

    //Constructor testing
    @Test
    public void UserTestSuccess(){
        Assert.assertEquals( expected: 1, user.getId());
        Assert.assertEquals( expected: "Dara", user.getUsername());
        Assert.assertEquals( expected: "123", user.getPassword());
        Assert.assertEquals( expected: "example@unyiedingmight.com", user.getEmail());
        Assert.assertEquals(User.Role.ADMIN, user.getUserClass());
    }
```

All Results

Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

1	0	0	1.306s
tests	failures	skipped	duration

100% successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
UserTest	com.example.unyieldingmight.UserTest	1	0	0	0.017s	100%

3.1.11 Database loginUser Success

```java
    11 usages
    public User justin;
    1 usage
    public User justin2;

    @Before
    public void setUp(){
//        Object justin is in a try block, since the method hashData throws an exception
//        Hashing is only one way; therefore, it cannot be unhashed.
        try{
            justin = new User( id: 1, username: "Justin Grumal", Security.hashData( text: "password123"), email: "grum7@manukaumail.com", User.Role.ADMIN );
            justin2 = new User( id: 2, username: "Justin Grumal", Security.hashData( text: "password123"), email: "grum7@manukaumail.com", User.Role.ADMIN );
        }
        catch (Exception ignored){

        }
    }

    @Test
    public void loginUserSuccess(){
        Assert.assertEquals(justin.getId(), loginUser( email: "grum7@manukaumail.com", password: "password123").getId());
        Assert.assertEquals(justin.getUsername(), loginUser( email: "grum7@manukaumail.com", password: "password123").getUsername());
        Assert.assertEquals(justin.getPassword(), loginUser( email: "grum7@manukaumail.com", password: "password123").getPassword());
        Assert.assertEquals(justin.getEmail(), loginUser( email: "grum7@manukaumail.com", password: "password123").getEmail());
        Assert.assertEquals(justin.getClass(), loginUser( email: "grum7@manukaumail.com", password: "password123").getClass());
    }
```

All Results

Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

1	0	0	1.271s
tests	failures	skipped	duration

100% successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
DatabaseTest	com.example.unyieldingmight.DatabaseTest	1	0	0	0.457s	100%

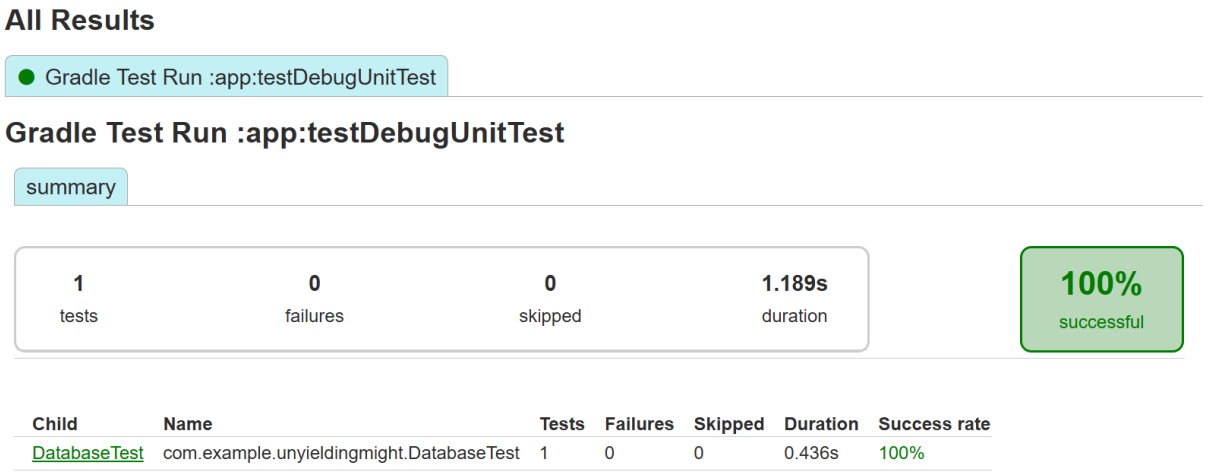
3.1.12 Database getUser Success

```
public class DatabaseTest {  Complexity is 3 Everything is cool!
    11 usages
    public User justin;
    1 usage
    public User justin2;

    @Before
    public void setUp(){
//        Object justin is in a try block, since the method hashData throws an exception
//        Hashing is only one way; therefore, it cannot be unhashed.
        try{
            justin = new User( id: 1,  username: "Justin Grumal", Security.hashData( text: "password123"),  email: "grum7@manukaumail.com", User.Role.ADMIN );
            justin2 = new User( id: 2,  username: "Justin Grumal", Security.hashData( text: "password123"),  email: "grum7@manukaumail.com", User.Role.ADMIN );
        }
        catch (Exception ignored){

        }
    }
```

```
@Test
public void getUserSuccess(){
    Assert.assertEquals(justin.getId(), getUser( email: "grum7@manukaumail.com",  password: "password123").getId());
    Assert.assertEquals(justin.getUsername(), getUser( email: "grum7@manukaumail.com",  password: "password123").getUsername());
    Assert.assertEquals(justin.getPassword(), getUser( email: "grum7@manukaumail.com",  password: "password123").getPassword());
    Assert.assertEquals(justin.getEmail(), getUser( email: "grum7@manukaumail.com",  password: "password123").getEmail());
    Assert.assertEquals(justin.getClass(), getUser( email: "grum7@manukaumail.com",  password: "password123").getClass());
}
```



All Results

Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

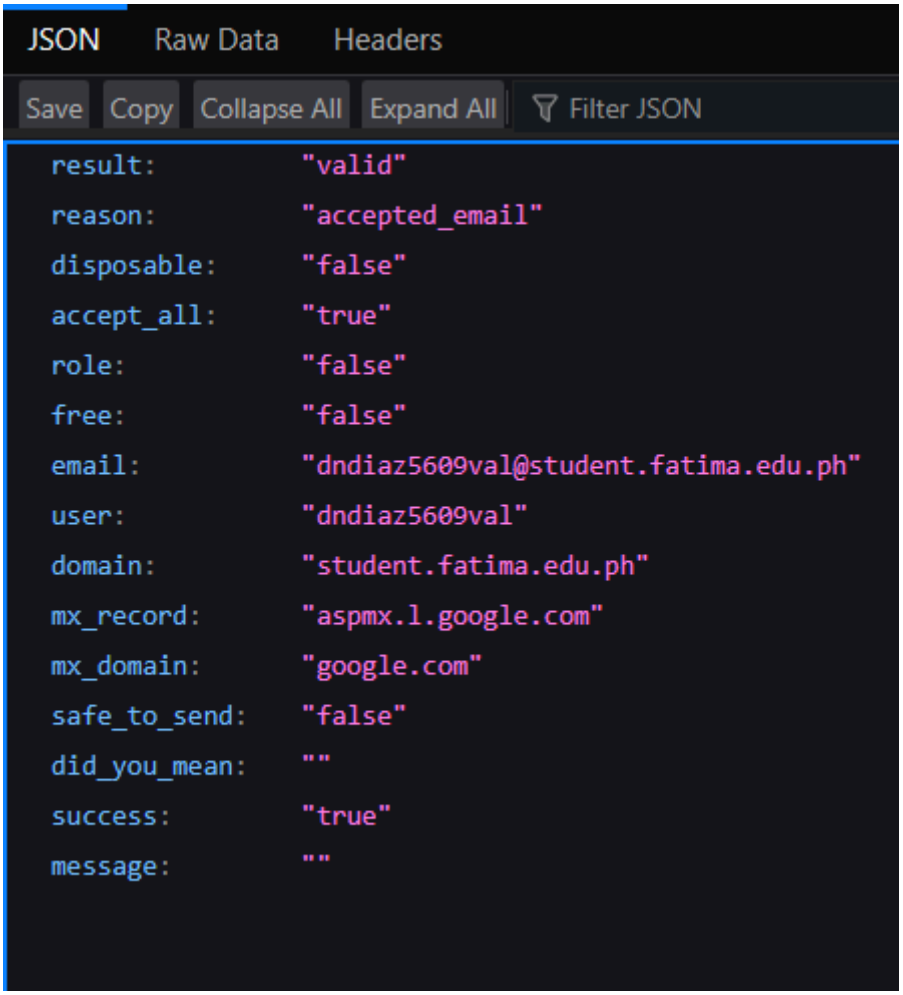
1	0	0	1.189s	100%
tests	failures	skipped	duration	successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
DatabaseTest	com.example.unyieldingmight.DatabaseTest	1	0	0	0.436s	100%

3.2 Integration Testing

3.2.1 Universal Email Verification

A bug that arose when all the model classes were used for doing the register functionality is when doing the email verification where there are some emails which are not safe to send even though it is an enterprise-owned email. This problem was found when the app was tested with a university account where the verification data returns safe to send for it as false. To fix this, the logic was changed from checking safe_to_send, result, and disposable to merging safe_to_send and result into one yet still checking the disposable factor.


JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
result: "valid"
reason: "accepted_email"
disposable: "false"
accept_all: "true"
role: "false"
free: "false"
email: "dndiaz5609val@student.fatima.edu.ph"
user: "dndiaz5609val"
domain: "student.fatima.edu.ph"
mx_record: "aspmx.l.google.com"
mx_domain: "google.com"
safe_to_send: "false"
did_you_mean: ""
success: "true"
message: ""




```
public static boolean validateEmail(String email) {  Complexity is 6 It's time to do so
    // Verify the email if it is
    EmailVerification ev = new EmailVerification().email(email).verify();
    EmailVerificationData evData = ev.getData();
    boolean isSafe = "true".equalsIgnoreCase(evData.safe_to_send());
    boolean isValid = "valid".equalsIgnoreCase(evData.result());
    boolean isDisposable = "true".equalsIgnoreCase(evData.disposable());

    // Check if email it is not safe, valid, or disposable
    if (!(isSafe || isValid) || isDisposable) {
        Log.w( tag: "DATABASE_ERROR",  msg: "Registration blocked by QEV for:
        return false;
    }
    return true;
}
```

3.2.2 Booking Class Limit

After all the components of the frontend and the backend were linked, The app did not get to implement a limit to booking a class for the current user; the user can just spam the book button and the database will just record it. In order to achieve

this, we tweaked the function of booking the class by adding a select query to see if the user is already booked in for the class in the GymClassBooking table.

	HistoryId	ProfileId	ClassId	BookingStatus	BookingDate
1	1	1	1	BOOKED	2026-05-30 18:17:36.663
2	2	7	1	BOOKED	2026-05-30 19:01:25.137
3	3	7	1	BOOKED	2026-05-30 19:01:30.777
4	4	7	1	BOOKED	2026-05-30 19:01:55.560
5	5	7	1	BOOKED	2026-05-30 19:02:03.240

Database record showing duplicates of booking

```
public static boolean bookClass(int classId) {  Complexity is 16 You must be kidding
    User user = getCurrentUser();
    Connection conn = getConnection();
    if (conn == null) return false;
    try {
        conn.setAutoCommit(false);

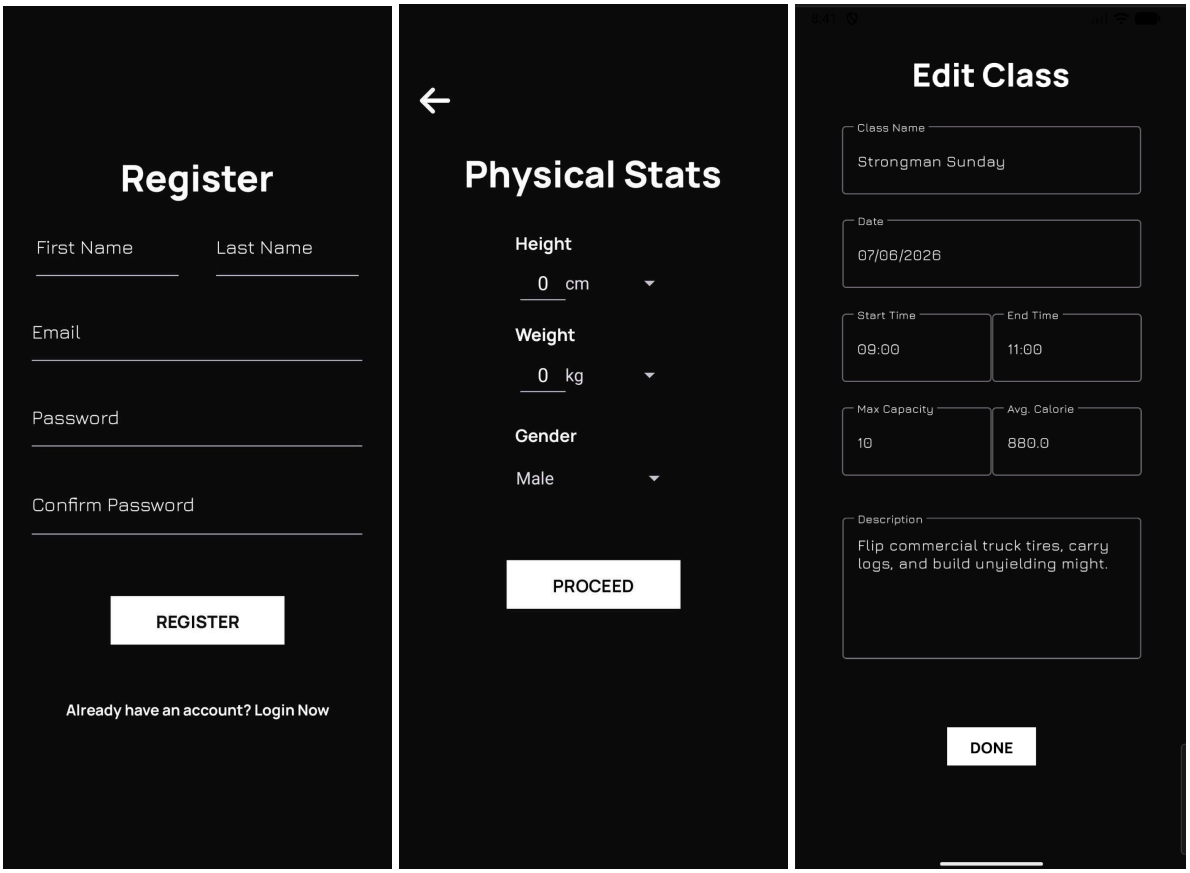
        // Check if already booked
        String checkBookedSql = "SELECT 1 FROM GymClassBooking WHERE ProfileId = ? AND ClassId = ? AND BookingStatus = 'BOOKED'";
        try (PreparedStatement checkPstmt = conn.prepareStatement(checkBookedSql)) {
            checkPstmt.setInt( i: 1, user.getId());
            checkPstmt.setInt( i: 2, classId);
            try (ResultSet rs = checkPstmt.executeQuery()) {
                if (rs.next()) {
                    conn.rollback();
                    return false;
                }
            }
        }
```

Code that is added to prevent duplicate booking by the users

3.2.3 Controller Polishment

The implementation of seamless input boxes and minimal validation is a must for a better user experience and reduces the inconvenience the user experience. The first prototype of the UI is working perfectly fine but it has its flaws such as the confirm password being not hidden, the OTP boxes were have to be manually clicked for each boxes, and some GUI were missing some inputs such as first name and last name for the registration activity, gender for stats, and avgburnedcaloriesperday for editing a class.

Final Controller Look:



Added “First Name” and “Last Name” inputs, Gender spinner, and Avg Calorie

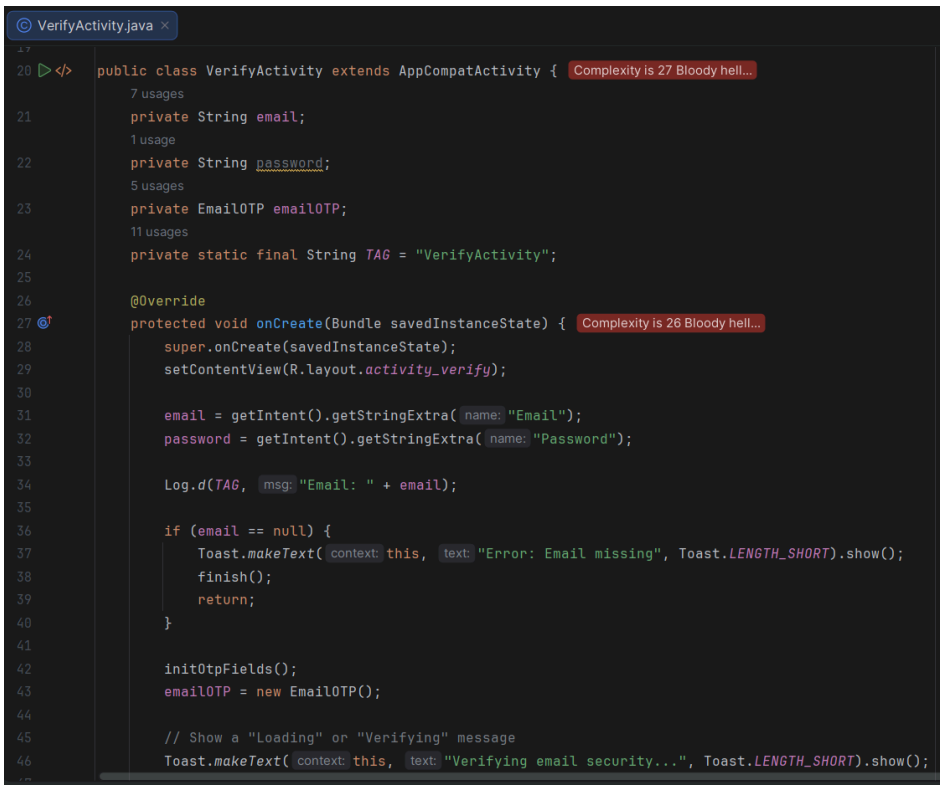
3.3 Software Quality Attribute

3.3.1 Maintainability Evaluation

By measuring the maintainability index for the Database indicates whether the backend and database is scalable and modifiable. To achieve this, we tested the database.java class by getting its method metrics maintainability and with the formula given below:

Maintainability Index Formula = 171 − 5.2 × ln(AvgV) − 0.23 × AvgV(G) − 16.2 × ln(AvgLOC)

Issue #1:



VerifyActivity.java

```java
public class VerifyActivity extends AppCompatActivity {  Complexity is 27 Bloody hell...
    7 usages
    private String email;
    1 usage
    private String password;
    5 usages
    private EmailOTP emailOTP;
    11 usages
    private static final String TAG = "VerifyActivity";

    @Override
    protected void onCreate(Bundle savedInstanceState) {  Complexity is 26 Bloody hell...
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_verify);

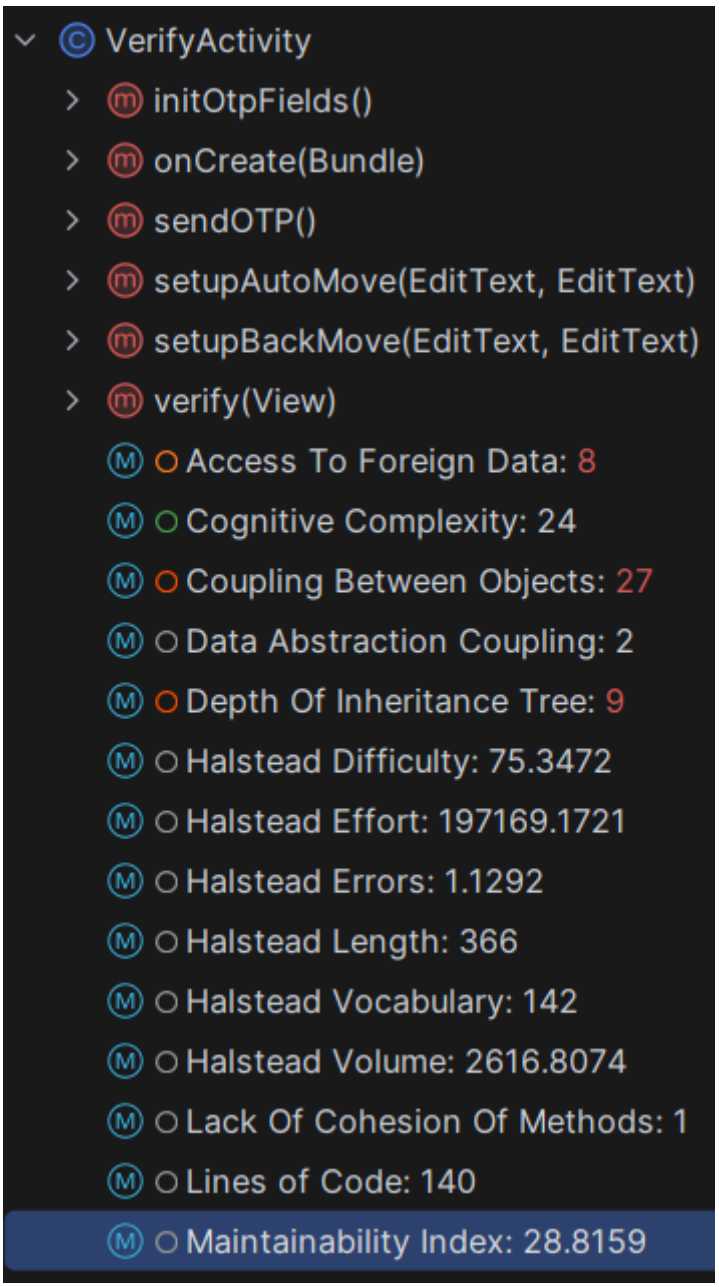
        email = getIntent().getStringExtra( name: "Email");
        password = getIntent().getStringExtra( name: "Password");

        Log.d(TAG,  msg: "Email: " + email);

        if (email == null) {
            Toast.makeText( context: this,  text: "Error: Email missing", Toast.LENGTH_SHORT).show();
            finish();
            return;
        }

        initOtpFields();
        emailOTP = new EmailOTP();

        // Show a "Loading" or "Verifying" message
        Toast.makeText( context: this,  text: "Verifying email security...", Toast.LENGTH_SHORT).show();
```



Maintainability Index = 28.8159 for VerifyActivity

Moderately maintainability of code

Issue #2:

Database.java

```
27  public class Database {  Complexity is 26 Bloody hell...
463     // Unused Functions
464
465     // For getting recommended gym class
        no usages
466 @   public static ArrayList<GymClass> getRecommendedGymClasses(Customer customer) {
467         ArrayList<GymClass> allOngoing = getGymClassesAvailable();
468         ArrayList<GymClass> recommended = new ArrayList<>();
469         float userTdee = customer.getTDEE();
470
471         // Check all gym classes that burns at least 12% of user's tdee
472         for (GymClass gc : allOngoing) {
473             float classBurn = gc.getAvgCaloriesBurnedPerDay();
474             if (classBurn >= (userTdee * 0.12f)) {
475                 recommended.add(gc);
476             }
477         }
478         return recommended;
479     }
480
        no usages
481     public static boolean updateClassStatus(int classId, ClassStatus status) {  Comple
482         String sql = "UPDATE GymClass SET ClassStatus = ? WHERE ClassId = ?";
483         Connection conn = getConnection();
484         if (conn == null) return false;
485         try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
486             pstmt.setString( i: 1, status.name());
487             pstmt.setInt( i: 2, classId);
488             int rows = pstmt.executeUpdate();
489             if (rows > 0) {
```

Method metrics

method	V	v(G)	LOC
com.example.unyieldingmight.Services.Database.bookClass(int)	782	9	49
com.example.unyieldingmight.Services.Database.Database()	2	1	3
com.example.unyieldingmight.Services.Database.editClass(int, String, Strir	335	3	19
com.example.unyieldingmight.Services.Database.getBookingHistory()	370	5	32
com.example.unyieldingmight.Services.Database.getConnection()	101	4	9
com.example.unyieldingmight.Services.Database.getCurrentUser()	11	1	1
com.example.unyieldingmight.Services.Database.getCustomer(String)	822	4	53
com.example.unyieldingmight.Services.Database.getGymClassesAvailable(	165	4	17
com.example.unyieldingmight.Services.Database.getInstance()	30	2	12
com.example.unyieldingmight.Services.Database.getNewsletterSubscribers	11	1	4
com.example.unyieldingmight.Services.Database.getRecommendedGymCla	99	3	14
com.example.unyieldingmight.Services.Database.getUser(String, String)	366	6	29
com.example.unyieldingmight.Services.Database.isAlreadyBooked(int)	222	4	17
com.example.unyieldingmight.Services.Database.loginUser(String, String)	50	2	8
com.example.unyieldingmight.Services.Database.mapResultSetToGymClas	404	3	24
com.example.unyieldingmight.Services.Database.reconnect()	90	2	9
com.example.unyieldingmight.Services.Database.registerCustomer(String,	1,448	9	68
com.example.unyieldingmight.Services.Database.updateClassStatus(int, Cl	278	5	19
com.example.unyieldingmight.Services.Database.validateEmail(String)	206	4	14
com.example.unyieldingmight.Services.Database.validateMembership(int, S	717	7	41
Total		**79**	**442**
Average	325.91	3.95	22.10

Maintainability Index = 89.85 with unused features

Database.java

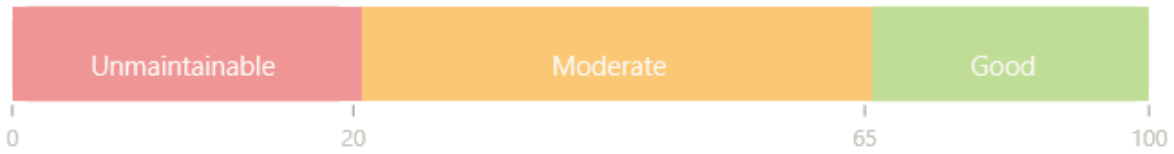
```
27  public class Database {  Complexity is 26 Bloody hell...
463 //    // Unused Functions
464 //
465 //    // For getting recommended gym class
466 //    public static ArrayList<GymClass> getRecommendedGymClasses(Customer customer) {
467 //        ArrayList<GymClass> allOngoing = getGymClassesAvailable();
468 //        ArrayList<GymClass> recommended = new ArrayList<>();
469 //        float userTdee = customer.getTDEE();
470 //
471 //        // Check all gym classes that burns at least 12% of user's tdee
472 //        for (GymClass gc : allOngoing) {
473 //            float classBurn = gc.getAvgCaloriesBurnedPerDay();
474 //            if (classBurn >= (userTdee * 0.12f)) {
475 //                recommended.add(gc);
476 //            }
477 //        }
478 //        return recommended;
479 //    }
480 //
481 //    public static boolean updateClassStatus(int classId, ClassStatus status) {
482 //        String sql = "UPDATE GymClass SET ClassStatus = ? WHERE ClassId = ?";
483 //        Connection conn = getConnection();
484 //        if (conn == null) return false;
485 //        try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
486 //            pstmt.setString(1, status.name());
487 //            pstmt.setInt(2, classId);
488 //            int rows = pstmt.executeUpdate();
489 //            if (rows > 0) {
490 //                // Sends email about the class being finished or cancelled.
491 //                NewsletterType type = (status == ClassStatus.COMPLETE) ? NewsletterType
```

Method metrics

method	V	v(G)	LOC
com.example.unyieldingmight.Services.Database.bookClass(int)	782	9	49
com.example.unyieldingmight.Services.Database.Database()	2	1	3
com.example.unyieldingmight.Services.Database.editClass(int, String, Strir	335	3	19
com.example.unyieldingmight.Services.Database.getConnection()	101	4	9
com.example.unyieldingmight.Services.Database.getCurrentUser()	11	1	1
com.example.unyieldingmight.Services.Database.getCustomer(String)	822	4	53
com.example.unyieldingmight.Services.Database.getGymClassesAvailable(	165	4	17
com.example.unyieldingmight.Services.Database.getInstance()	30	2	12
com.example.unyieldingmight.Services.Database.getNewsletterSubscribers	11	1	4
com.example.unyieldingmight.Services.Database.getUser(String, String)	366	6	29
com.example.unyieldingmight.Services.Database.isAlreadyBooked(int)	222	4	17
com.example.unyieldingmight.Services.Database.loginUser(String, String)	50	2	8
com.example.unyieldingmight.Services.Database.mapResultSetToGymClas	404	3	24
com.example.unyieldingmight.Services.Database.reconnect()	90	2	9
com.example.unyieldingmight.Services.Database.registerCustomer(String,	1,448	9	68
com.example.unyieldingmight.Services.Database.validateEmail(String)	206	4	14
com.example.unyieldingmight.Services.Database.validateMembership(int,	717	7	41
Total		**66**	**377**
Average	339.39	3.88	22.18

Maintainability Index = 89.60 without unused features

Commenting the unused features slightly increased the file's average line of code to 22.18 and Halstead Volume to 339.39 which lowered the score by 0.25 points of Maintainability Index. The decision for the final build of the application to go with the commented unused features in exchange for a lower maintainability index is to prioritise a clean and readable code with less code execution and that the functions might be later implemented later on.



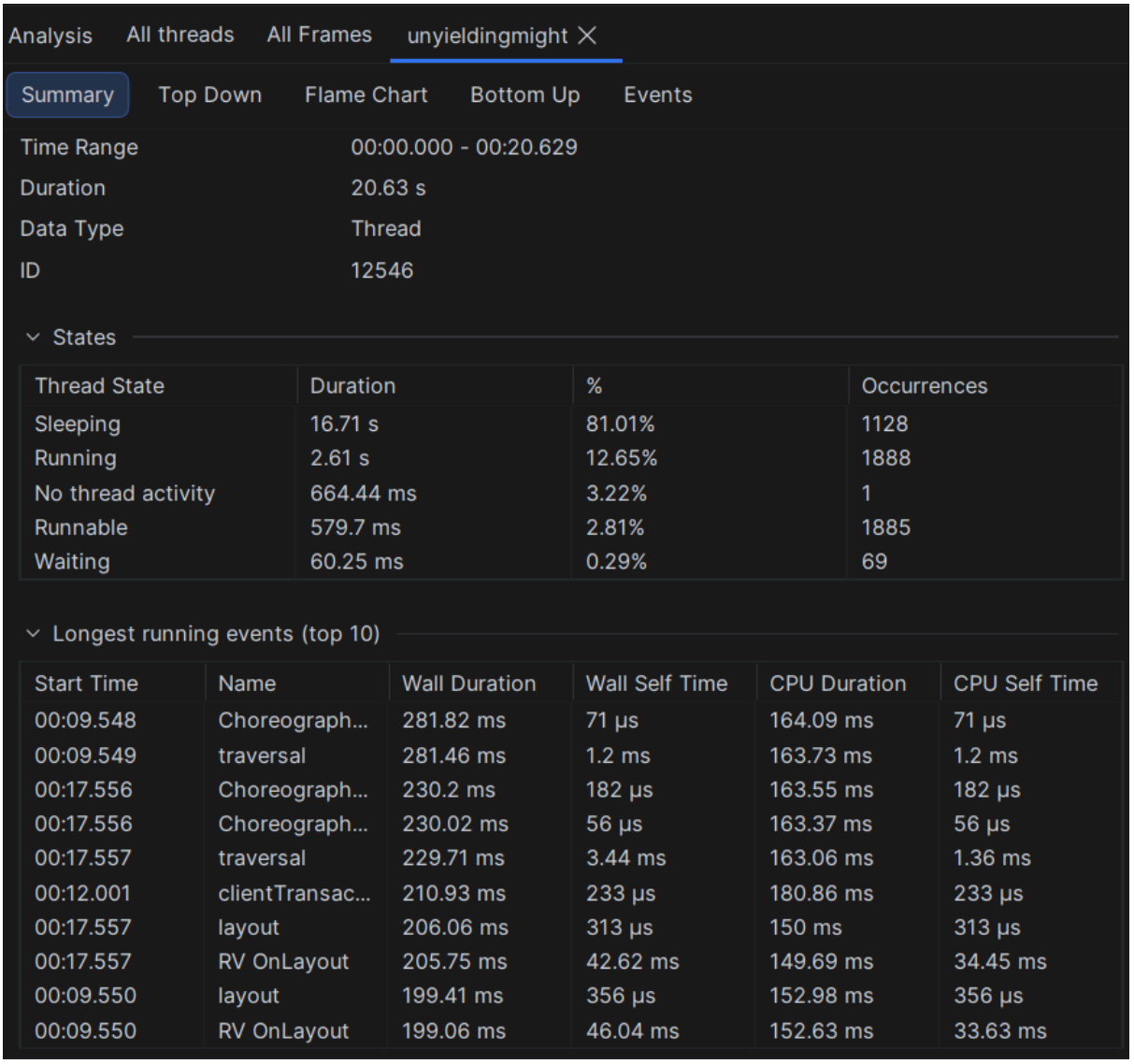
Maintainability Index Score

Index Value	Color	Meaning
0-20	Red	Low maintainability of code
20-65	Yellow	Moderate maintainability of code
65-100	Green	Good maintainability of code

(mikejo5000, 2025)

3.3.2 Performance Evaluation

Using the Profiler within Android Studio's built-in tools, performance of the app can be tested and evaluated by recording the system trace on a physical device it is running on to get the analysis of factors that can affect the runtime when using the app such as resource usage, UI rendering, and core threads and states (Android Developers, 2026). The system tracing was executed for the task of an admin logging in up until the class name has been edited where it finished with a time of 20.63s.



Analysis | All threads | All Frames | unyieldingmight

Summary | Top Down | Flame Chart | Bottom Up | Events

Time Range	00:00.000 - 00:20.629
Duration	20.63 s
Data Type	Thread
ID	12546

States

Thread State	Duration	%	Occurrences
Sleeping	16.71 s	81.01%	1128
Running	2.61 s	12.65%	1888
No thread activity	664.44 ms	3.22%	1
Runnable	579.7 ms	2.81%	1885
Waiting	60.25 ms	0.29%	69

Longest running events (top 10)

Start Time	Name	Wall Duration	Wall Self Time	CPU Duration	CPU Self Time
00:09.548	Choreograph...	281.82 ms	71 µs	164.09 ms	71 µs
00:09.549	traversal	281.46 ms	1.2 ms	163.73 ms	1.2 ms
00:17.556	Choreograph...	230.2 ms	182 µs	163.55 ms	182 µs
00:17.556	Choreograph...	230.02 ms	56 µs	163.37 ms	56 µs
00:17.557	traversal	229.71 ms	3.44 ms	163.06 ms	1.36 ms
00:12.001	clientTransac...	210.93 ms	233 µs	180.86 ms	233 µs
00:17.557	layout	206.06 ms	313 µs	150 ms	313 µs
00:17.557	RV OnLayout	205.75 ms	42.62 ms	149.69 ms	34.45 ms
00:09.550	layout	199.41 ms	356 µs	152.98 ms	356 µs
00:09.550	RV OnLayout	199.06 ms	46.04 ms	152.63 ms	33.63 ms

System Trace from Android Studio's Profiler

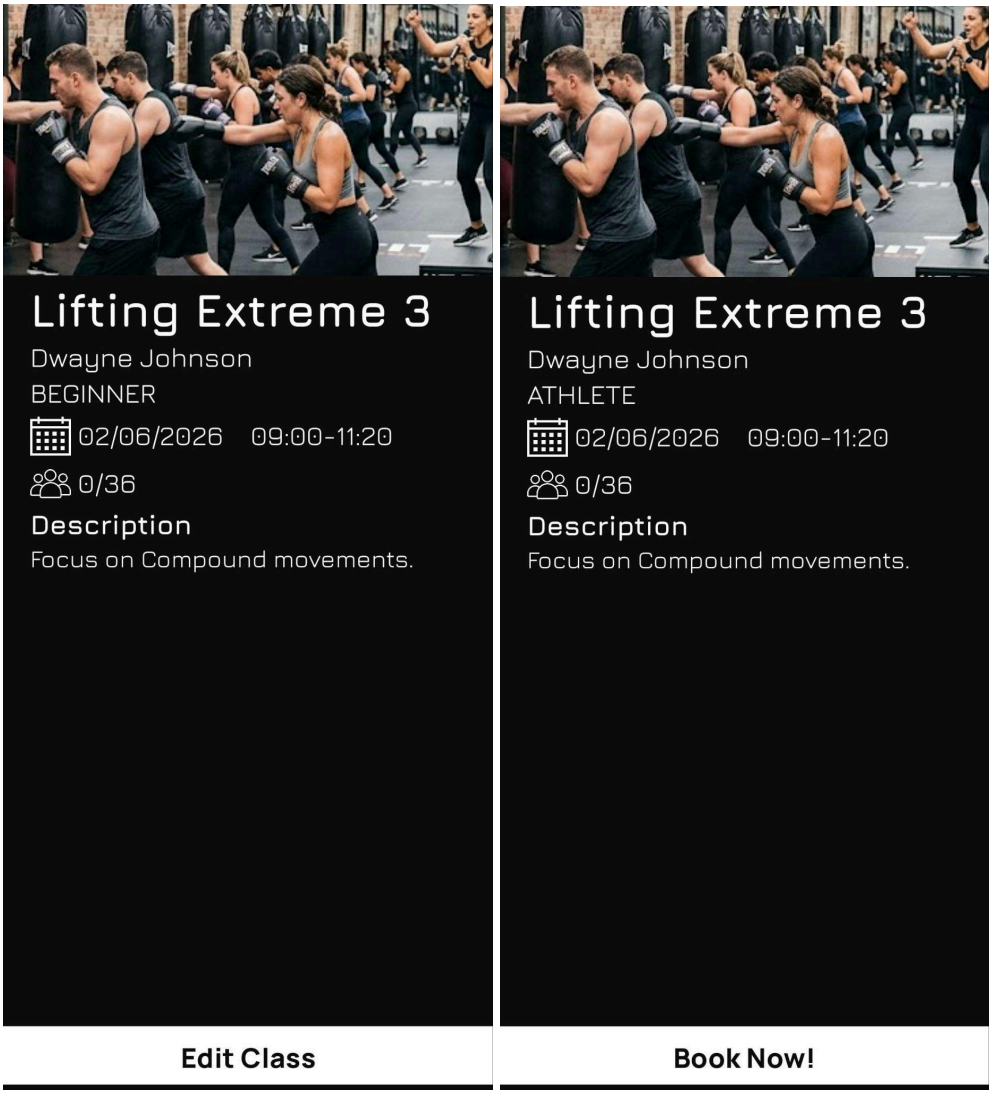
By looking at the stats in the charts above, the primary thread has processed 12.65% of the 20 second time that it took for the task to be finished; meaning that the CPU would not be overwhelmed by the app as it only took almost 1/8 of the processing time to finish the task.

As for the UI latency and responsiveness, it showed that the majority of the UI views are due to Choreographer and RecyclerView (RV) layouts which are responsible for dynamic output of data and constant rendering for all GUIs.

Overall, the data from the system trace indicates that the backend achieves a swift record with a minimal usage of the processing power of the device it was tested on.

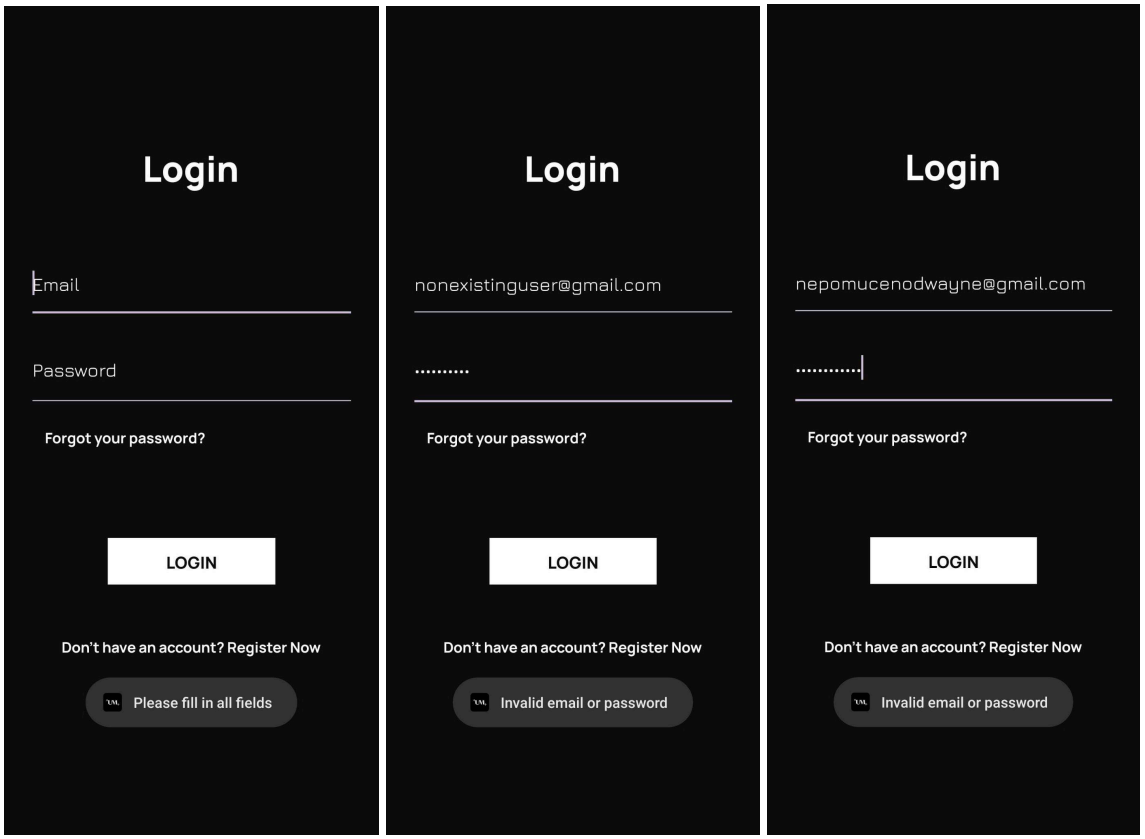
3.3.3 Security Evaluation

When logging in, the function, loginUser(), is executed to check the role of the user. Differentiating the role of the user prohibits unauthorised access by the customer; thereby, they are not able to edit the class details.



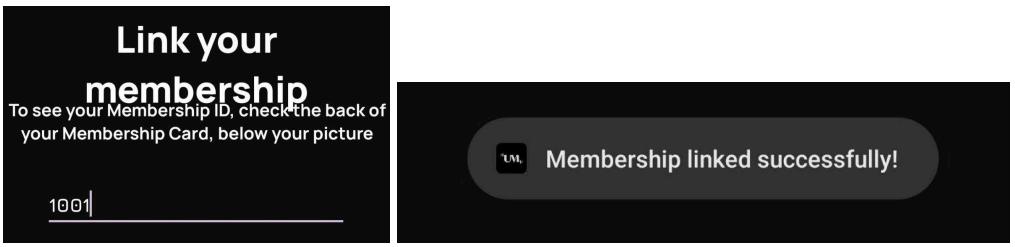
Admin View | Customer View

During email input validation within the login activity, checking in the database for invalid email is implemented as multiple checks are made to identify whether a field is empty, email is not found, or when the password to the email is invalid. Refer to the images below.

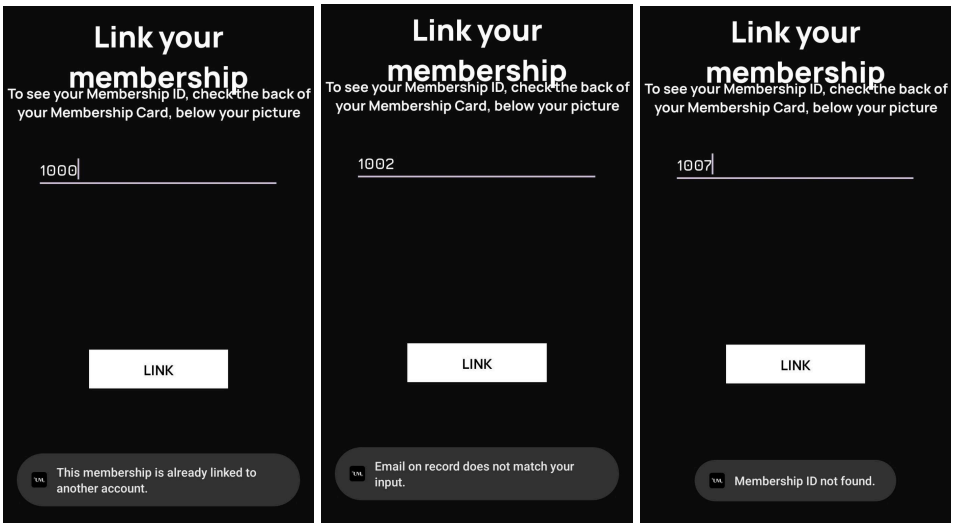


Empty Fields | *Non-existent User* | *Invalid Password*

For preventing mislinking a membership, safeguard rules were implemented for the scenarios where a membership ID is already linked to an account, the email on the membership does not match the registering user, or when the ID is not found in the database.



Valid Membership

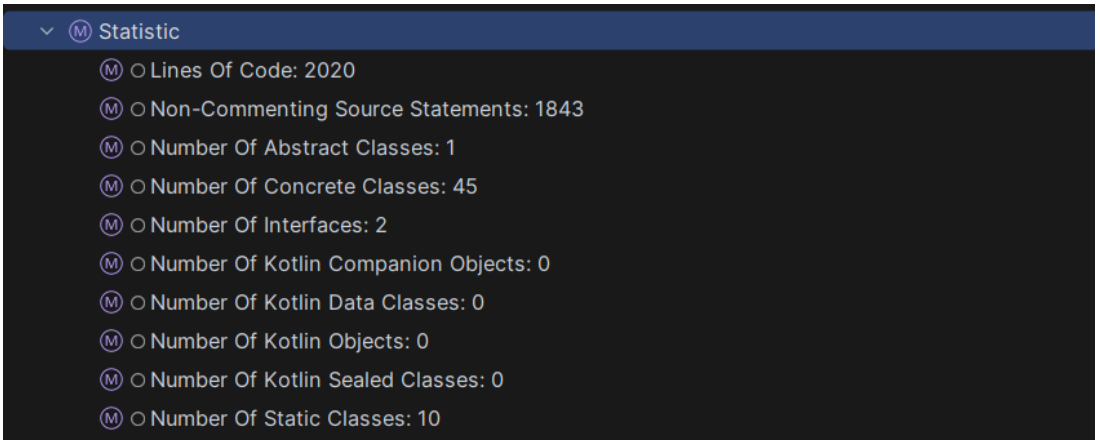


Linking a linked ID | Linking an ID with mismatched email | Linking a non-existent ID

4.0 Software Metrics Collection and Analysis

4.1 Lines of Codes

Lines of code is a count of code volume measuring the size of a software system (Timbó, 2026). It calculates the declaration of any variables, executable and on executable statements. In UM's software the overall line of codes are 2,020 and No Comment Line of Code is 177.



(Metric Tree plugin in Android Studio)

4.1.1 Analysis and Interpretation

LOC - Line of Code

NCLOC - No Comment Line of Code

Risks:

1. A low comment count directly impacts code readability. This makes it difficult for future developers to grasp the underlying business logic quickly, slowing down maintenance and increasing the risk of accidental bugs during updates.
2. Exceeding 2,000 lines of code in the early stages points directly to the complex feature set required for this mobile app. As these core functionalities are integrated, tracking the code volume helps ensure the system architecture stays clean and organised.

Recommendations:

1. Adding clear, intentional comments to every function is crucial for the long-term maintenance of the project. Documenting the underlying logic and execution rules provides essential context that raw code cannot convey on its own. This practice ensures that future team members can confidently read, understand, and update the codebase without the risk of introducing accidental bugs or spending excessive time reverse-engineering the architecture.
2. Evaluating the quality and purpose of the code is essential for balancing runtime efficiency with long-term maintainability. This process ensures functions are cleanly implemented and aligned with the app's core features. Prioritising this assessment during early development prevents unnecessary complexity and rigid coupling, keeping the codebase efficient at runtime and straightforward to debug, test, and scale.

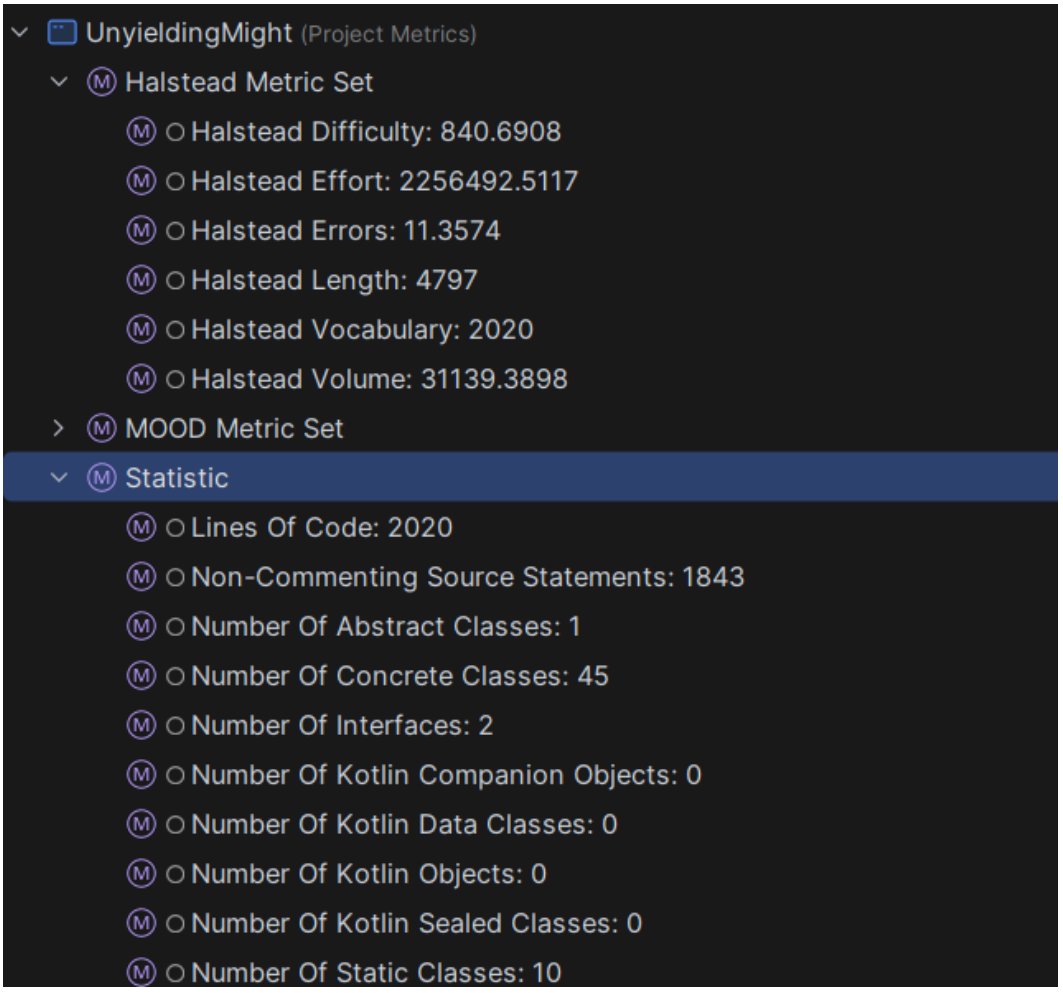
Maintaining a good balance between lines of code and comments shows that the project is built with teamwork in mind. Tracking this ratio makes it easier to see if a codebase is straightforward to read and understand, which helps new developers onboard and contribute quickly without facing a steep learning curve.

Hitting over 2,000 lines of code early in a project is not automatically bad, but it does add complexity. As the project grows, it becomes naturally harder to keep files readable, maintainable, and easy to test. If the underlying code is not implemented cleanly, such as, relying on heavy processing or tight coupling performance will quickly take a hit. To keep this on track, the focus needs to be on keeping the architecture modular, sticking to clean coding standards, and running tests early to catch logic bottlenecks.

4.2 Maintainability Index

A composite metric that indicates the maintainability of a software system based on several factors such as cyclomatic complexity, comment density and line of codes (Timbó, 2026). Maintainability index can be calculated using this formula MI = MAX(0, (171 - 5.2 * ln(Halstead Volume) - 0.23 * (Cyclomatic Complexity) - 16.2 * ln(Lines of Code)) * 100 / 171) which is originally designed to be applied per module. However, for calculating the whole project we will be using the average numbers because in MetricReloaded plugin only offers the average overall Cyclomatic Complexity for the whole project, we will apply this formula to calculate the MI of the overall project:

Maintainability Index = 171 − 5.2 × ln(AvgV) − 0.23 × AvgV(G) − 16.2 × ln(AvgLOC)



(Metric Tree in Android Studio plugin)

Total Halstead Volume = 31,139.38

Total LOC = 2,020

To get the average:

Total classes = 45

31,139.38 / 45 = 691.99

2020 / 45 = 44.89

AvgV (Average Halstead Volume) = 691.99

AvgV(G) (Average Cyclomatic Complexity) = 1.58 (Shown in the next screenshot)

AvgLOC (Average Lines of Code) = 44.89

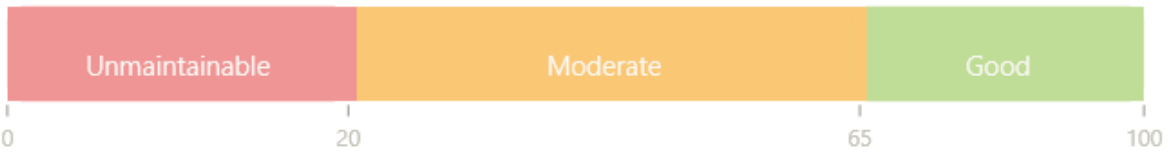
ln(691.99)≈6.5396

ln(44.89)≈3.8042

MI=171−(5.2×6.5396)−(0.23×1.58)−(16.2×3.8042)

MI=171−34.006−0.363−61.628

MI≈75.00



Maintainability Index Score

Index Value	Color	Meaning
0-20	Red	Low maintainability of code
20-65	Yellow	Moderate maintainability of code
65-100	Green	Good maintainability of code

(mikejo5000, 2025)

4.2.1 Analysis and Interpretation

Risks:

1. Total Halstead Volume of 31,139.38 indicates a high information density. Meaning developers are writing dense, heavy logic that increases the estimated Halstead Errors, making the project highly vulnerable to logical bugs that are easy to overlook during code reviews.

2. With 45 concrete classes, only 1 abstract class, and 2 interfaces, the codebase suffers from a severe lack of abstraction. This creates a major architectural risk: components are tightly coupled to specific implementations rather than flexible interfaces. As the project expands, this rigidity will make it incredibly difficult to mock dependencies for automated testing or swap out features, ultimately stalling scalability.

Recommendations:

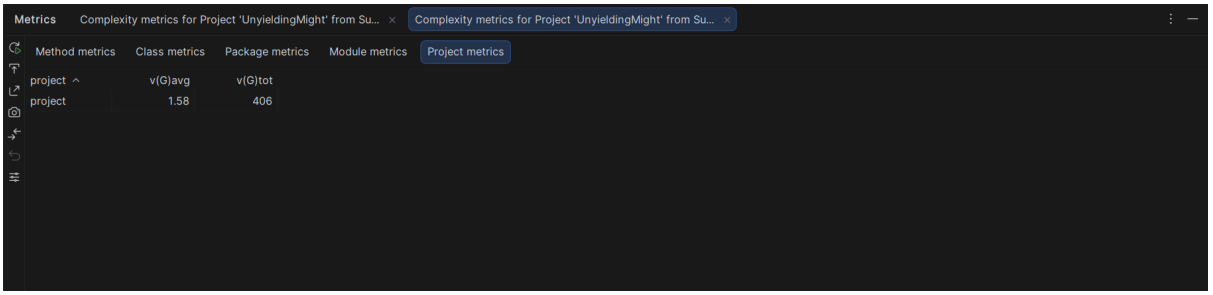
1. To lower the Halstead Volume, developers should reduce dense lines of code into smaller, single-purpose helper functions. Spreading the code will make it easier to read and turn confusing code into self-explanatory steps, making it simple for the team to spot mistakes during code review.
2. To fix this problem, we should have used interface-driven design for the core part of the project, such as database handling, instead of linking modules directly to concrete code. This simple shift can easily swap out features without breaking the rest of the application.

Maintainability Index supports the maintainability in this project by showcasing a score that will serve as a baseline to check if there is any repetitiveness in the code. For UM's project, it shows that our MI score is 75.00, which means the maintainability of the code is good, well structured and reasonably balanced.

This metric improves the system by developers tracking the MI consistently to see if there is any early-warning system, making sure that the index score does not drop below 65.00. Implementing this can alert the teams exactly when they need to step back and halt the development and refactor.

4.3 Cyclomatic Complexity

Cyclomatic Complexity (CC) is a software metric that measures the number of independent paths through a piece of code (GeeksforGeeks, 2018). For UM's project metrics CC is 1.58 v(G)avg and 406 v(G)tot. We used the MetricReloaded plugin to check the CC of the project because the MetricTree plugin does not have calculations of the CC.



(Metric Reloaded plugin in Android Studio)

Cyclomatic Complexity	Risk Level	What It Means
1-10	Low	Simple, well-structured code. Easy to test and maintain.
11-20	Moderate	More complex code. Requires careful testing and should be reviewed for refactoring opportunities.
21-50	High	Difficult to test thoroughly. Likely a candidate for decomposition.
50+	Very High	Very difficult to test thoroughly. Carries elevated defect risk. Review for redesign or decomposition.

(Tanner, 2025)

4.3.1 Analysis and Interpretation

Risks:

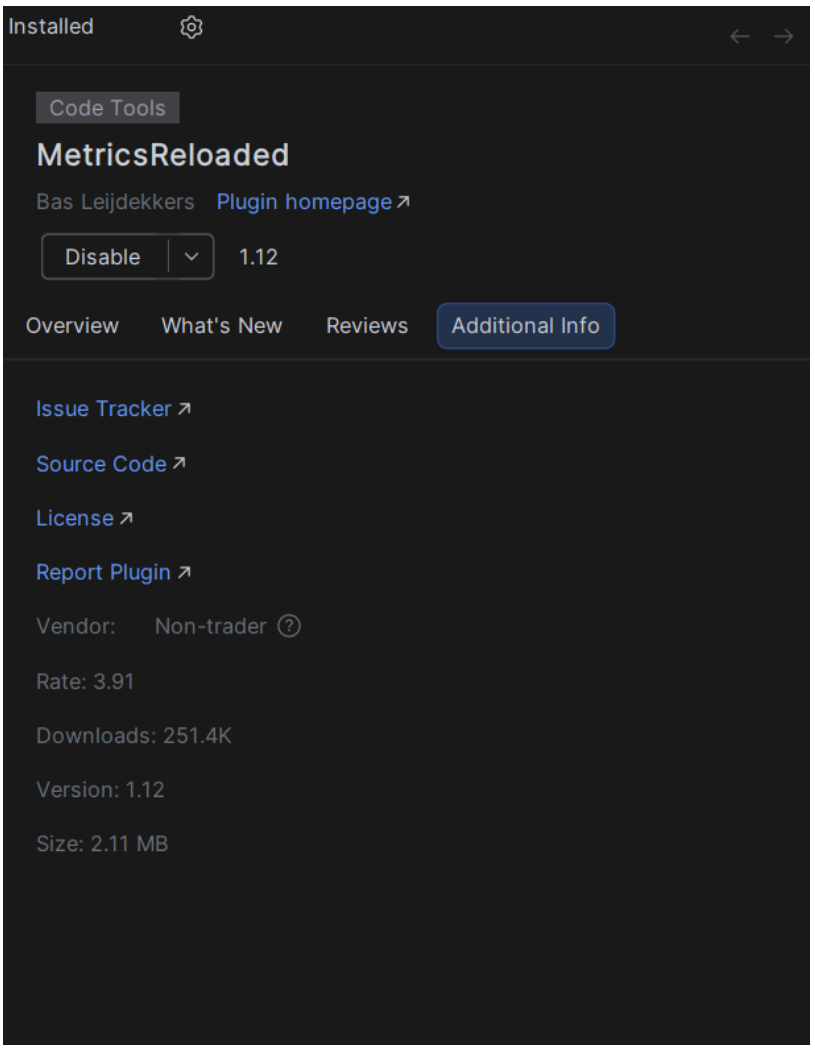
1. The high total cyclomatic complexity of 406 highlights an expanding codebase with an increased risk of software bugs. This accumulation of independent execution paths increases the cognitive load required to safely modify the project, making it highly prone to regression bugs where a change in one area unintentionally disrupts logic elsewhere in the system.
2. High total cyclomatic complexity could also affect integration testing, as tracking data flow and logic pathways across modules becomes significantly harder. Managing this complexity early is essential to prevent integration obstruction as new features are introduced.

Recommendations:

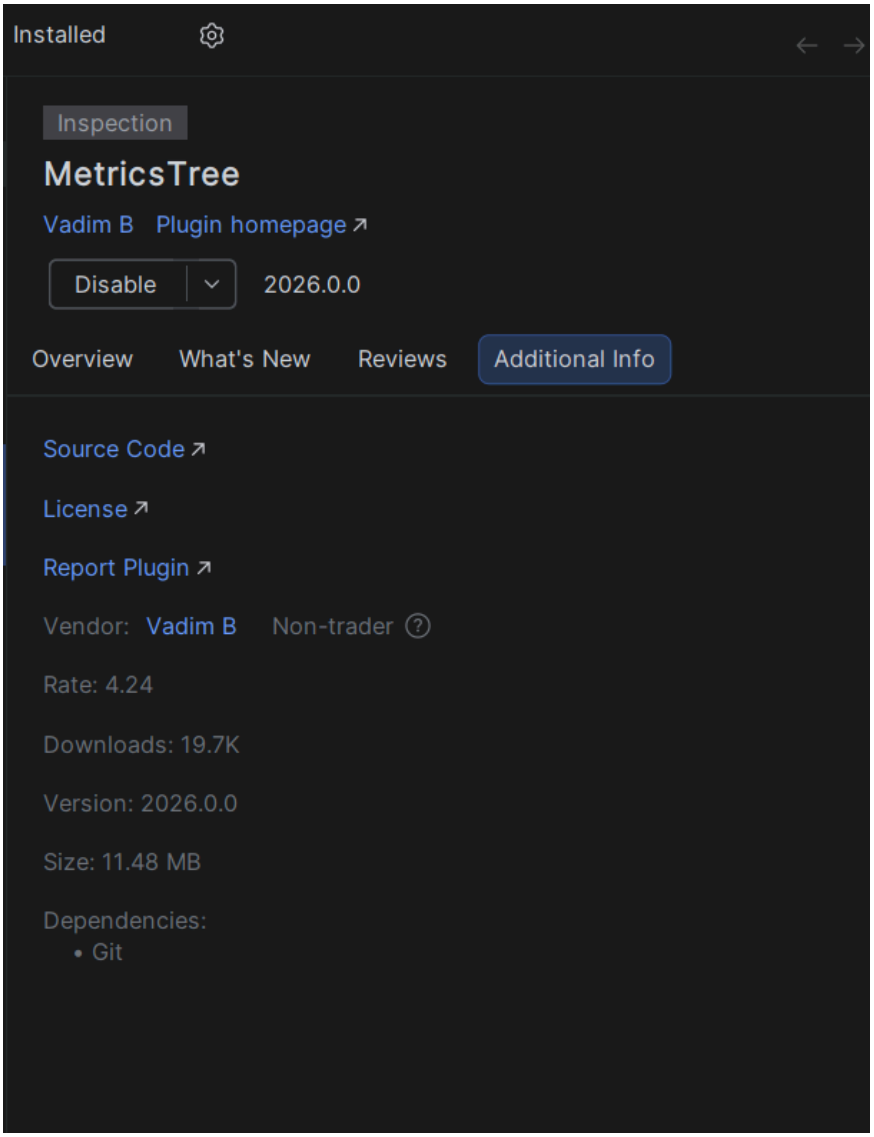
1. To reduce the risk of bugs and high cognitive load caused by the total cyclomatic complexity score, by actively refactoring the application's components and modules.
2. Implementing automated integration testing helps the modules interact with each other. This will simplify complex sequential logic before merging and will keep the testing manageable.

Cyclomatic Complexity supports the maintainability of independent pathways, such as if statements or loops inside a method. Based on our v(G)avg score of 1.58, it proves that our project's code is sufficiently structured and maintainable.

These metrics help improve performance and maintainability as an actionable strategy when developing the application. For maintainability, the low complexity score indicates the exact number of test cases that need to be covered, which allows the developers to write tests without repetitive paths. For performance, the cyclomatic metric improves the hardware efficiency by managing dense and deeply nested conditional branches. This prevents wasting branch mispredictions, which reduces the computational lag and keeps the application fast and responsive.



(MetricReloaded plugin in android studio used for calculating the cyclomatic complexity)



(MetricsTree plugin in android studio used for calculating the maintainability index and lines of code)

Bibliography

Android Developers. (2026, March 6). *Record a system trace*. https://developer.android.com/studio/profile/cpu-profiler

Android Developers. (2024, January 3). Processes and threads overview. https://developer.android.com/guide/components/processes-and-threads

Android Developers. (2026, May 28). Create dynamic lists with RecyclerView. https://developer.android.com/develop/ui/views/layout/recyclerview

Cernera, G. (2022, April 22). *Generating a message digest with Java, explained step-by-step with pictures*. Medium. https://gregorycernera.medium.com/generating-a-message-digest-with-java-explained-step-by-step-with-pictures-b9be6c6e5036

Deccon Tech. (2024, March 3). MySQL Server Android Studio JDBC connection. | *java [Video]. YouTube. www.youtube.com/watch?v=mbHvLSwhdLA*

Drumond, C. (n.d.). *What is scrum? Breaking down the Agile framework.* Atlassian. https://www.atlassian.com/agile/scrum

GeeksforGeeks. (2026, May 7). *Observer Design Pattern*. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/observer-pattern-set-1-introduction/

GeeksforGeeks. (2026, May 5). *Singleton Method Design Pattern*. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/singleton-design-pattern/

GeeksforGeeks. (2026, May 6). *Adapter Design Pattern*. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/adapter-pattern/

GeeksforGeeks. (2026, May 18). *MVC Design Pattern*. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/mvc-design-pattern/

GeeksforGeeks. (2025, September 27). *Cyclomatic Complexity*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/cyclomatic-complexity/

GeeksforGeeks. (2026, February 10). *ClientServer Model*. GeeksforGeeks. https://www.geeksforgeeks.org/system-design/client-server-model/

Jakarta EE. (n.d.). *Jakarta mail API documentation*. https://jakarta.ee/specifications/mail/1.6/apidocs/index.html

mikejo5000. (2025, January 9). *visualstudio-docs/docs/code-quality/code-metrics-maintainability-index-range-and-meaning.md at main · MicrosoftDocs/visualstudio-docs*. GitHub. https://github.com/MicrosoftDocs/visualstudio-docs/blob/main/docs/code-quality/code-metrics-maintainability-index-range-and-meaning.md

PreparedStatement (Java Platform SE 8). (n.d.). Docs.oracle.com. https://docs.oracle.com/javase/8/docs/api/java/sql/PreparedStatement.html

Refactoring Guru. (n.d.). *Singleton*. Refactoring.guru. https://refactoring.guru/design-patterns/singleton

Refactoring Guru. (n.d.). *Builder*. Refactoring.guru. https://refactoring.guru/design-patterns/builder

Refactoring Guru. (n.d.). *Observer*. Refactoring.guru. https://refactoring.guru/design-patterns/observer

Tanner, M. (2025, November 12). *Cyclomatic complexity: What it is and how to reduce it*. Sourcegraph. https://sourcegraph.com/blog/cyclomatic-complexity-what-it-is-and-how-to-reduce-it

Timbó, R. (2026, May 13). *What is Code Complexity and How Is It Measured?* https://www.revelo.com/blog/code-complexity

Easy Tuto. (2021, July 28). *RecyclerView Android Studio Tutorial | 2024* [Video]. YouTube. https://www.youtube.com/watch?v=TAEbP_ccjsk&list=LL&index=4&t=556s